

1.6.3 운영자매뉴얼(CICD)

- 개정이력

1. 개요

- 1.1. 목적
- 1.2. 대상

2. 구성 관리

- 2.1. 시스템 구성
 - 2.1.1. 시스템 구성도
 - 2.1.1.1. 시스템 물리 구성도
 - 2.1.2. 시스템 SW 목록
 - 2.1.3. 시스템 디렉토리 구성
 - 2.1.4. 시스템 기동과 정지

3. 운영 관리

- 3.1. 이미지 리파지토리 관리
 - 3.1.1. Image Repository 구성
 - 3.1.2. Docker-base Repository
 - 3.1.3. 서비스 이미지 이름 명명 규칙
 - 3.1.3.1. 이미지 명에 사용되는 변수와 값
 - 3.1.3.2. 포트 별 리파지토리 구성 및 용도
 - 3.1.4. 디스크 용량 관리
 - 3.1.4.1. 가용 공간 확인
 - 3.1.4.2. 수동 삭제
 - 3.1.4.3. 삭제 반영
 - 3.1.4.4. Cleanup policies
 - 3.1.4.5. 디스크 용량 자동 관리(Task)
 - 3.1.4.6. 디스크 용량 관리 주의사항 및 조치방법
- 3.2. Devops resources 관리
 - 3.2.1. CI파이프라인 리소스 관리
 - 3.2.2. 쿠버네티스 매니페스트 리소스 관리
 - 3.2.3. Gitlab 프로젝트 생성 제한
 - 3.2.3.1. 그룹내 프로젝트 및 서브 그룹 생성 제한 설정
 - 3.2.3.2. 사용자별 프로젝트 및 그룹 생성 제한 설정
- 3.3. Jenkins 주요 관리
 - 3.3.1. 시스템 설정
 - 3.3.1.1. Global properties
 - 3.3.1.2. 동시 빌드 실행할 job 수 조절
 - 3.3.2. 플러그인 관리
 - 3.3.2.1. Deploy Plugin
 - 3.3.2.2. Retrieve Plugin
 - 3.3.3. 사용자 인증 관리
 - 3.3.4. 그룹 권한 관리
 - 3.3.4.1. 공통 그룹 권한 생성
 - 3.3.4.2. 공통 그룹 권한 부여
 - 3.3.4.3. 업무 그룹 권한 생성
 - 3.3.4.4. 업무 그룹 권한 부여
 - 3.3.5. 사용자 권한 관리
 - 3.3.5.1. 관리자 권한 관리(권한 부여 / 회수)
 - 3.3.5.2. 업무 권한 관리(권한 부여 / 회수)
 - 3.3.6. Manage Credentials
 - 3.3.6.1. Credentials 생성
 - 3.3.6.2. Credentials 수정
 - 3.3.7. API Token 관리
 - 3.3.8. 신규 프로젝트 생성
 - 3.3.8.1. 어플리케이션 Gitlab 리파지토리 생성 (Gitlab 관리자 협업)
 - 3.3.8.2. 배포 Gitlab 리파지토리 생성 (OCP 인프라 관리자 협업)
 - 3.3.8.3. (Optional) 빌드 파이프라인 Jenkinsfile 생성 (Jenkins 빌드 관리자 & OCP 인프라 관리자 협업)
 - 3.3.8.4. (Optional) Jenkins 환경변수 추가 (Jenkins 관리자 협업)
 - 3.3.8.5. Jenkins View(카테고리) 생성 (Jenkins 관리자 협업)
 - 3.3.8.6. CI/CD Jenkins Job 생성 (Jenkins 관리자 협업)
 - 3.3.8.7. ArgoCD application 생성 (OCP 인프라 관리자 협업)
 - 3.3.9. PRD Job 긴급 수동 실행
 - 3.3.10. PRD Job 롤백
- 3.4. ArgoCD 주요 관리
 - 3.4.1. 세팅 관리
 - 3.4.2. Repository 추가
 - 3.4.3. Application 생성
 - 3.4.4. Application 수동 배포
- 3.5. CICD 이관 절차 및 방법
 - 3.5.1. 환경별 소스 배포
 - 3.5.2. CICD 파이프라인 스크립트
 - 3.5.2.1. DEVON-BOOT
 - 3.5.2.2. EX-BUILDER
 - 3.5.2.3. SPRING-BATCH
 - 3.5.2.4. SPRING-ANT
 - 3.5.2.5. SPRING-BOOT
 - 3.5.2.6. API-GATEWAY

- 3.5.2.7. API-PORTAL
 - 3.5.2.8. BLAZE-ADVISOR
 - 3.5.2.9. MINERVA-HTML5VIEWER
 - 3.5.2.10. MINERVA-IMAGE-CONVERTER
 - 3.5.2.11. NEXS-FRAMEWORK
 - 3.5.2.12. OZ-SCHEDULER
 - 3.5.2.13. OZ-SERVER
 - 3.5.2.14. PX-CALCULATOR
 - 3.5.2.15. SVN
 - 3.5.2.16. TOBE-SFA
 - 3.5.2.17. U-COVERAGE
 - 3.5.2.18. VANILLA-HTML
 - 3.5.2.19. ZIPI-T
 - 3.5.3. ArgoCD를 통한 배포 결과 확인
 - 3.6. 운영간 주의사항
 - 3.6.1. 주기적 SSL 인증서 교체
 - 3.6.2. 반복적인 다수의 job 빌드 timeout 실패 발생시
 - 3.6.3. Jenkins 로그인에 안된다는 유저 발생시
 - 3.6.4. Jenkins build 권한이 없다고 하는 경우
- #### 4. 장애 관리
- 4.1. 장애 케이스 별 조치방안
 - 4.1.1. Nexus disk full
 - 4.1.1.1. Blob 디렉토리 삭제
 - 4.1.1.2. Repository 삭제
 - 4.1.1.3. Blob 삭제
 - 4.1.1.4. Blob 생성
 - 4.1.1.5. Repository 생성
 - 4.1.1.6. Docker-group에 docker-ocp 포함
 - 4.1.2. Jenkins Job failure 유형 별 조치 방안
 - 4.1.2.1. Fatal: Remote branch xxx not found in upstream origin 유형

별첨 - CI/CD 관련 서버

개정이력

No.	버전	변경일	변경내용	작성자
1	v.1.0	2024-02-02	• 운영자 매뉴얼 작성	정민성

1. 개요

1.1. 목적

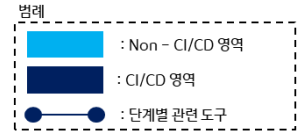
본 운영자 매뉴얼에서는 CICD를 위한 시스템을 정확하게 이해하고 안정적이고 효율적으로 관리할 수 있도록 정보를 제공하는 것을 목적으로 한다.

1.2. 대상

이 문서는 CICD 구성 요소인 Jenkins, Gitlab(LOS CI, LOS CD), ArgoCD 운영자 및 시스템 관리자를 대상으로 합니다.

2. 구성 관리

2.1. 시스템 구성



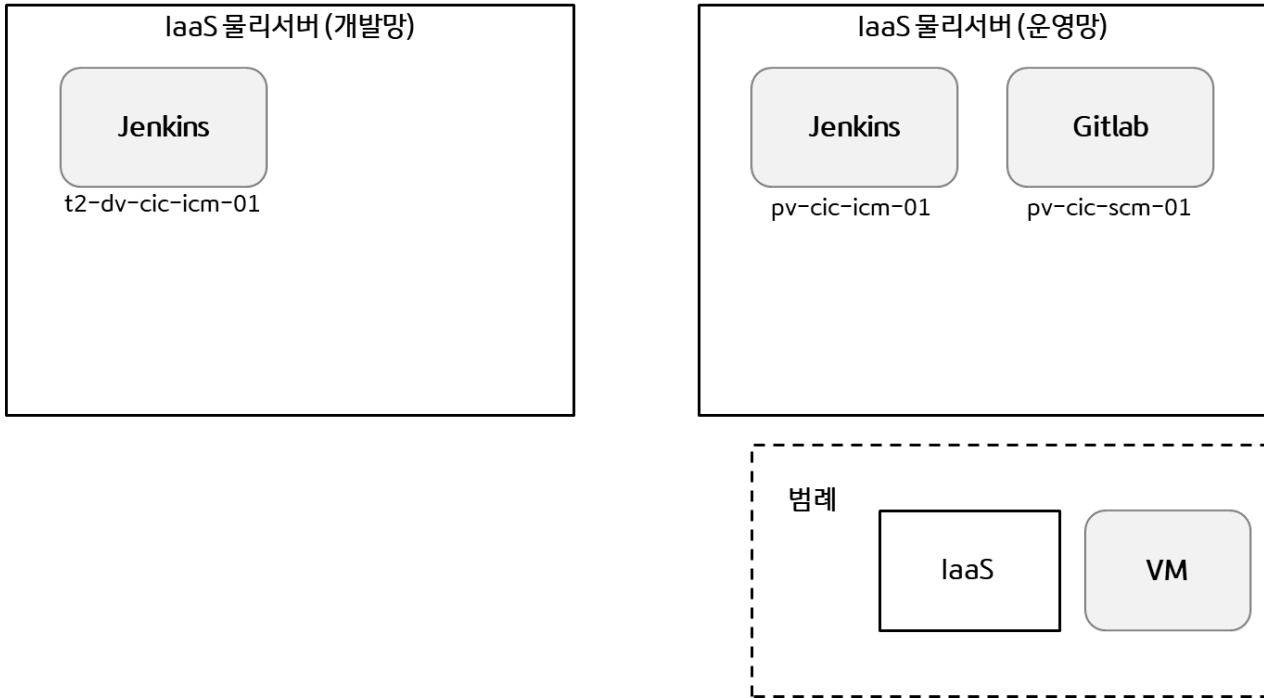
CICD 단계	단계별 프로세스 설명
1. Plan	무엇을 개발하고 어떻게 배포 계획을 수립하고 수립한 계획을 실행하는 단계로 관련 도구는 ITSM 다.
2. Code	개발을 위한 코드를 작성하고, 라이브러리 및 코드를 관리하는 단계로 관련 도구는 STS, Gitlab, Nexus 다.
3. Build	개발된 소스 코드를 빌드하는 단계로 관련 도구는 Jenkins, Gradle, Java 다.
4. Test	소스 코드 또는 이미지의 품질과 취약점을 위해 테스트하는 단계로 관련 도구는 DevOn Tester 다.
5. Release	테스트를 pass 한 산출물을 정제하여 배포 서버에서 사용할 수 있도록 릴리즈하는 단계로 관련 도구는 Jenkins, Podman 이다.
6. Deploy	배포 가능한 상태가 된 산출물을 서버에 배포하는 단계로 관련 도구는 ArgoCD, Gitlab 이다.
7. Operate & Monitor	DevOps 체제에서는 운영하고 관측하는 단계로 나누어 정의하였지만, CI/CD 설계 영역이 아닌 관계로 본 문서에서는 다루지 않지만 운영과 모니터링의 관련 도구는 OCP, Intermax 이다.

2.1.1. 시스템 구성도

환경	구분	호스트 이름	IP Address	OS 버전	커널 버전	역할
개발망	가상머신	t2-dv-cic-icm-01	172.10.30.47	RHEL (Red Hat Enterprise Linux) 8.6 - Ootpa	4.18.0-372.32.1.el8_6.x86_64	ICM
운영망	가상머신	pv-cic-scm-01	10.236.160.84	RHEL (Red Hat Enterprise Linux) 8.6 - Ootpa	4.18.0-372.32.1.el8_6.x86_64	SCM
운영망	가상머신	pv-cic-icm-01	10.236.160.85	RHEL (Red Hat Enterprise Linux) 8.6 - Ootpa	4.18.0-372.32.1.el8_6.x86_64	ICM

2.1.1.1. 시스템 물리 구성도

물리구성도



시스템	도메인	위치	비고
개발망 Jenkins	dlosjenkins.kblife.co.kr	VM	DEV, UAT CI 빌드 수행
운영망 Jenkins	plosjenkins.kblife.co.kr	VM	PRD CI 빌드 수행
Gitlab	plogitlab.kblife.co.kr	VM	Application source, CI script, CD yaml 관리 리파지토리
개발망 ArgoCD	openshift-gitops-server-openshift-gitops.apps.losdev.kblife.co.kr	OCP Node	dev, uat branch 기반으로 CD yaml 정보로 OCP 에 CD
운영망 ArgoCD	openshift-gitops-server-openshift-gitops.apps.losprd.kblife.co.kr	OCP Node	prd branch 기반으로 CD yaml 정보로 OCP 에 CD

2.1.2. 시스템 SW 목록

SW	위치	버전	비고
Jenkins	Jenkins(VM)	2.426.3	CI 도구
Gitlab	Gitlab(VM)	16.7(CE)	Application source, CI script, CD yaml 형상 관리 도구
Java	Jenkins(VM)	1.8.0_322	CICD 서버 구동을 위한 JDK
Gradle	Jenkins(VM)	7.4	Back end Framework 소스코드 빌드 도구
NPM	Jenkins(VM)	6.14.17	Front end Framework 소스코드 빌드 도구
Node	Jenkins(VM)	v14.20.0	Front end Framework 소스코드 빌드 도구
Podman	Jenkins(VM)	4.4.1	이미지 관리 도구

2.1.3. 시스템 디렉토리 구성

시스템	구분	위치	설명
Jenkins	Home	/DATA/jenkins	Jenkins가 구동되는데 필요한 파일 위치
	Data	/DATA/jenkins	Jenkins가 파이프라인 구동에 필요한 데이터 파일 위치

Gitlab	Home	/var/opt/gitlab	Gitlab이 구동되는데 필요한 파일 위치
	Data	/DATA/gitlab	Gitlab이 구동되는데 필요한 파일 위치
	Log	/LOG/gitlab	Gitlab log 파일 위치

2.1.4. 시스템 기동과 정지

시스템	실행	명령어
Jenkins	시작	<pre>systemctl start jenkins.service</pre>
	중지	<pre>systemctl stop jenkins.service</pre>
	상태	<pre>systemctl status jenkins.service</pre>
Gitlab	시작	<pre>gitlab-ctl start</pre>
	중지	<pre>gitlab-ctl stop</pre>
	상태	<pre>gitlab-ctl status</pre>

3. 운영 관리

3.1. 이미지 리파지토리 관리

ArgoCD에 의해 배포 될 이미지 리파지토리 구성과 주요 관리 방법을 기술한다.

3.1.1. Image Repository 구성

망 분리 환경의 제약으로 PRD / DEV(UAT) Image Repository 분리 구성 필요하다.

망 구분	도메인	비고
PRD	plosnexus.kblife.co.kr	Track#1 에 구성된 nexus 이용
DEV/UAT	dlosnexus.kblife.co.kr	Track#1 에 구성된 nexus 이용

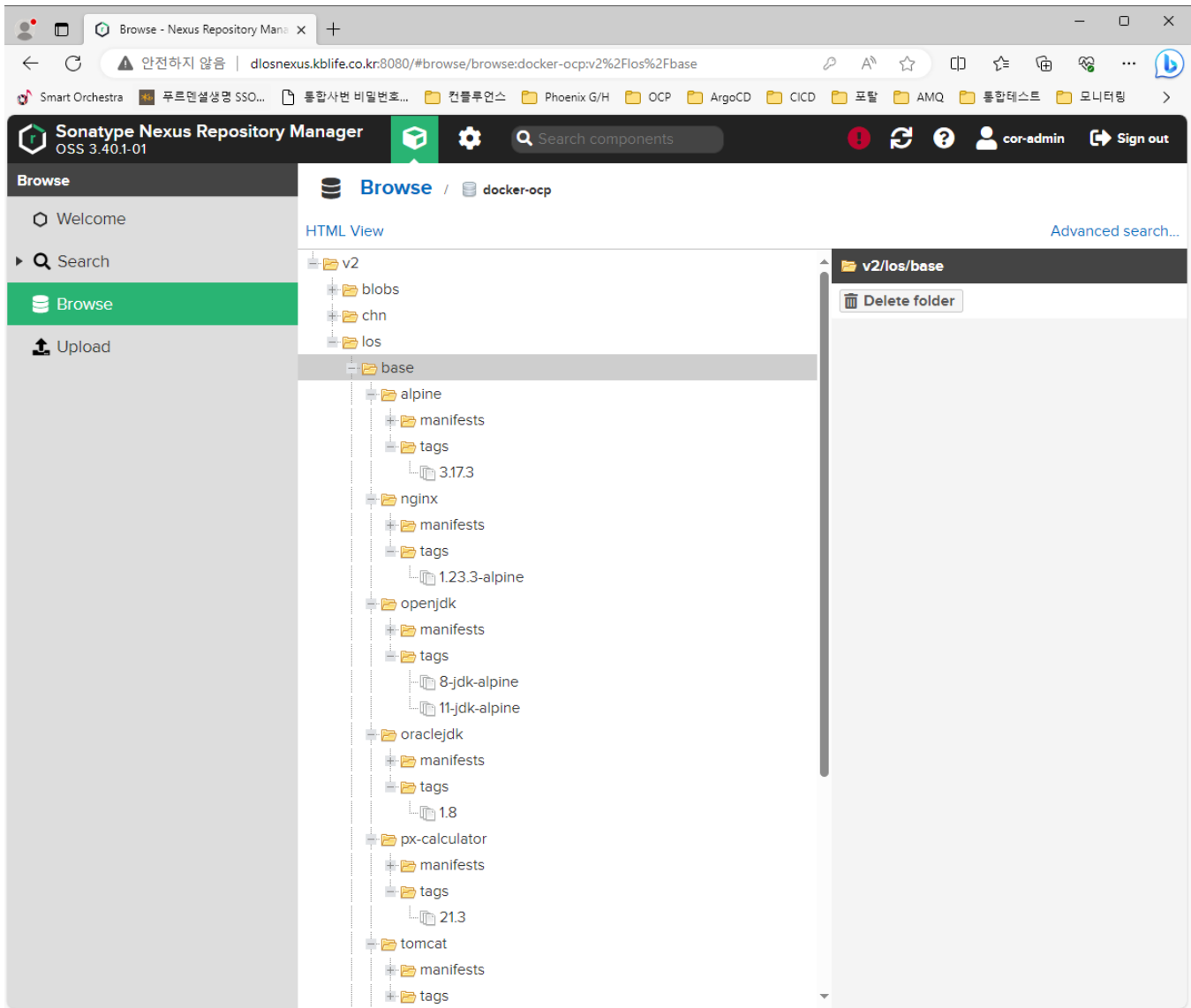
3.1.2. Docker-base Repository

서비스 이미지 생성 시, base 이미지로 사용 되는 이미지가 관리되는 리파지토리이다.

Nexus > Browse > docker-ocp > v2/los/base 폴더 확인

기본적으로 컨테이너 이미지는 경량화된 alpine linux 를 베이스 이미지로 사용한다. 하지만 어떠한 이유에서 alpine linux 를 사용하기 힘든 경우 예외적으로 ubuntu 이미지를 사용하는 경우도 있다.

이미지 이름	설명	용도	Cl gitlab 리파지토리 명
alpine	docker registry 에서 제공하는 alpine 이미지	주소정제 솔루션 base 이미지로 사용 중	1. ZIPI-T
nginx	docker registry 에서 제공하는 nginx 이미지	Front-end base 이미지로 사용 중	1. EX-BUILDER 2. TOBE-SFA 3. VANILLA-HTML
openjdk	docker registry 에서 제공하는 openjdk 이미지	Back-end base 이미지로 사용 중	1. DEVON-BOOT 2. OPEN-LABS 3. OZ-SCHEDULER (rpt-dmn) 4. SPRING-BOOT 5. U-COVERAGE
oraclejdk	docker registry 에서 제공하는 oraclejdk 이미지	BRE 시스템 base 이미지로 사용 중	1. BLAZE-ADVISOR 2. NEXS-FRAMEWORK
tomcat	docker registry 에서 제공하는 tomcat 이미지	Tomcat 으로 서비스 되는 솔루션 base 이미지로 사용 중	1. API-PORTAL 2. MINERVA-HTML5VIEWER 3. MINERVA-IMAGE-CONVERTER 4. OZ-SERVER 5. SPRING-ANT
ubuntu	docker registry 에서 제공하는 ubuntu 이미지	Alpine linux 개발 역량이 부족한 서비스의 이미지로 사용 중	1. OZ-SCHEDULER (mark-any)



3.1.3. 서비스 이미지 이름 명명 규칙

이미지 컨텍스트: [프로젝트 코드] / [시스템 코드]

이미지 명: [시스템 코드] > [업무 코드] > [서비스 종류] > [프로파일]

태그: v[Psr No] > [빌드 번호]

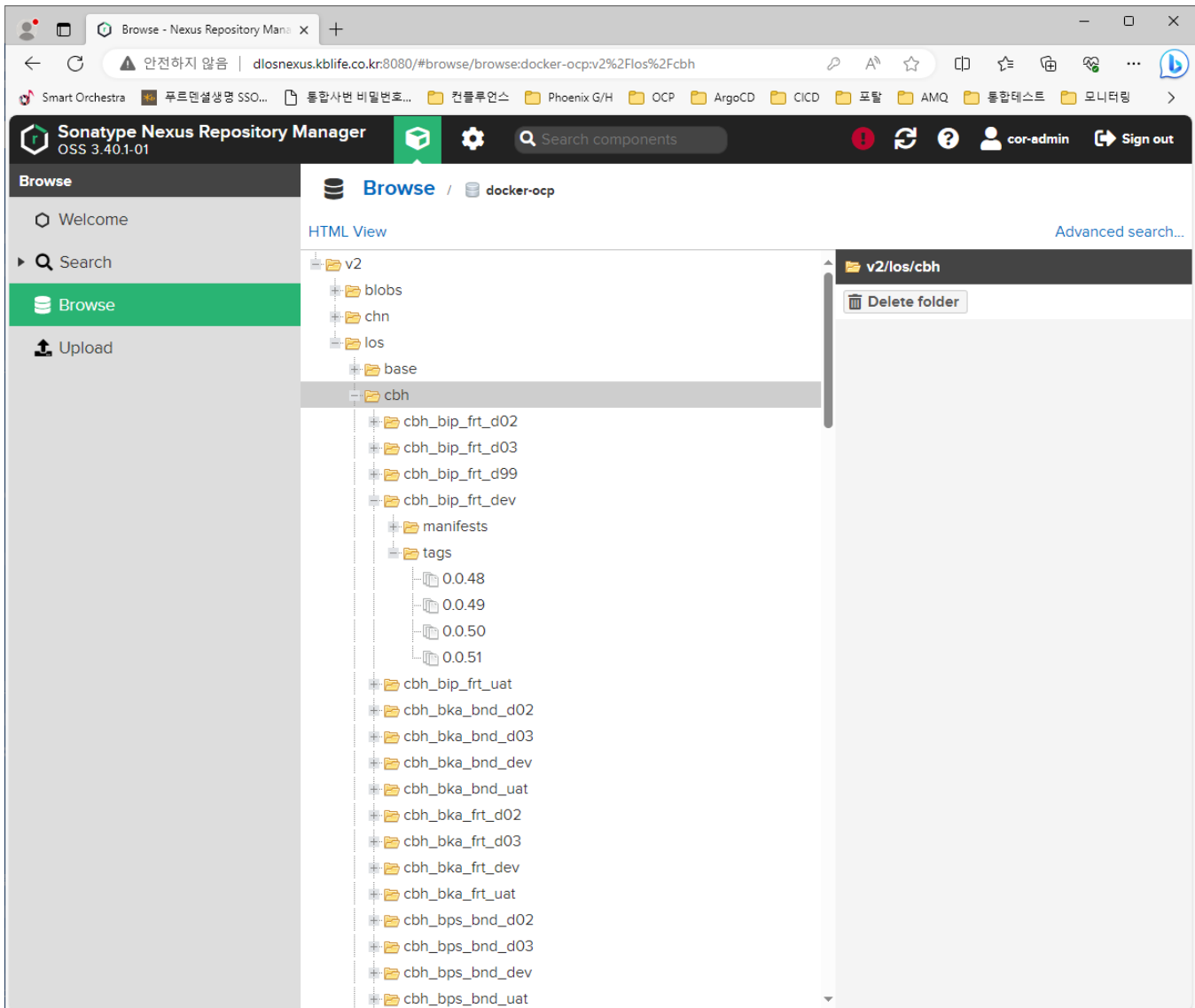
포트: 8000(base image), 8010(ocp image), 8020(aks image), 5000(group)

예) dlosnexus.kblife.co.kr:5000/los/base/tomcat:9.0.8-jre8-alpine

3.1.3.1. 이미지 명에 사용되는 변수와 값

변수	값	비고
시스템 코드	'비고' 란의 시스템 코드	시스템/업무코드 정의 > LOS Track2_31.Architecture > Confluence (prudential.co.kr)
업무 코드	'비고' 란의 업무 코드	시스템/업무코드 정의 > LOS Track2_31.Architecture > Confluence (prudential.co.kr)

서비스 종류	frt, fpr, fmr, bnd, sck, dmn, bat	
프로파일	dev, uat, prd	
Psr No	ITSM 에서 Jenkins Job 호출 시, 파라미터로 전달 된 값	
빌드 번호	Jenkins Job 실행 시, 자동 증가하는 고유한 값 \${BUILD_NUMBER}	



3.1.3.2. 포트 별 리파지토리 구성 및 용도

포트 별로 다른 Blob Store 에 저장되고, Group 포트를 통해 Pull 할 경우 저장 위치에 상관없이 다운로드 된다.

리파지토리를 생성할 때 포트 번호와 해당 리파지토리가 사용할 Blob Store 를 선택한다.

따라 해당 포트를 통해 업로드 / 다운로드 하게 되면 설정된 Blob Store 에 저장하고 제공한다.

이는 디스크 용량 관리에 중요한 부분이며, 용도에 맞춰 포트를 사용한다.

서비스 포트	리파지토리 명	Blob 명	용도
--------	---------	--------	----

8000	docker-base	docker-base	Base 이미지 관리
8010	docker-ocp	docker-ocp	OCP 배포용 서비스 이미지 관리
8020	docker-aks	docker-aks	AKS 배포용 서비스 이미지 관리 (AKS 환경이 구축되고 ACR 을 사용하지 않을 경우에 사용)
5000	docker-group	docker-hosted	다운로드 전용 (이미지 그룹)

	Name ↑	Type	Format	Status
	docker-aks	hosted	docker	Online
	docker-base	hosted	docker	Online
	docker-group	group	docker	Online
	docker-ocp	hosted	docker	Online

Browser: Blob Stores - Nexus Repository | dloxnexus.kblife.co.kr:8080/#admin/repository/blobstores

Smart Orchestra | 푸르덴셜생명 SSO... | 통합사번 비밀번호... | 컨플루언스 | Phoenix G/H | OCP | ArgoCD | CICD | 포탈 | AMQ | 통합테스트 | 모니터링

Sonatype Nexus Repository Manager OSS 3.40.1-01

Administration

- Repository
 - Repositories
 - Blob Stores**
 - Log4j Visualizer
 - Proprietary Repositories
 - Content Selectors
 - Cleanup Policies
 - Routing Rules
- Security
 - Privileges
 - Roles
 - Users
 - Anonymous Access
 - LDAP
 - Realms
 - SSL Certificates
- Support
 - Logging

Blob Stores Create Blob Store

Configure local and cloud blob storage

Filter by name

NAME	TYPE	STATE	BLOB COUNT	TOTAL SIZE	AVAILABLE SPACE
default	File	Started	178169	36.03 GB (38692062773)	1.52 TiB
docker-aks	File	Started	622	10.56 GB (11342127209)	1.52 TiB
docker-base	File	Started	0	0.00 Bytes (0)	1.52 TiB
docker-hosted	File	Started	0	0.00 Bytes (0)	1.52 TiB
docker-ocp	File	Started	15824	357.66 GB (384033223563)	1.52 TiB
maven_3rdparty	File	Started	14474	1.17 GB (1251214331)	1.52 TiB
maven_cyb	File	Started	338855	345.38 MB (362153108)	1.52 TiB
maven_int	File	Started	1367391	1.88 GB (2020749938)	1.52 TiB
maven_proxy	File	Started	0	0.00 Bytes (0)	1.52 TiB

3.1.4. 디스크 용량 관리

3.1.4.1. 가용 공간 확인

관리자 로그인 > 설정 > Blob Stores > AVAILABLE SPACE 확인

AVAILABLE SPACE 는 nexus VM 에서 남은 스토리지 용량을 의미하고, 이는 모든 Blob Store 가 공유해서 사용하는 자원이다.

Sonatype Nexus Repository Manager OSS 3.40.1-01

Administration

- Repository
 - Repositories
 - Blob Stores**
 - Log4j Visualizer
 - Proprietary Repositories
 - Content Selectors
 - Cleanup Policies
 - Routing Rules
- Security
 - Privileges
 - Roles
 - Users
 - Anonymous Access
 - LDAP
 - Realms
 - SSL Certificates
- Support
 - Logging

Blob Stores

Configure local and cloud blob storage

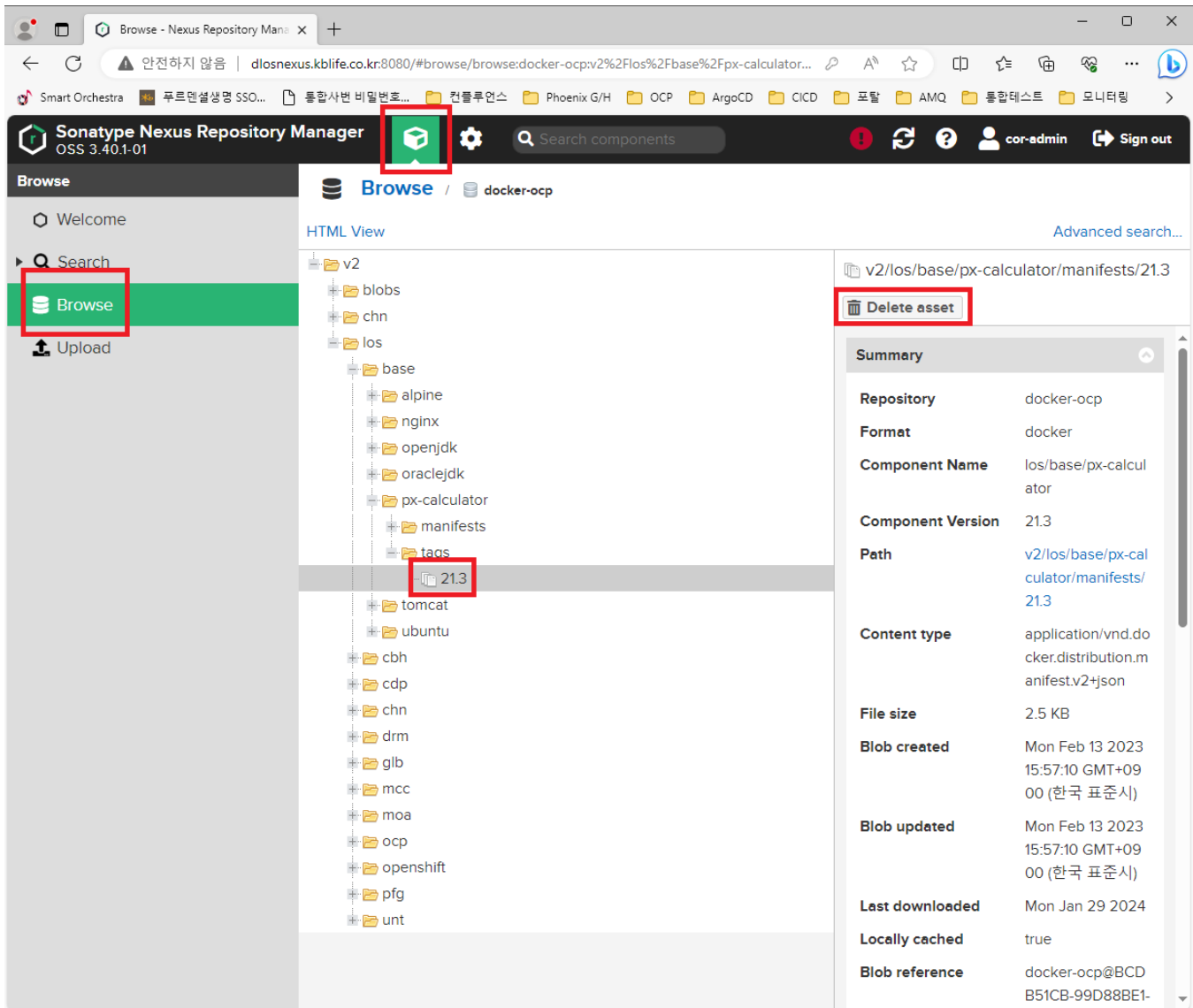
Create Blob Store

Filter by name

NAME	TYPE	STATE	BLOB COUNT	TOTAL SIZE	AVAILABLE SPACE
default	File	Started	178169	36.03 GB (38692062773)	1.52 TiB
docker-aks	File	Started	622	10.56 GB (11342127209)	1.52 TiB
docker-base	File	Started	0	0.00 Bytes (0)	1.52 TiB
docker-hosted	File	Started	0	0.00 Bytes (0)	1.52 TiB
docker-ocp	File	Started	15824	357.66 GB (384033223563)	1.52 TiB
maven_3rdparty	File	Started	14474	1.17 GB (1251214331)	1.52 TiB
maven_cyb	File	Started	338855	345.38 MB (362153108)	1.52 TiB
maven_int	File	Started	1367391	1.88 GB (2020749938)	1.52 TiB
maven_proxy	File	Started	0	0.00 Bytes (0)	1.52 TiB

3.1.4.2. 수동 삭제

관리자 로그인 > Contents > Browse > repository 이름 > 삭제할 대상(tag 또는 상위 폴더) 선택 > Delete asset 클릭



3.1.4.3. 삭제 반영

관리자 로그인 > 설정 > Task > Incomplete delete 실행 > unused delete 실행 > compact 실행 > 3.1.3.1. 의 가용 공간 확인

(Compact 를 마지막에 실행)

Tasks - Nexus Repository Manager

안전하지 않음 | dloxnexus.kblife.co.kr:8080/#admin/system/tasks

Smart Orchestra | 푸르덴셜생명 SSO... | 통합사번 비밀번호... | 컨플루언스 | Phoenix G/H | OCP | ArgoCD | CICD | 포탈 | AMQ | 통합테스트 | 모니터링

Sonatype Nexus Repository Manager
OSS 3.40.1-01



Search components

    cor-admin  Sign out

Administration

LDAP

Realms

SSL Certificates

IQ Server

Support

Logging

Logs

Status

Support ZIP

System Information

System

API

Bundles

Capabilities

Email Server

HTTP

Licensing

Nodes

Tasks

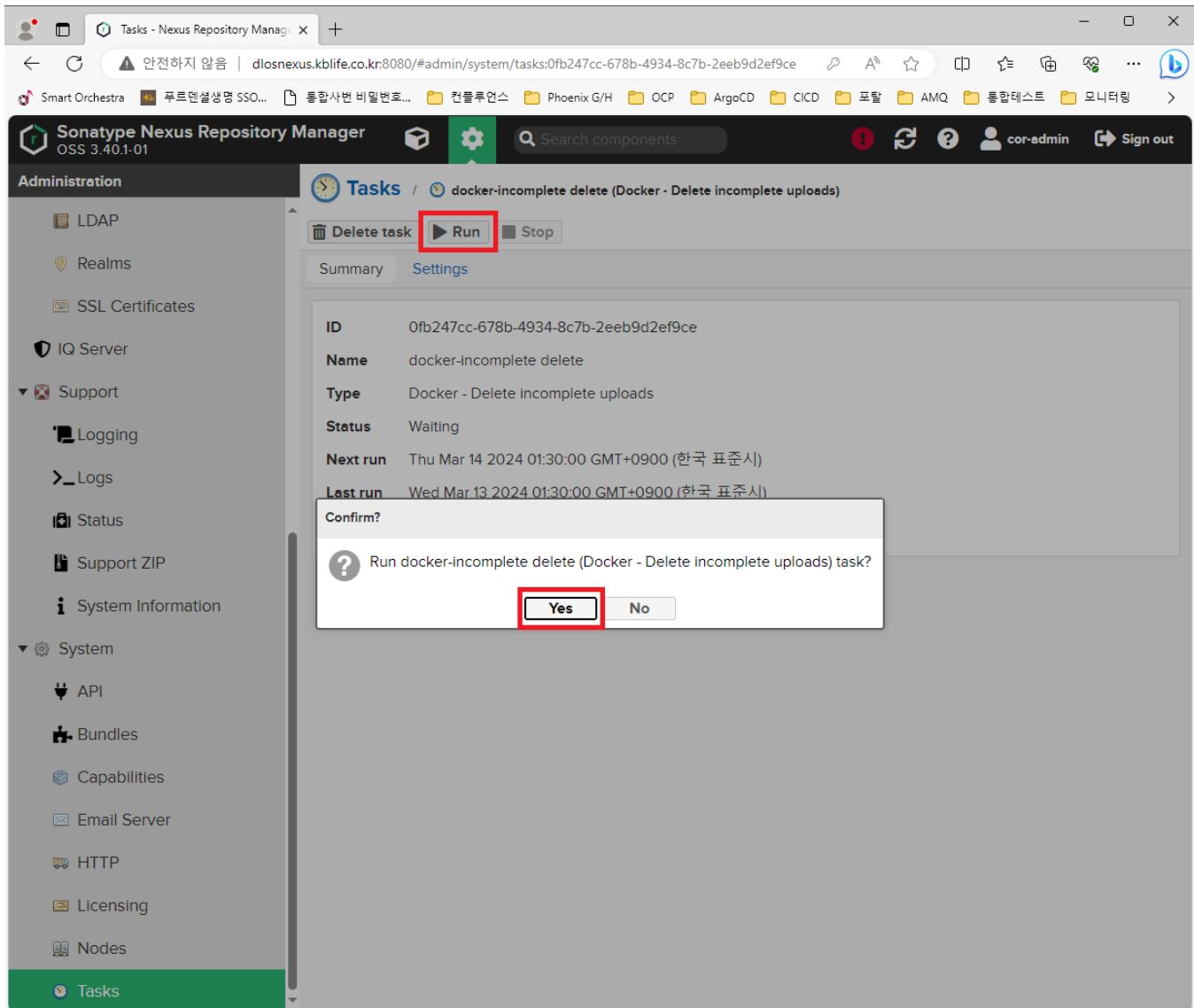
Tasks

Manage scheduled tasks

Create task

Filter

	Name ↑	Type	Status	Schedule	Next run	Last run	Last result	
	Cleanup service	Admin - Cleanu...	Waiting	advanced	Thu Mar 14 20...	Wed Mar 13 20...	Ok [1s]	
	docker-incomp...	Docker - Delet...	Waiting	daily	Thu Mar 14 20...	Wed Mar 13 20...	Ok [2s]	
	docker-ocp co...	Admin - Comp...	Waiting	daily	Thu Mar 14 20...	Wed Mar 13 20...	Ok [0s]	
	docker-unused...	Docker - Delet...	Waiting	daily	Thu Mar 14 20...	Wed Mar 13 20...	Ok [3s]	
	Statistics - reca...	Statistics - reca...	Waiting	advanced	Thu Mar 14 20...	Wed Mar 13 20...	Ok [26s]	



3.1.3.4. Cleanup policies

관리자 로그인 > Repository > Cleanup Policies 선택 > Create Cleanup Policy 클릭

- 정책 명, 보존 기간(생성일자 경과일, 실사용 경과일), 삭제 대상 이름 패턴(AND 조건)
- 대상 레파지토리 선택 하고 삭제 예정 리스트를 검색 가능 (위 설정한 조건의 대상 확인 가능)
- Repository > Repositories 메뉴에서 cleanup policies 정책 선택하여 repository별 정책 설정 가능

Cleanup Policies - Nexus Repository Manager OSS 3.40.1-01

Settings

Search components

cor-adminSign out

Administration

- Repository
 - Repositories
 - Blob Stores
 - Log4j Visualizer
 - Proprietary Repositories
 - Content Selectors
 - Cleanup Policies
 - Routing Rules
- Security
 - Privileges
 - Roles
 - Users
 - Anonymous Access
 - LDAP
 - Realms
 - SSL Certificates
- IQ Server
- Support
- Logging

Cleanup Policies

Manage component removal configuration

Create Cleanup Policy

Filter

NAME	FORMAT	NOTES
cleanup-chn-dev-1d	docker	Track#1 의 chn 이미지를 지운다.
cleanup-chn-uat-3d	docker	
cleanup-cyb-1d-1	maven2	사이버센터(홈페이지) SNAPSHOT 제거1
cleanup-cyb-1d-2	maven2	사이버센터(홈페이지) SNAPSHOT 제거2
cleanup-dev-1d	docker	
cleanup-dev-2d	docker	
cleanup-uat-1d	docker	
cleanup-uat-3d	docker	
cleanup-uat-cdp-bnd-3d	docker	
cleanup-uat-sfa-bnd-3d	docker	

What is a cleanup policy?

Cleanup policies can be used to remove content from your repositories. These policies will execute at the

Cleanup Policies - Nexus Repository

안전하지 않음 | dlo.nexus.kblife.co.kr:8080/#admin/repository/cleanuppolicies

Smart Orchestra | 푸르덴셜생명 SSO... | 통합사번 비밀번호... | 컨플루언스 | Phoenix G/H | OCP | ArgoCD | CICD | 포탈 | AMQ | 통합테스트 | 모니터링

Sonatype Nexus Repository Manager

OSS 3.40.1-01

Search components

cor-admin

Sign out

Administration

Repository

Repositories

Blob Stores

Log4j Visualizer

Proprietary Repositories

Content Selectors

Cleanup Policies

Routing Rules

Security

Privileges

Roles

Users

Anonymous Access

LDAP

Realms

SSL Certificates

IQ Server

Support

Logging

Logs

Status

Name

Use a unique name for the cleanup policy

create-cleanup-policy

Format

The format that this cleanup policy can be applied to

docker

Notes Optional

Cleanup Criteria

☒ Component Age

Remove components that were published over:

3 days ago

☒ Component Usage

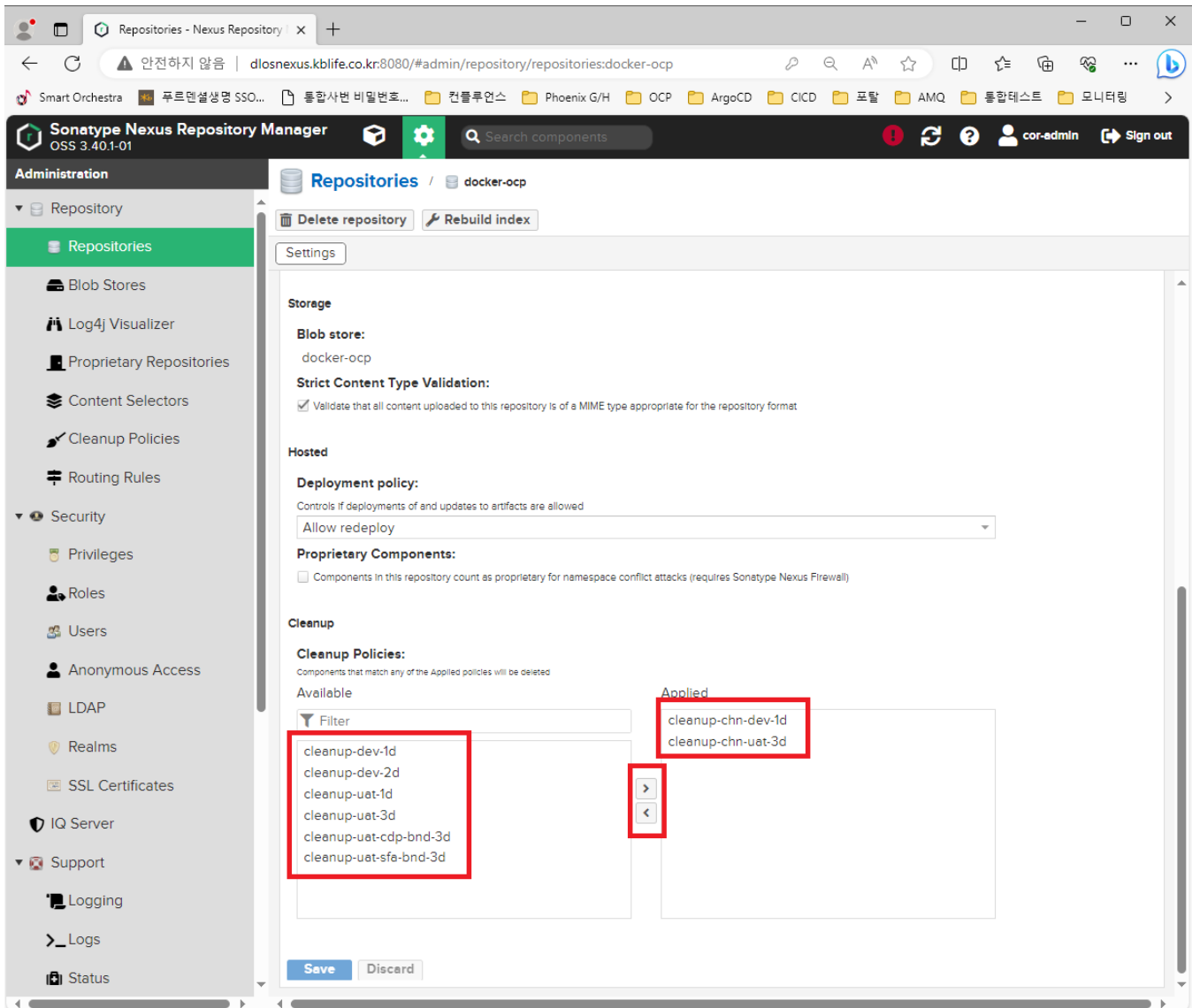
Remove components that haven't been downloaded in:

3 days

☒ Asset Name Matcher

Remove components that have at least one asset name matching the following regular expression pattern:

*-dev



3.1.4.5. 디스크 용량 자동 관리(Task)

관리자 로그인 > System > Task 선택

Schedule 설정과는 별도로 필요한 시점에 Manual 로 실행 가능

Task 목록

Task Name	Schedule	용도
Cleanup service	매일 01:00	3.1.3.4. cleanup policies 에서 수립된 정책 수행
Docker-Incomplete delta	매일 01:30	업로드 중단 등으로 발생한 파일 삭제
Docker-Unused delete	매일 02:00	이미지의 특정 레이어 삭제로 사용되지 않는 데이터 삭제
Docker-ocp Compact	매일 02:30	3.1.3.3. 삭제 반영을 통해 물리적인 삭제 수행

3.1.4.6. 디스크 용량 관리 주의사항 및 조치방법

위에 기술된 작업은 지워진 컨테이너 이미지 용량을 정리하는 것으로 실제 nexus 에 존재하는 컨테이너 이미지를 제거하는 아래 cleanup-images.sh 에 시스템 코드를 추가해야한다.

신규 업무 추가시 서비스 이미지 제거하는 스크립트에 추가되는 업무 서비스 코드 추가해야한다.

추가하지 않으면 해당 업무의 이미지가 지워지지 않고 계속 쌓여 nexus 용량을 모두 사용할 수 있다.

위 스크립트는 각 넥서스 VM 의 /root/scripts/cleanup-images.sh 에 위치하고 있으며, 해당 파일은 crontab 으로 일정 시간마다 수행되며 이미지를 지우고있다.

dev, uat 환경은 2개의 컨테이너 이미지를 유지하고, prd 환경은 3개의 컨테이너 이미지를 유지하게 스크립트가 구성되어있다.

넥서스 VM 에 할당된 스토리지를 참고하여 컨테이너 이미지 관리 정책을 수립하고 정책에 부합하는 스크립트로 적용하여야한다.

/root/scripts/cleanup-images.sh 하위의 projects 변수에 해당하는 시스템 코드에 추가해야한다.

/root/scripts/cleanup-images.sh

```
#!/bin/bash

declare -A projects
projects["glb"]="glb_asc_bnd glb_asp_bnd glb_smp_frt glb_dbs_bnd glb_pub_frt glb_tit_bnd"
projects["chn"]="chn_pmd_bnd chn_sfa_bnd chn_sfa_fmr chn_sfa_fpr chn_sfa_frt chn_stc_bnd chn_stc_frt
chn_voc_bnd chn_voc_frt"
projects["cbh"]="cbh_bip_frt cbh_bka_bnd cbh_bka_frt cbh_bps_bnd cbh_cct_bnd cbh_cct_frt cbh_clm_bnd
cbh_clm_frt cbh_cme_bnd cbh_cmn_bnd cbh_cmn_frt cbh_dep_bnd cbh_dep_frt cbh_iap_bnd cbh_jpm_bnd cbh_jps_bnd
cbh_mhc_bnd cbh_mhc_frt cbh_nbs_bnd cbh_nbs_frt cbh_pay_bnd cbh_pay_frt cbh_pgh_bnd cbh_pgh_bnd_for_cct
cbh_pos_bnd cbh_pos_frt cbh_var_bnd cbh_var_frt"
projects["cdp"]="cdp_cip_frt cdp_cim_bnd cdp_cim_frt cdp_civ_bnd cdp_civ_frt cdp_clc_bnd cdp_clc_frt"
projects["pfg"]="pfg_pim_bnd pfg_pis_bnd pfg_plm_bnd pfg_pmm_bnd"
projects["unt"]="unt_amp_bat unt_amp_frt unt_amq_dmn unt_ass_sck_zipit unt_ass_sck_init unt_ass_sck_zipbae
unt_ass_sck_srchsrv unt_cvt_frt unt_efm_frt unt_efs_frt unt_img_frt unt_pgw_frt unt_rbm_frt unt_rbs_frt
unt_mic_frt unt_pfm_bnd unt_rpt_dmn unt_rpt_dmn_mark_any unt_rpt_frt unt_ssp_bnd unt_ssp_fmr unt_ssp_fpr
unt_ucg_frt"

env_codes="dev uat d02 d03"

for system_code in "${!projects[@]}"; do
  for biz_code in ${projects[${system_code}]}; do
    for env_code in ${env_codes}; do
      items=$(curl -s -X GET -H "Authorization: 246b90b5-62c6-3855-8f04-a200d2834329" "dlosnexus.kblife.co.kr:8080/service/rest/v1/search/assets?repository=docker-ocp&docker.imageName=los%2F${system_code}%2F${biz_code}_${env_code}" | jq "[.items[] | {downloadUrl: .downloadUrl, id: .id, lastModified: .lastModified}] | sort_by(.lastModified)")
      length=$(echo $items | jq ". | length")

      for ((i = 0; i < $length - 2; i++)); do
        item=$(echo $items | jq ".[${i}]")
        downloadUrl=$(echo $item | jq -r ".downloadUrl")
        id=$(echo $item | jq -r ".id")
        curl -X DELETE -u cor-admin:adm123$ "dlosnexus.kblife.co.kr:8080/service/rest/v1/assets/$id"
        echo "Deleted $downloadUrl"
      done
    done
  done
done

compact_task_id="b57e18d8-3455-4f7f-951d-f3c1e3aff11e"
curl -X POST -u cor-admin:adm123$ "dlosnexus.kblife.co.kr:8080/service/rest/v1/tasks/$compact_task_id/run"
```

3.2. Devops resources 관리

CICD Pipeline에서 사용되는 리소스를 Gitlab 의 LOS CI / LOS CD 그룹에서 관리

URL : <https://plogitlab.kblife.co.kr>

구성 : 어플리케이션 소스 / [CI 스크립트](#) (관리자 또는 CI 권한 필요) / [CD yam](#) (관리자 또는 CD, OCP 권한 필요)

3.2.1. CI 파이프라인 리소스 관리

서비스 build 에 필요한 CI 파이프라인 리소스 제공

URL : <https://plogitlab.kblife.co.kr/los-ci>

- Project Name : [제품명 / 프레임워크명 / 솔루션명] 로 총 21 중
- Branch : prd / uat / dev 로 구성되며, 각각의 환경에 맞게 구분
- Jenkinsfile : 서비스 배포를 위한 빌드 과정을 설정한 스크립트 파일
- Dockerfile : 컨테이너 이미지를 생성하기 위한 설정 스크립트 파일 (**OCP 에 구동되는 서비스는 필수!**)
- (Optional) add, copy : 컨테이너 이미지 내에 배포될 서비스 구동과 관련한 설정 파일

CI 프로젝트 목록

	프로젝트명	설명	URL	CI가 적용된 서비스명	비고
1	API-GATEWAY	API Gateway 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/api-gateway	1. UNT_IGS_BND 2. UNT_AGW_BND	
2	API-PORTAL	APIM 포탈, 배치 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/api-portal	1. UNT_AMP_BAT 2. UNT_AMP_FRT	
3	BLAZE-ADVISOR	BRE 업무의 RBMS 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/blaze-advisor	1. UNT_RBM_FRT 2. UNT_RBS_FRT	
4	DEVON-BOOT	DevOn Boot 프레임워크 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/devon-boot	1. CBH_BKA_BND 2. CBH_BPS_BND 3. CBH_CCT_BND 4. CBH_CLM_BND 5. CBH_CME_BND 6. CBH_CMN_BND 7. CBH_DEP_BND 8. CBH_IAP_BND 9. CBH_JPS_BND 10. CBH_MHC_BND 11. CBH_NBS_BND 12. CBH_PAY_BND 13. CBH_PGH_BND 14. CBH_POS_BND 15. CBH_VAR_BND 16. CDP_CIM_BND 17. CDP_CIV_BND 18. CDP_CLC_BND 19. CHN_PMD_BND 20. CHN_SFA_BND 21. CHN_STC_BND 22. CHN_TMS_BND 23. CHN_VOC_BND 24. GLB_CBH_BND 25. GLB_CMN_BND 26. GLB_DBE_BND 27. GLB_DBS_BND 28. PFG_PIS_BND 29. PFG_PMM_BND 30. PFG_SFT_BND 31. UNT_AMQ_DMN 32. UNT_SSP_BND	
5	E-GOV-FRAMEWORK	전자정부 프레임워크 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/e-gov-framework	1. PFG_PIM_BND	상품플랫폼 파트에서 자체 관리하는 프로젝트

6	EX-BUILDER	Ex Builder 프레임워크 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/ex-builder	1. CBH_BIP_FRT 2. CBH_BKA_FRT 3. CBH_CCT_FRT 4. CBH_CLM_FRT 5. CBH_CMN_FRT 6. CBH_DEP_FRT 7. CBH_NBS_FRT 8. CBH_PAY_FRT 9. CBH_POS_FRT 10. CBH_VAR_FRT 11. CDP_CIM_FRT 12. CDP_CIP_FRT 13. CDP_CIV_FRT 14. CDP_CLC_FRT 15. CHN_VOC_FRT 16. GLB_BAS_FRT 17. GLB_SMP_FRT	
7	MINERVA-HTML5VIEWER	이미지 뷰어 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/minerva-html5viewer	1. UNT_IMG_FRT	
8	MINERVA-IMAGE-CONVERTER	이미지 컨버터 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/minerva-image-converter	1. UNT_CVT_FRT	
9	NEXS-FRAMEWORK	BRE 업무의 EUS/FDS 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/nexs-framework	1. UNT_EFM_FRT 2. UNT_EFS_FRT	
10	NON-STANDARD	비표준 업무 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/non-standard	1. UNT_EFS_FRT_[환경]_RESTART	
11	OZ-SCHEDULER	리포트 스케줄러 데몬 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/oz-scheduler	1. UNT_RPT_DMN	
12	OZ-SERVER	리포트 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/oz-server	1. UNT_RPT_FRT	
13	PX-CALCULATOR	PX calculator 서버 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/px-calculator	1. CBH_JPM_BND	
14	SPRING-ANT	Spring 프레임워크 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/spring-ant	1. CHN_SFA_FRT 2. UNT_AMS_FRT 3. UNT_CRX_FRT 4. UNT_EAP_BND 5. UNT_MIC_FRT 6. UNT_OMN_FRT	
15	SPRING-BATCH	Spring Batch 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/spring-batch	1. CBH_BKA_BAT 2. CBH_CBH_BAT 3. CDP_CDP_BAT 4. CHN_TMS_BAT 5. CHN_VOC_BAT 6. PFG_PFG_BAT 7. UNT_SSP_BAT	
16	SPRING-BOOT	Spring Boot 기반 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/spring-boot	1. PFG_SFT_FRT 2. UNT_PGW_FRT	
17	SVN	SVN 으로 관리되는 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/svn	1. UNT_SCT_FRT	
18	TEMP-Cis	임시 개발 프로젝트	https://plogitlab.kblife.co.kr/los-ci/temp-cis	1. UNT_PMS_FRT	
19	TOBE-KBL	구KB생명 CI/CD 프로젝트	https://plogitlab.kblife.co.kr/los-ci/tobe-kbl	1. FCP_MRS_BAT 2. FCP_MRS_BND 3. FCP_MRS_FRT 4. UNT_RKS_ADM 5. UNT_RKS_BND	SI 기간동안 임시로 사용하는 프로젝트 명으로, 프로젝트 명 재정의 필요

20	TOBE-SFA	SFA대응개발 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/tobe-sfa	1. CHN_SFA_FMR 2. CHN_SFA_FPR 3. UNT_SSP_FMR 4. UNT_SSP_FPR	SI 기간동안 임시로 사용하는 프로젝트 명으로, 프로젝트 명 재정의 필요
21	U-COVERAGE	유 커버리지 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/u-coverage	1. UNT_UCG_FRT	
22	VANILLA-HTML	HTML 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/vanilla-html	1. CBH_MHC_FRT 2. CHN_TMS_FRT	
23	ZIPI-T	주소조회 서비스 빌드 프로젝트	https://plogitlab.kblife.co.kr/los-ci/zipi-t	1. UNT_ASS_SCK	

왼쪽 상단 메뉴 > Groups > Explore groups > LOS CI 그룹 > 확인을 할 프로젝트 클릭

The screenshot displays the GitLab 'Explore groups' interface. On the left sidebar, the 'Groups' menu item is highlighted with a red box. The main content area lists various groups, including 'Z-CMN-MEMBERS', 'Z-CLM-MEMBERS', 'Z-PAY-MEMBERS', 'Z-DEP-MEMBERS', 'Z-VAR-MEMBERS', 'Z-POS-MEMBERS', 'Z-NBS-MEMBERS', 'Z-VOC-MEMBERS', 'Z-DBA-MEMBERS', 'LOS UNT', 'LOS CD', 'LOS CI' (highlighted with a red box), 'LOS GLB', 'LOS CDP', and 'LOS PFG'. At the bottom of the page, a pagination control shows the current page is 3, which is also highlighted with a red box.

LOS CI / DEVON-BOOT · GitLab

https://plogitlab.kblife.co.kr/los-ci/devon-boot

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

LOS CI / DEVON-BOOT

DEVON-BOOT

704 Commits 3 Branches 0 Tags 58.3 MiB Project Storage

Topics: CBH CHN CDP + 3 more

BKA, BPS, CCT, CLM, CME, CMN, DEP, IAP, JPS, MHC, NBS, PAY, PGH, POS, VAR, CIM, CIV, CLC, PMD, SFA, STC, VOC, DBE, DBS, PIS, PMM, SSP

Update endpoint.sh
AA파트 authored 2 days ago fde6091a

dev devon-boot +

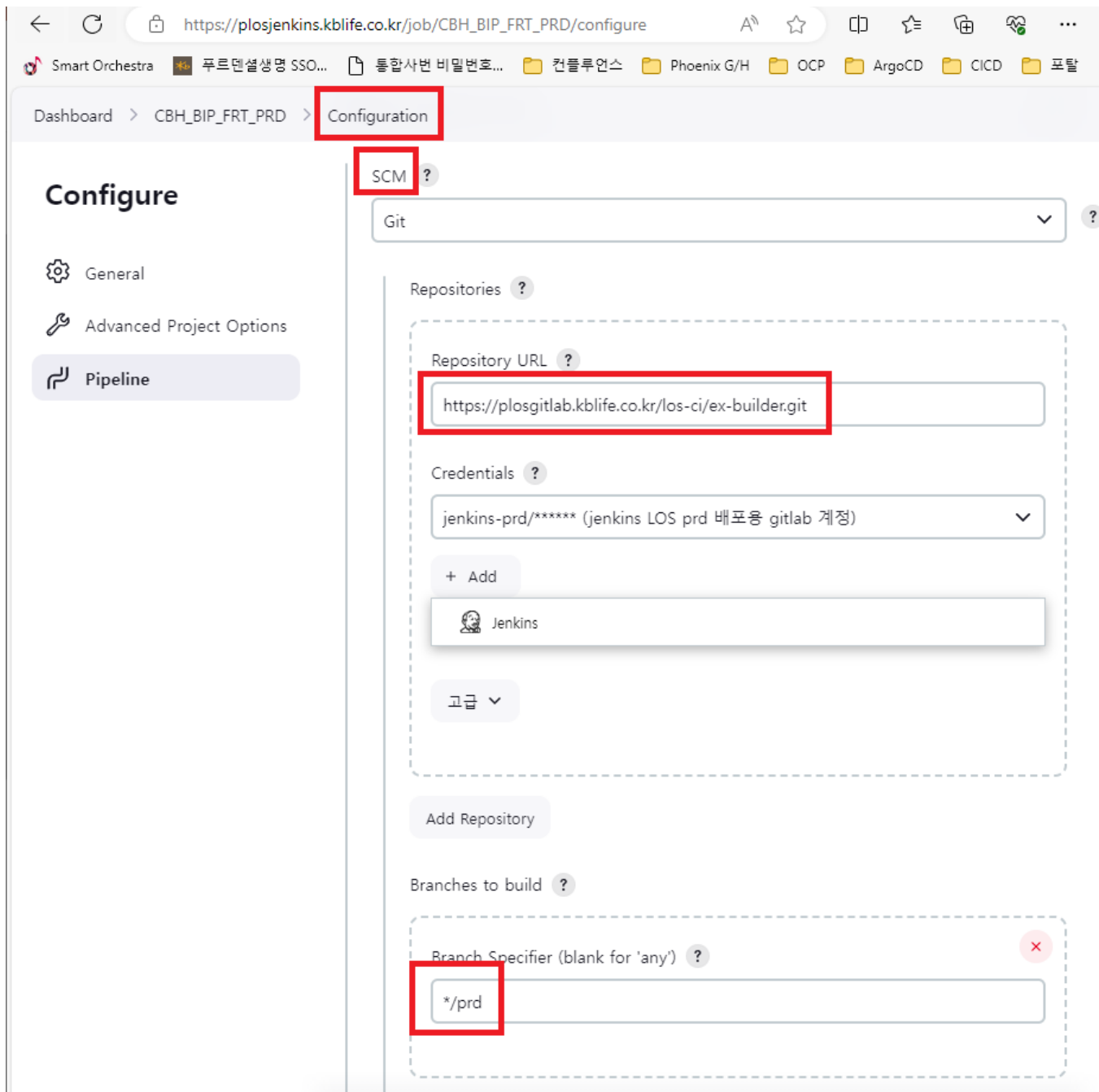
History Find file Edit Code

Add README Add LICENSE Add CHANGELOG Add CONTRIBUTING + Enable Auto DevOps

Add Kubernetes cluster + Set up CI/CD Add Wiki Configure Integrations

Name	Last commit	Last update
copy	Update endpoint.sh	2 days ago
Dockerfile	Update 8 files	2 months ago
Dockerfile.jps	add jps	4 weeks ago
Dockerfile.jsp	Update 8 files	2 months ago
Dockerfile.sfa	Update 8 files	2 months ago
Dockerfile.stc	Update 8 files	2 months ago
Jenkinsfile	Update 2 files	1 week ago
Jenkinsfile.jps	Update 2 files	1 week ago
Jenkinsfile.jsp	Update 5 files	1 month ago
Jenkinsfile.lib	Update Jenkinsfile.lib	5 days ago
Jenkinsfile.non-standard	Update 2 files	4 weeks ago
Jenkinsfile.sfa	Update 5 files	1 month ago

Jenkins job에서 실제로 LOS CI와 연결되는 정보를 관리하는 영역은 아래와 같으며, 해당 정보를 확인함으로 특정 job이 어떤 CI 파이프라인 스크립트로 구동되는지 정확히 확인할 수 있다.



3.2.2. 쿠버네티스 매니페스트 리소스 관리

ArgoCD 배포에 필요한 쿠버네티스 환경설정 yaml 리소스 제공

URL : <https://plogitlab.kblife.co.kr/los-cd>

- Project Name : [시스템 코드 - 업무 코드 - 서비스 종류 / 쿠버네티스 리소스 명] 총 64 종
- Branch : prd / uat / d.. 로 구성되며, 각각의 환경에 맞게 구분 (**dev** 본개발, **d02** > #2 개발환경, **d03** > #3 개발환경)
- Directory : 네임스페이스 별 [시스템 코드] 로 총 6개 시스템 구분
- Yaml : istio-system/*.yaml, virtual-service/*.yaml, service.yaml, deployment.yaml, config-map/*.yaml, secret/*.yaml, horizontal-pod-autoscalers/*.yaml

CD 1레벨 그룹 / 프로젝트 목록

하위 그룹은 ArgoCD 또는 pod 명의 맨 앞 부분(시스템 코드 영역)으로 확인할 수 있다.

	그룹 / 프로젝트 명	설명	URL	비고
1	CBH	LOS-CBH-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/cbh	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
2	CDP	LOS-CDP-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/cdp	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
3	CHN	LOS-CHN-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/chn	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
4	GLB	LOS-GLB-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/glb	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
5	PFG	LOS-PFG-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/pfg	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
6	UNT	LOS-UNT-* 네임스페이스에 배포되는 서비스 gitlab 프로젝트 그룹	https://plogitlab.kblife.co.kr/los-cd/unt	Gitlab 그룹으로 서브 프로젝트에서 deployment.yaml 과 service.yaml 관리
7	CONFIG-MAP	LOS-* 네임스페이스 공통으로 적용되는 시스템 환경 변수인 config map 리소스 프로젝트	https://plogitlab.kblife.co.kr/los-cd/config-map	
8	HORIZONTAL-POD-AUTOSCALERS	LOS-* 네임스페이스 공통으로 적용되는 auto scale 설정인 config map 리소스 프로젝트	https://plogitlab.kblife.co.kr/los-cd/horizontal-pod-autoscalers	
9	ROUTE	LOS-* 네임스페이스 공통으로 적용되는 외부 → 내부 유입 설정인 route 리소스 프로젝트	https://plogitlab.kblife.co.kr/los-cd/route	
10	SECRET	LOS-* 네임스페이스 공통으로 적용되는 시크릿 시스템 환경 변수인 secret 리소스 프로젝트	https://plogitlab.kblife.co.kr/los-cd/secret	
11	VIRTUAL-SERVICE	LOS-* 네임스페이스 공통으로 적용되는 Route → Pod 유입 설정인 virtual service 리소스 프로젝트	https://plogitlab.kblife.co.kr/los-cd/virtual-service	

왼쪽 상단 메뉴 > Groups > Explore groups > LOS CD 그룹 > 확인을 할 그룹 내 프로젝트 / 프로젝트 클릭

Explore groups · GitLab

https://plogitlab.kblife.co.kr/explore/groups?page=3

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

Search or go to...

Explore

Projects

Groups

CI/CD Catalog

Topics

Snippets

Help

Admin Area

Explore / Groups

	Z-CMN-MEMBERS				
	Z-CLM-MEMBERS				
	Z-PAY-MEMBERS				
	Z-DEP-MEMBERS				
	Z-VAR-MEMBERS				
	Z-POS-MEMBERS				
	Z-NBS-MEMBERS				
	Z-VOC-MEMBERS				
	Z-DBA-MEMBERS				
	LOS UNT				
	LOS CD				
	LOS CI				
	LOS GLB				
	LOS CDP				
	LOS PFG				

< Prev

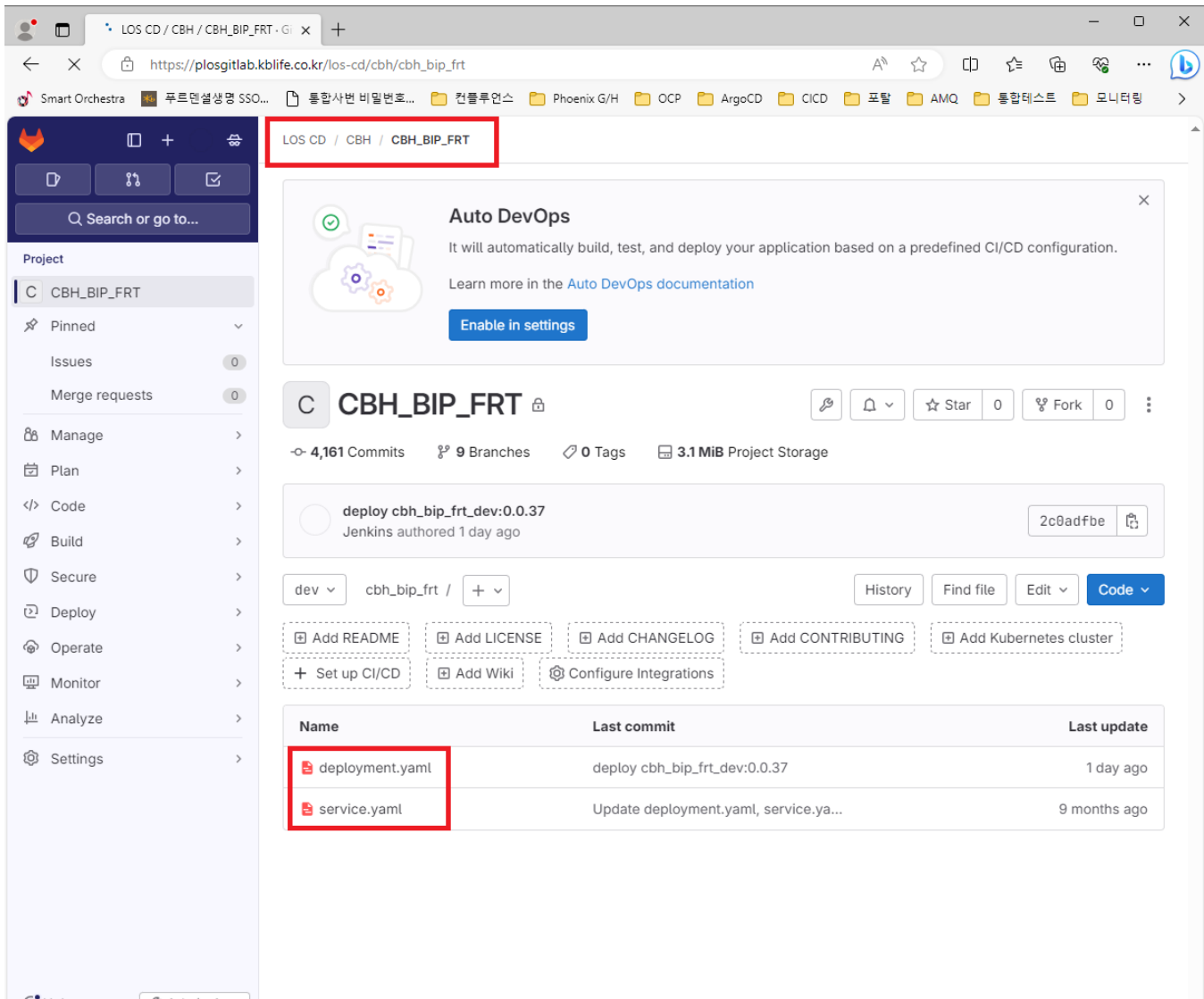
1

2

3

4

Next >



3.2.3. Gitlab 프로젝트 생성 제한

Administrator 권한의 관리자만 프로젝트를 생성할 수 있도록 설정 필요 (Track#1 고객 요구 사항)

- 그룹 설정과 사용자 설정을 통해 프로젝트 생성 제한 설정이 가능 (그룹 생성 / 사용자 생성시 default 로 아래 권한 적용되게 구성 되어있음)
- 한 그룹내에 프로젝트 및 서브 그룹 생성을 제한하도록 설정
- 사용자별로 프로젝트 및 그룹 생성을 제한하도록 설정

3.2.3.1. 그룹내 프로젝트 및 서브 그룹 생성 제한 설정

왼쪽 상단 메뉴 > Admin > Overview > Groups > 설정할 그룹 클릭 > Edit 클릭

아래와 같이 권한 설정 (기본적으로 group 생성시 아래와 같이 설정되도록 default 값으로 구성 되어있음)

1. Allowed to create projects : Maintainers
2. Allowed to create subgroups : Maintainers

그룹의 Maintainer 권한은 gitlab 관리자만 부여하여 관리자만 project / subgroup 생성 가능하도록 설정

사용자는 무조건 그룹 권한을 Reporter 로 지정

Groups - GitLab

https://plogitlab.kblife.co.kr/dashboard/groups

Smart Orchestra

푸르덴셜생명 SSO...

통합사번 비밀번호...

컨플루언스

Phoenix G/H

OCP

ArgoCD

CICD

포탈

AMQ

통합테스트

모니터링

Groups

Projects

Issues

Merge requests

To-Do List

Milestones

Snippets

Activity

Search or go to...

Groups

Explore groups

New group

Search by name

Last created

Z	Z-SMP-MEMBERS	Maintainer	0	0	3	
L	LOS FCP		0	5	3	
C	CYB		0	24	32	
M	MIRROR KB PRD		4	0	1	
Z	Z-ADMIN-MEMBERS	Owner	0	0	12	
L	LOS UNT		0	34	6	
L	LOS CD		6	5	4	
L	LOS CI		0	24	4	
L	LOS GLB		0	9	4	
L	LOS CDP		0	8	3	
L	LOS PFG		0	8	3	
L	LOS CHN		0	12	3	
L	LOS CBH		0	31	3	
L	LG CNS		0	4	3	

Help

Admin Area

Groups · Admin Area · GitLab

https://plogitlab.kblife.co.kr/admin/groups

Smart Orchestra 푸르연설생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

Admin Area / Groups

Search by name Last created New group

Search or go to...

Admin Area

- Overview
- Dashboard
- Projects
- Users
- Groups
- Topics
- GitLab Servers
- CI/CD
- Analytics
- Monitoring
- Messages
- System Hooks
- Applications
- Abuse Reports
- Deploy Keys
- Labels
- Settings

L	LG CNS	378.8 MiB	4	3		Edit	Delete
L	LOS CBH	1.2 GiB	31	3		Edit	Delete
L	LOS CHN	1.5 GiB	12	3		Edit	Delete
L	LOS PFG	426.5 MiB	8	3		Edit	Delete
L	LOS CDP	165.6 MiB	8	3		Edit	Delete
L	LOS GLB	223.9 MiB	9	4		Edit	Delete
L	LOS CI	1.3 GiB	24	4		Edit	Delete
L	LOS CD	490 KIB	5	4		Edit	Delete
L	LOS UNT	4.2 GiB	34	6		Edit	Delete
Z	Z-DBA-MEMBERS	0 B	0	4		Edit	Delete
Z	Z-VOC-MEMBERS	0 B	0	11		Edit	Delete
Z	Z-NBS-MEMBERS	0 B	0	48		Edit	Delete
Z	Z-POS-MEMBERS	0 B	0	50		Edit	Delete
Z	Z-VAR-MEMBERS	0 B	0	42		Edit	Delete
Z	Z-DEP-MEMBERS	0 B	0	59		Edit	Delete

Smart Orchestra

푸르덴셜생명 SSO...

통합사번 비밀번호...

컨플루언스

Phoenix G/H

OCF

ArgoCD

CICD

포탈

AMQ

통합테스트

모니터링

Admin Area

Groups

Groups

Overview

Dashboard

Projects

Users

Groups

Topics

Gitaly Servers

CI/CD

Analytics

Monitoring

Messages

System Hooks

Applications

Abuse Reports

Deploy Keys

Labels

Settings

Admin Area / Groups / LOS GLB

Group: LOS GLB

Edit

Group info:

L

Name: LOS GLB

Path: los-**glb**

Description:

Visibility level: Internal

Created on: Jan 20, 2023 1:47pm

ID: 112

Storage: 223.9 MiB (Repository: 223.9 MiB / Wikis: 31 KiB / Build Artifacts: 0 B / Pipeline Artifacts: 0 B / LFS: 0 B / Snippets: 0 B / Packages: 0 B / Uploads: 0 B)

Group Git LFS status: Enabled for all projects

Projects 9

LOS GLB / GLB_CBH_BND 2.4 MiB

los-glb/glb_cbh_bnd.git

LOS GLB / GLB_DBE_BND 110.9 MiB

los-glb/glb_dbe_bnd.git

LOS GLB / GLB_PUB_FRT 72.7 MiB

LOS GLB group members 3

Manage access

msjeong @msjeong

Given access 1 year ago

Owner

Jenkins @jenkins

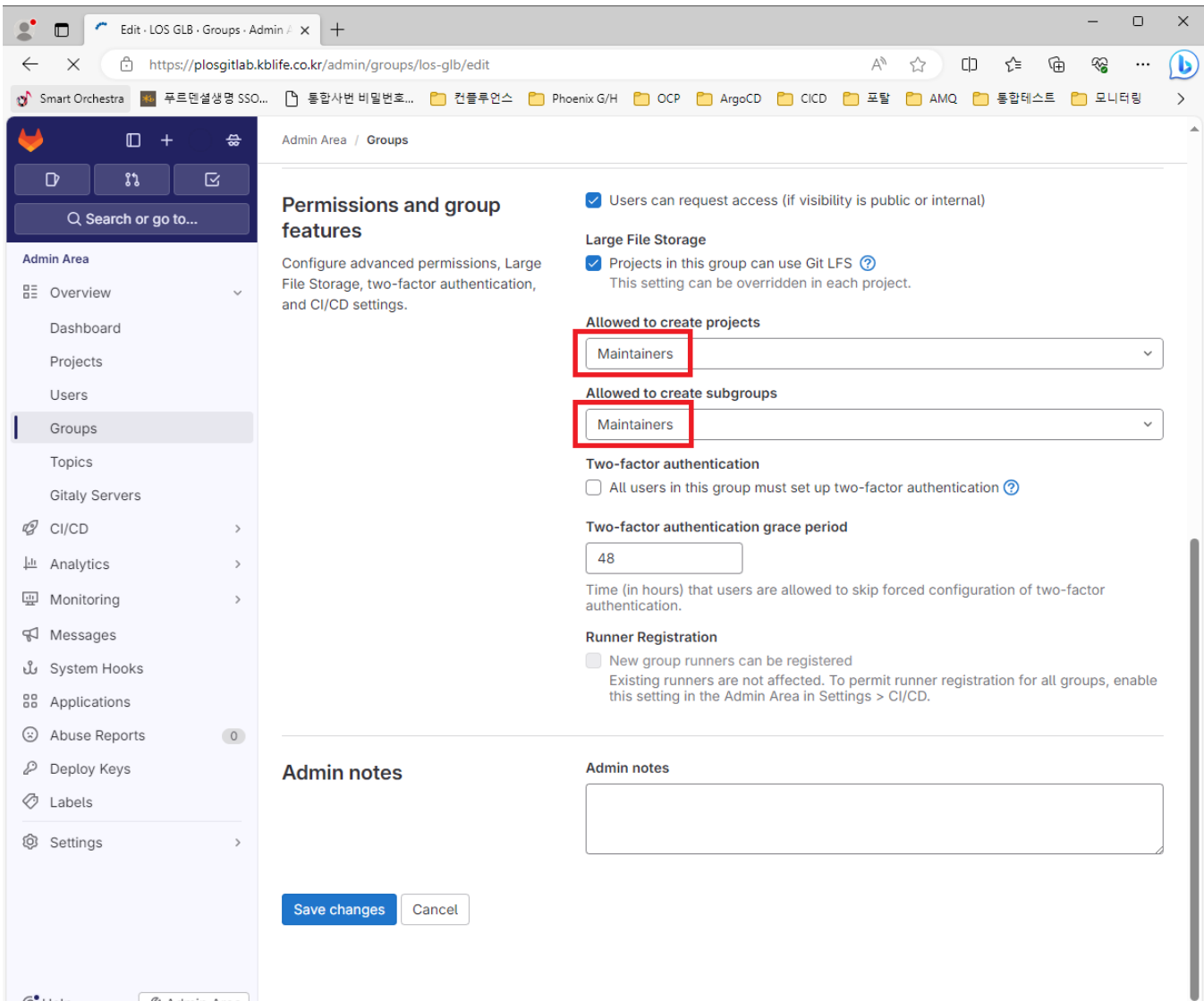
Given access 1 year ago

Maintainer

Jenkins 운영 @jenkins-prd

Given access 1 week ago

Maintainer



3.2.3.2. 사용자별 프로젝트 및 그룹 생성 제한 설정

왼쪽 상단 메뉴 > Admin > Overview > Users > 설정할 사용자 클릭 > Edit 클릭

아래와 같이 권한 설정 (기본적으로 사용자 생성시 아래와 같이 설정되도록 default 값으로 세팅 되어있음)

1. Project limit : 0
2. Can create group : 체크 해제

해당 사용자는 더이상 그룹이나 프로젝트를 생성하지 못함

Groups - GitLab

https://plogitlab.kblife.co.kr/dashboard/groups

Smart Orchestra

푸르덴셜생명 SSO...

통합사번 비밀번호...

컨플루언스

Phoenix G/H

OCP

ArgoCD

CICD

포탈

AMQ

통합테스트

모니터링

Groups

Projects

Issues

Merge requests

To-Do List

Milestones

Snippets

Activity

Search or go to...

Groups

Explore groups

New group

Search by name

Last created

Z	Z-SMP-MEMBERS	Maintainer	0	0	3
L	LOS FCP		0	5	3
C	CYB		0	24	32
M	MIRROR KB PRD		4	0	1
Z	Z-ADMIN-MEMBERS	Owner	0	0	12
L	LOS UNT		0	34	6
L	LOS CD		6	5	4
L	LOS CI		0	24	4
L	LOS GLB		0	9	4
L	LOS CDP		0	8	3
L	LOS PFG		0	8	3
L	LOS CHN		0	12	3
L	LOS CBH		0	31	3
L	LG CNS		0	4	3

Help

Admin Area

Users · Admin Area · GitLab

https://plogitlab.kblife.co.kr/admin/users

Smart Orchestra

푸르덴셜생명 SSO...

통합사번 비밀번호...

컨플루언스

Phoenix G/H

OCF

ArgoCD

CICD

포탈

AMQ

통합테스트

모니터링

Admin Area

Overview

Dashboard

Projects

Users

Groups

Topics

Gitlab Servers

CI/CD

Analytics

Monitoring

Messages

System Hooks

Applications

Abuse Reports

Deploy Keys

Labels

Settings

Admin Area / Users

Users

Cohorts

Active 399

Admins 15

2FA Enabled 1

2FA Disabled 501

External 0

Blocked 1

Banned 0

New user

Search by name, email, or username

Sort by Name

Name	Projects	Groups	Created on	Last activity	
AA파트 Admin los-aa@kblife.co.kr	367	59	Feb 01, 2023	Mar 07, 2024	Edit
Administrator Admin admin@example.com	364	37	Dec 21, 2022	Mar 07, 2024	Edit
apim apim1@prudential.co.kr	4	3	Aug 17, 2023	Feb 15, 2024	Edit
apiPortal Bot project418_bot1@noreply.dlogitlab.kbli fe.co.kr	1	1	Nov 15, 2023	Nov 15, 2023	Edit
ArgoCD argocd@kblife.co.kr	78	7	Jan 25, 2023	Mar 07, 2024	Edit
DBA파트 los-dba@sample.com	106	8	Feb 02, 2023	Feb 16, 2024	Edit
FEP 파트 계정 unt-fep@kblife.co.kr	1	2	Feb 27, 2023	Mar 04, 2024	Edit
for-sparrow for-sparrow@kblife.co.kr	258	28	Mar 07, 2024	Mar 07, 2024	Edit
FW파트 los-fw@kblife.co.kr	233	27	Feb 01, 2023	Mar 06, 2024	Edit

강경우 · Users · Admin Area · Gitl

https://plogitlab.kblife.co.kr/admin/users/la926

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

Admin Area

Overview

Dashboard

Projects

Users

Groups

Topics

Gitaly Servers

CI/CD

Analytics

Monitoring

Messages

System Hooks

Applications

Abuse Reports

Deploy Keys

Labels

Settings

HelpAdmin Area

Admin Area / Users / 강경우

강경우

ImpersonateNew identityEdit

AccountGroups and projectsSSH keysIdentitiesImpersonation Tokens

강경우



Profile page: la926

Account:

Name: 강경우

Username: la926

Email: la926@prudential.co.kr Verified

ID: 463

Namespace ID: 1008

Two-factor Authentication: Disabled

External User: No

Can create top level groups: No

Private profile: No

Personal projects limit: 0

Member since: Oct 4, 2023 11:14am

Confirmed at: Oct 4, 2023 11:14am

Profile

Member since Oct 4, 2023 11:14am

Admin Area / Users

Edit user: 강경우

Account

Name: 강경우

Username: la926

Email: la926@prudential.co.kr

Password

Password:

Password confirmation:

Access

Projects limit: 0

☒ Can create top level group

☐ Private profile

Access level: Regular

Regular users have access to their groups and projects.

[Save changes](#) [Cancel](#)

3.3. Jenkins 주요 관리

Jenkins 운영을 위한 주요 설정 내용을 기술한다. (Administrator 권한의 계정으로만 설정 가능)

Jenkins 운영을 위한 주요 설정은 "Jenkins 관리" 기능을 이용해 설정한다.

Dashboard >

+ 새로운 Item

사람

빌드 기록

프로젝트 연관 관계

파일 핑거프린트 확인

Jenkins 관리

My Views

P4 Trigger

새로운 뷰

빌드 대기 목록

빌드 대기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

5 대기 중

6 대기 중

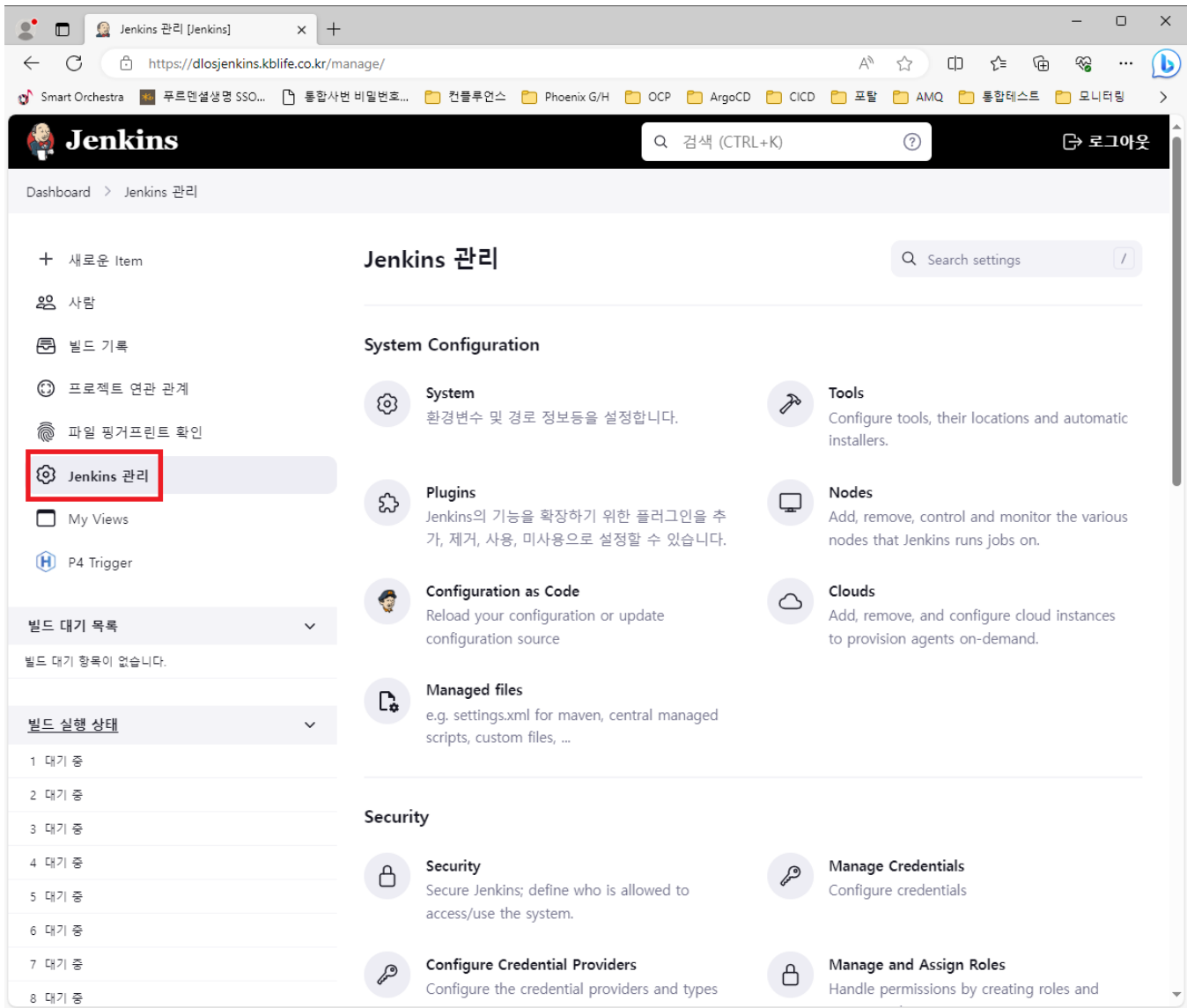
7 대기 중

상세 내용 입력

All LOS-CBH LOS-CDP LOS-CHN LOS-FEP LOS-GLB LOS-PFG LOS-UNT TMP

+

S	W	Name ↓	최근 성공	최근 실패	최근 소요 시간	Built On	Coverage	# Issues
✓		0. ALL_REGULAR_MERGE_AND_BUILD	5 hr 8 min #551	3 days 5 hr #547	18 min	Jenkins		n/a -
✓		1. ALL_GIT_MERGE	5 min 3 sec #749	—	37 sec	Jenkins		n/a -
✓		2. ALL_BUILD_JOB_PRD	5 hr 6 min #564	2 days 0 hr #561	17 min	Jenkins		n/a -
✓		CBH_BIP_FRT_PRD	5 hr 6 min #670	—	11 min			n/a -
✓		CBH_BKA_BAT_PRD	5 hr 6 min #577	—	1 min 10 sec			n/a -
✓		CBH_BKA_BND_PRD	3 min 15 sec #604	—	1 min 37 sec			n/a -
✓		CBH_BPS_BND_PRD	3 min 11 sec #588	—	1 min 30 sec			n/a -
✓		CBH_CBH_BAT_PRD	5 hr 6 min #589	—	3 min 56 sec			n/a -



3.3.1. 시스템 설정

3.3.1.1. Global properties

Jenkins 관리 > 시스템 설정 > Global properties > Environment variables

CI/CD 파이프라인에서 사용할 전역 변수 설정이며 CI/CD 에 필요한 아래 정보들을 환경 변수로 정의한다.

Configure System [Jenkins] x +

https://plosjenkins.kblife.co.kr/configure

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ

Dashboard > Configure System >

Global properties

☐ Disable deferred wipeout on this node ?

☒ Environment variables

키-값 목록 ?

이름

APPLICATION_TYPE_IDX

값

9

이름

BIZ_IDX

값

5

이름

BRANCH_IDX

저장 Apply

전역으로 설정된 환경 변수 목록

	변수	설정된 값	설명	비고
1	SYSTEM_IDX	1	표준 Jenkins job name 에서 시스템 코드가 시작하는 index 값	
2	BIZ_IDX	5	표준 Jenkins job name 에서 업무 코드가 시작하는 index 값	
3	APPLICATION_TYPE_IDX	9	표준 Jenkins job name 에서 서비스 종류 코드가 시작하는 index 값	
4	BRANCH_IDX	13	표준 Jenkins job name 에서 브랜치 명이 시작하는 index 값	
5	GITLAB_URL	plogitlab.kblife.co.kr	사용하는 gitlab 주소 (application 소스, CD yaml)	DNS 만 설정
6	NEXUS_IMAGE_DOWNLOAD_URL	plosnexus.kblife.co.kr:5000	이미지 다운로드하는 nexus 주소	DNS 와 port 값 모두 설정
7	NEXUS_IMAGE_UPLOAD_URL	plosnexus.kblife.co.kr:8010	이미지 업로드하는 nexus 주소	DNS 와 port 값 모두 설정
8	\${시스템 코드}_\${업무 코드}	LOS	업무별 구축한 프로젝트 코드	프로젝트 명 : LOS

전역으로 설정된 환경 변수 사용하는 Jenkinsfile 예시

```
pipeline {
    ...

    environment {
        ...

        PROJECT_KEY = "${sh(script: 'echo ${JOB_NAME} | cut -c ${SYSTEM_IDX}-${(BIZ_IDX + 2)}', returnStdout:
true).trim()}"
        SYSTEM_CODE = "${sh(script: 'echo ${JOB_NAME} | cut -c ${SYSTEM_IDX}-${(SYSTEM_IDX + 2)} | awk \'{print
tolower($0)}\'', returnStdout:true).trim()}"
        BIZ_CODE = "${sh(script: 'echo ${JOB_NAME} | cut -c ${BIZ_IDX}-${(BIZ_IDX + 2)} | awk \'{print tolower
($0)}\'', returnStdout:true).trim()}"

        ...
    }
    stages {
        stage("Prepare"){
            environment {
                GITLAB_CREDS = credentials('2088cac1-c042-4946-adece-75e9e35c366d')
                GITLAB_REPO_NAME = "${SYSTEM_CODE}-${BIZ_CODE}-${APPLICATION_TYPE}"
                GITLAB_APPLICATION_GROUP = "${sh(script: 'echo \"${(echo ${PROJECT_KEY} | awk \'{print tolower
($0)}\'')}-${(echo ${SYSTEM_CODE})}\'', returnStdout:true).trim()}"
                GITLAB_CD_GROUP = "${sh(script: 'echo \"${(echo ${PROJECT_KEY} | awk \'{print tolower($0)}\'')
}-${(echo ${CD_CODE})}\'', returnStdout:true).trim()}"
                BRANCH_NAME = "${sh(script: 'echo ${JOB_NAME} | cut -c ${BRANCH_IDX}-${(BRANCH_IDX + 2)}',
returnStdout:true).trim()}"
            }
            steps {
                sh '''
                ...

                git clone https://${GITLAB_CREDS_USR}:${GITLAB_CREDS_PSW}@${GITLAB_URL}
/${GITLAB_APPLICATION_GROUP}/${GITLAB_REPO_NAME}.git -b ${BRANCH_NAME}

                ...
            }
        }
        ...
    }
}
```

3.3.1.2. 동시 빌드 실행할 job 수 조절

Jenkins 관리 > 시스템 설정 > Maven Project Configuration > # of executors

Jenkins 에서 동시에 실행할 job 의 개수를 설정한다.

주의사항: executor 는 jenkins 서버의 리소스 (cpu, memory 등) 수치를 고려하며 설정한다.

현재 특정 환경(dev, uat, prd) 에 대해 전체 서비스(43건) 배포시 cpu 90% ~ 100% 사용률을 사용하는 18개로 설정되어있다.

Dashboard > Configure System >

빌드 대기 항목이 없습니다.

Maven Project Configuration

Global MAVEN_OPTS ?

Local Maven Repository ?

Default ("~/m2/repository", or the value of 'localRepository' in Maven's settings file, if defined)

of executors

18

Labels

Usage ?

Use this node as much as possible

Quiet period ?

5

SCM checkout retry count

0

Default view

저장 Apply

3.3.2. 플러그인 관리

Jenkins 에서 사용할 플러그인을 등록하고 설치된 목록을 확인한다.

3.3.2.1. Deploy Plugin

외부 인터넷 환경에서 jenkins 공식 사이트에 접속하여 plugin 다운로드 후, 내부 반입하여 등록한다.

- <http://updates.jenkins-ci.org/download/plugins> 접속하여 hpi 파일 다운로드
- Hpi 파일 반입 후, Jenkins 관리 > Plugins > Advanced settings > Deploy Plugin > 파일 선택 > Deploy

Advanced settings - Plugins [Jen] x

https://dlosjenkins.kblife.co.kr/manage/pluginManager/advanced

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

Jenkins

검색 (CTRL+K) 로그아웃

Dashboard > Jenkins 관리 > Plugins

Plugins

- Updates
- Available plugins
- Installed plugins
- Advanced settings**

Deploy Plugin

You can select a plugin file from your local system or provide a URL to install a plugin from outside the configured update site(s).

파일

파일 선택 선택된 파일 없음

Or

사이트경로

Deploy

업데이트 사이트

사이트경로

https://updates.jenkins.io/update-center.json

Unable to connect to the URL.
(show details)

Submit

REST API Jenkins 2.426.3

3.3.2.2. Retrieve Plugin

Jenkins 관리 > 플러그인 관리 > 설치된 플러그인 목록 > 검색

Installed plugins - Plugins [Jenkins x] +

← ↻ 🔍 <https://dlosjenkins.kblife.co.kr/manage/pluginManager/installed> A ☆ 📄 📌 📁 ... 🔍

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨틀루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링 >

Jenkins 🔍 검색 (CTRL+K) 👤 정민성 🚪 로그아웃

Dashboard > Jenkins 관리 > Plugins

Plugins

📁 Updates

📁 Available plugins

⚙️ **Installed plugins**

⚙️ Advanced settings

🔍 Search installed plugins

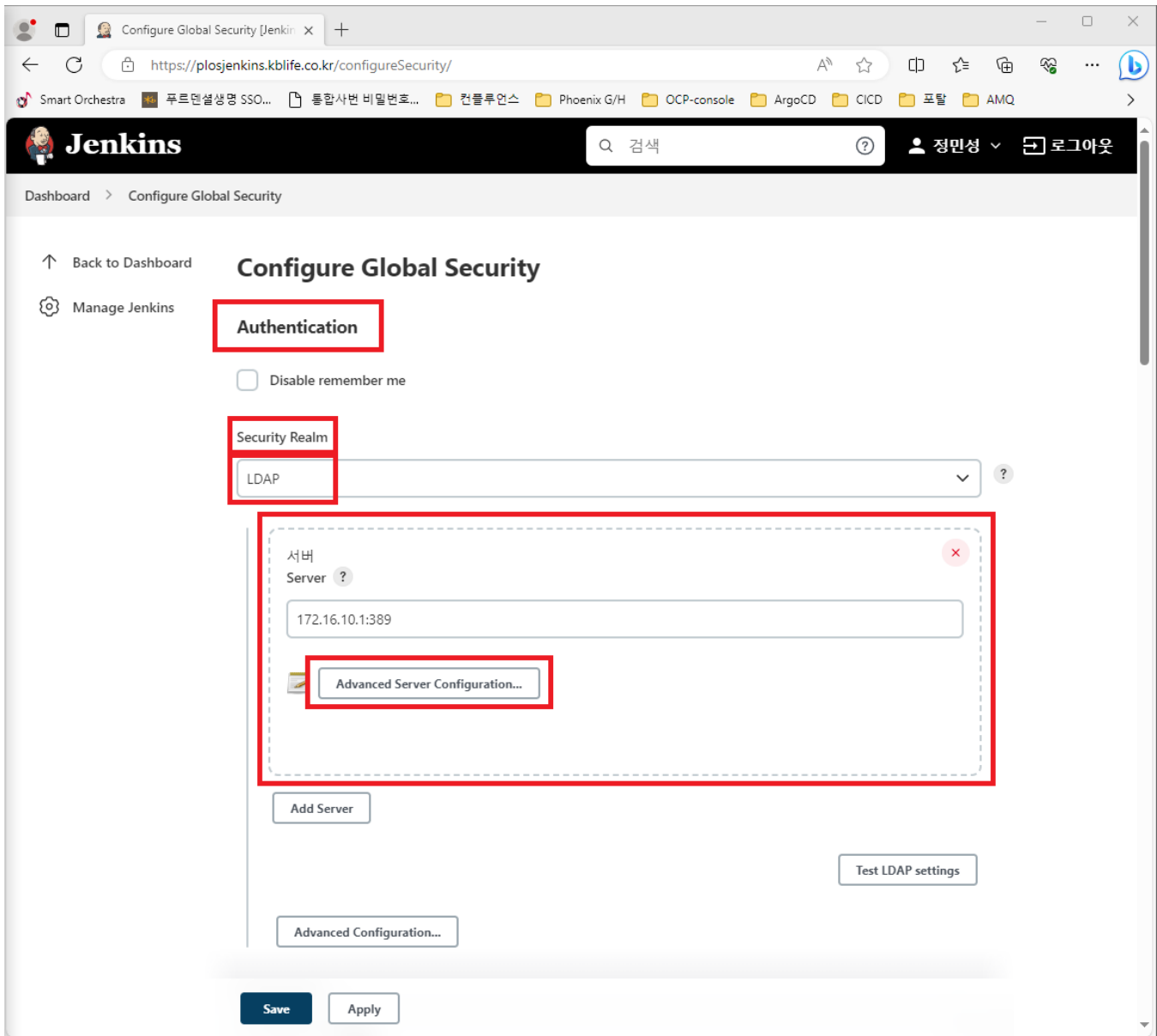
이름 ↓	사용가능
Active Directory plugin 2.29 This plugin enables authentication through Active Directory.	☒
Analysis Model API Plugin 10.17.0 A library wrapped into a Jenkins plugin to read static analysis reports into a Java object model.	☒
Ant Plugin 481.v7b_09e538fcc Adds Apache Ant support to Jenkins	☒
Apache HttpComponents Client 4.x API Plugin 4.5.14-150.v7a_b_9d17134a_5 Bundles Apache HttpComponents Client 4.x and allows it to be used by Jenkins plugins.	☒
Bitbucket Plugin 223.vd12f2bca5430 Integrates with BitBucket	☒
Bootstrap 5 API Plugin 5.3.0-1 Provides Bootstrap 5 for Jenkins Plugins. Bootstrap is (according to their self-perception) the world's most popular front-end component library to build responsive, mobile-first projects on the web.	☒
bouncycastle API Plugin 2.28 This plugin provides a stable API to Bouncy Castle related tasks.	☒
Branch API Plugin 2.1051.v9985666b_f6cc This plugin provides an API for multiple branch based projects.	☒
Build Name and Description Setter 2.2.0 This plugin sets the display name and description of a build to something other than	☒

3.3.3. 사용자 인증 관리

Jenkins 관리 > Configure Global Security > Authentication > Security Realm

- LDAP 선택하여, 사용자 계정을 관리하는 LDAP 서버 설정

사용자 계정을 LDAP 으로 연계하여 위임하여 관리하기에 사용자 추가, 제거, 초기화 등과 같은 작업은 연계된 LDAP 에서 수행한다.



3.3.4. 그룹 권한 관리

Jenkins 관리 > Configure Global Security > Authentication > Authorization

- Role-Based Strategy 선택

Role-Based Strategy 플러그인과 정규 표현식을 활용하여 사용자 권한을 업무 그룹 단위로 관리한다.

Configure Global Security [Jenkin x]

https://pjosjenkins.kblife.co.kr/configureSecurity/

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨트롤루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ

Dashboard > Configure Global Security

Authorization

Role-Based Strategy

Markup Formatter

Markup Formatter

Plain text

Treats all input as plain text. HTML unsafe characters like < and & are escaped to their respective character entities.

Agents

TCP port for inbound agents ?

☐ Fixed

☐ Random

☒ Disable

Agent protocols...

CSRF Protection

Crumb Issuer

Default Crumb Issuer

Save Apply

3.3.4.1. 공통 그룹 권한 생성

Jenkins 관리 > Manage and Assign Roles > Manage Roles

그룹 권한 관리(권한 레벨 관리)는 Manage Roles 에서 관리한다.

Jenkins 관리 [Jenkins] x +

← ↻ 🔍 https://dlosjenkins.kblife.co.kr/manage/ A ☆ 📄 📌 📁 📂 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링 >

Dashboard > Jenkins 관리

Jenkins 관리

🔍 Search settings /

+ 새로운 Item

👤 사람

📁 빌드 기록

🕒 프로젝트 연관 관계

🖨 파일 핑거프린트 확인

⚙ Jenkins 관리

📄 My Views

🔗 P4 Trigger

빌드 대기 목록 ▾

빌드 대기 항목이 없습니다.

빌드 실행 상태 ▾

- 1 대기 중
- 2 대기 중
- 3 대기 중
- 4 대기 중
- 5 대기 중
- 6 대기 중
- 7 대기 중
- 8 대기 중
- 9 대기 중
- 10 대기 중

System Configuration

- System**
환경변수 및 경로 정보등을 설정합니다.
- Plugins**
Jenkins의 기능을 확장하기 위한 플러그인을 추가, 제거, 사용, 미사용으로 설정할 수 있습니다.
- Configuration as Code**
Reload your configuration or update configuration source
- Managed files**
e.g. settings.xml for maven, central managed scripts, custom files, ...

Security

- Security**
Secure Jenkins; define who is allowed to access/use the system.
- Configure Credential Providers**
Configure the credential providers and types
- In-process Script Approval**

Tools

Configure tools, their locations and automatic installers.

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

Clouds

Add, remove, and configure cloud instances to provision agents on-demand.

Manage Credentials

Configure credentials

Manage and Assign Roles

Handle permissions by creating roles and assigning them to users/groups

Job 전체적으로 적용되는 공통 권한은 Global roles 에 의해 관리된다.

새로운 공통 권한을 생성하기 위해선 Global roles 에 새로운 그룹 명을 작성하고 Add 버튼을 클릭한다.

The screenshot shows the Jenkins 'Manage Roles' interface. On the left is a sidebar with navigation links. The main area is titled 'Manage Roles' and contains a table of 'Global roles'. The table has columns for 'Role', 'Overall', 'Credentials', 'Agent', 'Job', and 'Run'. The 'admin' role is selected. Below the table, there is a 'Role to add' field containing 'new_comon_group1' and an 'Add' button.

Role	Overall	Credentials	Agent	Job	Run
admin	☒	☐	☐	☐	☐
everyone	☐	☒	☐	☐	☐

Role to add:

3.3.4.2. 공통 그룹 권한 부여

Jenkins 관리 > Manage and Assign Roles > Manage Roles

Global roles 에서 세로 축에 해당하는 그룹명 중 권한을 부여하고 싶은 그룹명 행에 가로 축에 해당하는 권한 중 권한을 부여하고 싶은 권한의 체크 박스들을 클릭하고 Save 버튼을 클릭한다.

Manage Roles

Global roles

Role	Overall	Credentials	Agent	Job																			
	Administrator	Read	Create	Delete	Manage Domains	Update	View	Build	Configure	Connect	Create	Delete	Disconnect	Provision	Cancel	Configure	Create	Delete	Discover	Move	Read	ViewStatus	Workspace
admin	☒																						
everyone		☒																					
new_comon_group1			☒																				

Role to add

new_comon_group1

Add

Item roles

Filter by Role

Save Apply

본 프로젝트에서 기본 공통 그룹은 다음과 같이 관리된다.

- 공통 권한은 관리자 그룹(admin)과 관리자가 아닌 그룹(everyone)으로 구분한다.
- 관리자 그룹은 Administrator 권한을 부여하고, 관리자가 아닌 그룹은 Read 권한을 부여한다.

The screenshot shows the Jenkins 'Manage Roles' interface. On the left is a sidebar with navigation links like '새로운 Item', '사람', '빌드 기록', etc. The main area is titled 'Manage Roles' and contains a table of 'Global roles'. The table has columns for 'Role', 'Overall', 'Credentials', 'Agent', 'Job', and 'Run'. Under 'Overall', there are 'Administer' and 'Read' sub-columns. Under 'Credentials', there are 'ManageDomains', 'Create', 'Delete', and 'Update'. Under 'Agent', there are 'View', 'Build', 'Configure', 'Connect', 'Create', 'Delete', and 'Disconnect'. Under 'Job', there are 'Provision', 'Build', 'Cancel', 'Configure', 'Create', 'Delete', 'Discover', 'Move', 'Read', 'ViewStatus', 'Workspace', 'Delete', and 'Replay'. Under 'Run', there is a 'Run' column. The 'admin' role is highlighted with a red box and has 'Administer' and 'Read' permissions checked. The 'everyone' role is also highlighted with a red box and has 'Read' permission checked. Below the table is a 'Role to add' input field and an 'Add' button.

3.3.4.3. 업무 그룹 권한 생성

Jenkins 관리 > Manage and Assign Roles > Manage Roles

업무 그룹 권한 관리는 Item roles 에서 관리한다.

새로운 업무 그룹 권한을 생성하기 위해선 Item roles 에 새로운 그룹 이름을 작성하고, 그룹에 해당하는 정규 표현식을 작성하고 Add 버튼을 클릭한다.

- Role to add: 업무 그룹 권한의 그룹 이름을 입력한다.
- Pattern: 그룹에 포함할 정규 표현식을 정의한다.
 - 정규 표현식은 (?i) 를 앞에 작성하고 뒤에 \${시스템 코드}_\${업무 코드}.*_\${서버 환경} 으로 작성한다. "(?i)" 의 의미는 대소문자를 무시한다는 의미이고, ".*" 은 이후 모든 문자를 의미한다.
 - 서버 환경은 DEV / UAT 환경에서만 사용한다.
 - 예시: (?i)cbh_cmnm_.*_dev, (?i)cbh_cmnm_.*_uat 또는 (?i)cbh_cmnm_.*

Dashboard > Manage and Assign Roles

Role	Pattern
PFM 생성	(?)/ptg_*
PFM PFMS	(?)unt_pfm_*
POS 계약보전	(?)cbh_pos_ (?cbh_cbh_bat_ (?cbh_pgh_ (?cbh_bip_*
RPT 리포트	(?)unt_rpt_*
VAR 번역	(?)cbh_var_ (?cbh_cbh_bat_ (?cbh_pgh_ (?cbh_bip_*
VOC VOC	(?)chn_voc_ (?cbh_pgh_ (?cbh_bip_*
구KB생명 RKS	(?)unt_rks_ (?cyb_*
대응개발 CCT	(?)cbh_cct_ (?cbh_mhc_*
대응개발 SFA	(?)chn_sfa_ (?chn_stc_ (?chn_pmd_ (?unt_ssp_ (?unt_mic_ (?unt_omn_ (?unt_eap_ (?u
대응개발 TMS 텔레마케팅	(?)chn_tms_*

Role to add

new_group1

Pattern ?

(?)시스템코드_업무코드_*_서버환경

Add

Node roles

Save Apply

3.3.4.4. 업무 그룹 권한 부여

Jenkins 관리 > Manage and Assign Roles > Manage Roles

Item roles 에서 세로 축에 해당하는 그룹 명 중 권한을 부여하고 싶은 그룹 명 행에 가로 축에 해당하는 권한 중 권한을 부여하고 싶은 권한의 체크 박스들을 클릭하고 Save 버튼을 클릭한다.

Item roles

Filter by Role

Role	Pattern	Credentials				Job									
		Create	Delete	Update	View	Build	Cancel	Configure	Create	Delete	Discover	Move	Read	View/Status	Workspace
AMP API포탈	"(?i)unt_amp_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
AMQ 카프카	"(?i)unt_amq_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
BKA 방카	"(?i)cbh_bka_.*"(?)cbh_bps_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
BRE 업무	"(?i)unt_efs_.*"(?)unt_efm_.*"(?)unt_rbs_.*"(?)unt_rbm_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
CDP 고객데이터	"(?i)cdp_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
CLM 클레임	"(?i)cbh_clm_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
CMN 업무공통	"(?i)cbh_cm_.*"(?)cbh_cme_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
DEP 입금	"(?i)cbh_dep_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
FEP 대외연계	"(?i)fep_.*"	☐	☐	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐
IAP 통합인증	"(?i)cbh_iap_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
IMG 이미지	"(?i)unt_img_.*"(?)unt_cvt_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
JPS 자바PX	"(?i)cbh_jps_.*"(?)cbh_jpm_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
NBS 신계약	"(?i)cbh_nbs_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
PAY 제지급	"(?i)cbh_pay_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
PFG 상품	"(?i)pfg_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
PFM PFMS	"(?i)unt_pfm_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
POS 계약보전	"(?i)cbh_pos_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
RPT 리포트	"(?i)unt_rpt_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
VAR 변액	"(?i)cbh_var_.*"(?)cbh_cbh_bat_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
VOC VOC	"(?i)chn_voc_.*"(?)cbh_pgh_.*"(?)cbh_bip_.*"	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
구KB생명 RKS	"(?i)unt_rks_.*"(?)cyb_.*"	☐	☐	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐

Save Apply

3.3.5. 사용자 권한 관리

Jenkins 관리 > Configure Global Security > Authentication > Authorization

- **Role-Based Strategy** 선택 (3.3.4. 그룹 권한 관리에서 기 선택한 방법을 사용한다.)

Role-Based Strategy 플러그인과 정규 표현식을 활용하여 사용자 권한을 업무 그룹 단위로 관리한다.

Dashboard > Configure Global Security

Authorization

Role-Based Strategy

Markup Formatter

Markup Formatter

Plain text

Treats all input as plain text. HTML unsafe characters like < and & are escaped to their respective character entities.

Agents

TCP port for inbound agents ?

☐ Fixed

☐ Random

☒ Disable

Agent protocols...

CSRF Protection

Crumb Issuer

Default Crumb Issuer

Save Apply

3.3.5.1. 관리자 권한 관리(권한 부여 / 회수)

Jenkins 관리 > Manage and Assign Roles > Assign Roles

사용자 권한 관리(권한 부여 / 회수)는 Assign Roles 에서 관리한다.

Assign Roles [Jenkins] x +

← ↻ https://dlosjenkins.kblife.co.kr/manage/role-strategy/assign-roles

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP ArgoCD CICD 포탈 AMQ 통합테스트 모니터링

Jenkins 🔍 검색 (CTRL+K) 👤 정민성 🚪 로그아웃

Dashboard > Jenkins 관리 > Manage and Assign Roles

🔑 Manage Roles

Assign Roles

⚙️ Role Strategy Macros

Assign Roles

Global roles

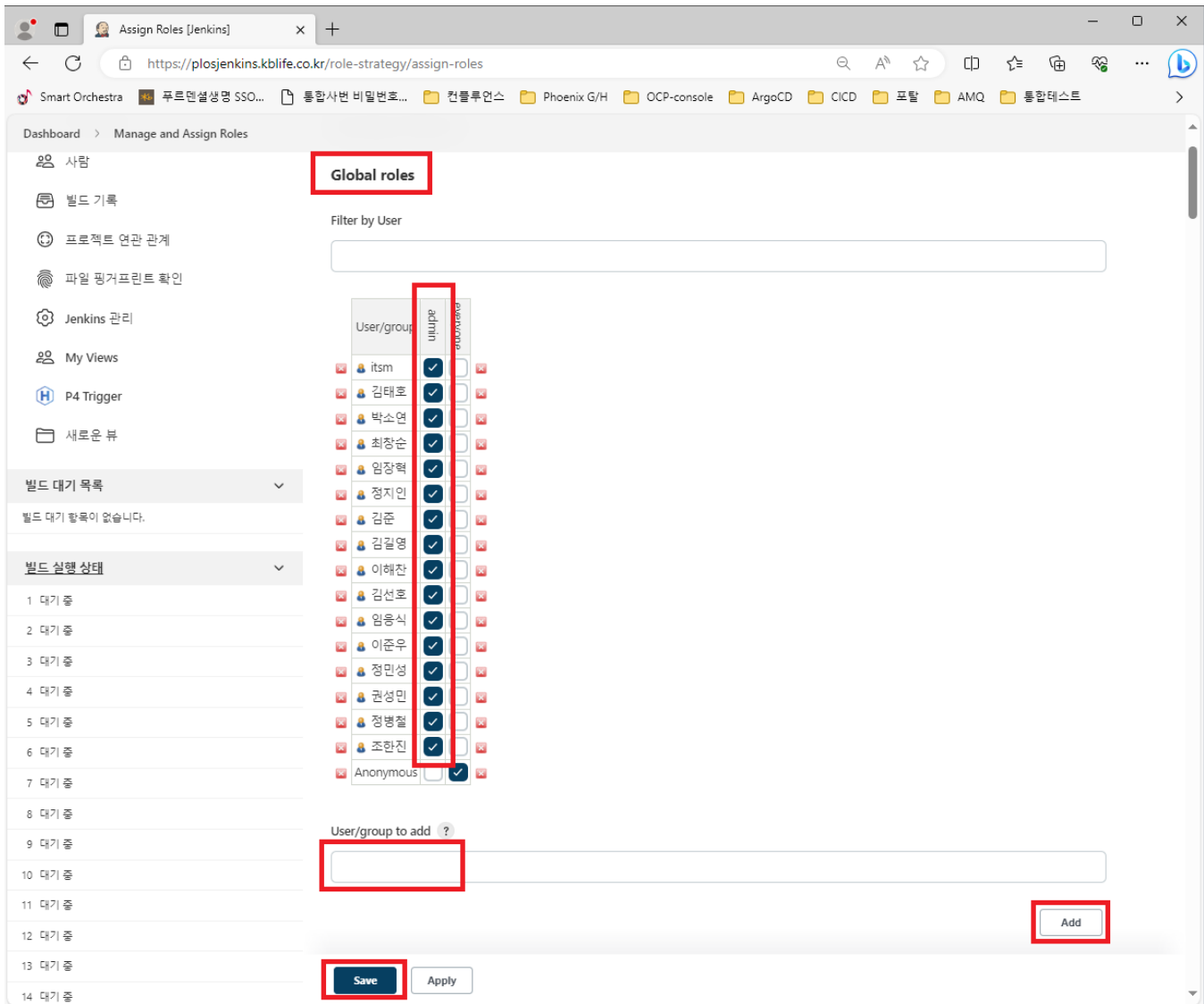
Filter by User

User/group	admin	everyone
Anonymous	☐	☐
authenticated	☒	☒
itsm	☒	☐
신종민	☒	☐
신희옥	☒	☐
박소연	☒	☐
김진태	☒	☐
이철우	☒	☐
권은동	☒	☐
정지인	☒	☐
la765	☒	☐
op517	☒	☐
op921	☒	☐
op922	☒	☐
op928	☒	☐

Save Apply

Jenkins 시스템 전체적으로 적용되는 관리자 권한은 Global roles 에 의해 관리된다.

- User/group to add: 관리자로 등록할 사용자의 ID(사번) 입력 후 Add 버튼 클릭한다. (사용자의 ID(사번) 입력시 **반드시 소문자로 입력**한다.)
- 테이블에서 해당 사용자의 체크 박스를 권한 부여시 체크 / 권한 회수시 체크 해제하여 Save 버튼 클릭한다.



3.3.5.2. 업무 권한 관리(권한 부여 / 회수)

Jenkins 관리 > Manage and Assign Roles > Assign Roles

권한 부여 페이지에 들어오면 Jenkins는 자동으로 ldap과 연동하여 사번을 ldap의 계정명으로 sync를 맞추는 작업을 진행한다.

위와 같은 작업으로 인해 페이지가 상당히 버벅거리며, 아주 많은 권한 목록에서 특정 사용자에게 권한을 부여하기란 쉽지 않을 수 있다.

따라서, **권한 부여 / 회수 시 아래의 Filter를 사용하여 작업하는 것을 적극 권장**한다.

- **Filter by User**: 사용자로 필터하는 것으로, 사용자 이름 or 사용자 사번을 입력한다. (ldap sync가 모두 이루어지지 않았을 수 있으므로 사용자 이름 / 사용자 사번 모두 입력해본다.)
- **Filter by Role**: 권한을 부여 / 회수 할 파트 코드를 입력한다. (예: 공통 업무 = cmn, 입금 = dep 등)

개발 Jenkins에서 권한 부여시 **PL 권한은 "PL - [업무코드] [업무명]"**으로 된 항목에 체크박스를 선택하고, **개발자 권한은 "개발자 - [업무코드] [업무명]"**으로 된 항목에 체크박스를 선택하여 권한 부여한다.

참고 사항

- PL - [업무코드] [업무명] 그룹: DEV, UAT job 빌드 권한 부여
- 개발자 - [업무코드] [업무명] 그룹: DEV job 빌드 권한만 부여

파이프라인 내에서 사용할 특정 데이터(계정, 시크릿 등)를 관리한다.

Jenkins 관리 [Jenkins]

https://pjosjenkins.kblife.co.kr/manage

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Dashboard >

할스 내기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

5 대기 중

6 대기 중

7 대기 중

8 대기 중

9 대기 중

10 대기 중

11 대기 중

12 대기 중

13 대기 중

14 대기 중

15 대기 중

16 대기 중

17 대기 중

18 대기 중

Security

Configure Global Security

Secure Jenkins; define who is allowed to access/use the system.

Configure Credential Providers

Configure the credential providers and types

In-process Script Approval

Allows a Jenkins administrator to review proposed scripts (written e.g. in Groovy) which run inside the Jenkins process and so could bypass security restrictions. **1 scripts pending approval.**

Manage Credentials

Configure credentials

Manage and Assign Roles

Handle permissions by creating roles and assigning them to users/groups

Status Information

시스템 정보

문제 해결을 돕기위한 다양한 환경 정보를 보여줍니다.

부하 통계

Check your resource utilization and see if you need more computers for your builds.

Active Directory Health Status

System Log

System log captures output from `java.util.logging` output related to Jenkins.

About Jenkins

See the version and license information.

Jenkins » Credentials [Jenkins]

← → ↺ https://pjosjenkins.kblife.co.kr/credentials/ Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Dashboard > Credentials

빌드 대기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

5 대기 중

6 대기 중

7 대기 중

8 대기 중

9 대기 중

10 대기 중

11 대기 중

12 대기 중

13 대기 중

14 대기 중

15 대기 중

16 대기 중

17 대기 중

18 대기 중

Jenkins

(global)

8c185007-3c8c-444b-9315-dbf28ff5aa78

Git

Jenkins

(global)

GITLAB_INT_DEV

GITLAB_INT_DEV

Jenkins

(global)

GITLAB_INT_UAT

GITLAB_INT_UAT

Jenkins

(global)

GITLAB_INT_PRD

GITLAB_INT_PRD

Jenkins

(global)

TAS_INT_DEV

TAS_INT_DEV

Jenkins

(global)

TAS_INT_UAT

TAS_INT_UAT

Jenkins

(global)

TAS_INT_PRD

TAS_INT_PRD

Jenkins

(global)

NEXUS_INT

NEXUS_INT

Jenkins

(global)

2088cac1-c042-4946-0000-75e9e35c366d

jenkins/***** (prd gitlab)

Stores scoped to Jenkins

P Store Domains

Jenkins

(global)

아이콘: S M L

REST API Jenkins 2.346.1

System [Jenkins]

https://pjosjenkins.kblife.co.kr/credentials/store/system/

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

검색

정민성

로그아웃

Dashboard > Credentials > System >

+ 새로운 Item

사람

빌드 기록

프로젝트 연관 관계

파일 핑거프린트 확인

Jenkins 관리

My Views

P4 Trigger

새로운 뷰

빌드 대기 목록

빌드 대기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

5 대기 중

6 대기 중

7 대기 중

System

Domain ↓	Description
 Global credentials (unrestricted)	Credentials that should be available irrespective of domain specification to requirements matching.

아이콘: S M L

System > Global credentials (unre

https://plosjenkins.kblife.co.kr/credentials/store/system/domain/

Smart Orchestra

푸르덴셜생명 SSO...

통합사번 비밀번호...

컨플루언스

Phoenix G/H

OCP-console

ArgoCD

CICD

포탈

AMQ

통합테스트

Dashboard > Credentials > System > Global credentials (unrestricted) >

↑ Back to credential domains

+ Add Credentials

Global credentials (unrestricted)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
2088cac1-c042-4946-adec-75e9e35c366d	jenkins/*****	Username with password	
22626871-9a6e-456a-944d-1cd0d2707e0e	nexus pw	Secret text	nexus pw
87204c27-cdeb-48af-a177-e426ebf4812f	nexus id	Secret text	nexus id
fe1fc089-b606-498e-a3ee-255aac95fd9b	solution home	Secret text	solution home
0002a3ba-62ef-491e-ab8a-b9923d6b0a20	배치 devonboot 계정 password	Secret text	배치 devonboot 계정 password
309ebe01-b2ad-4cd7-903b-c759a8696e1f	PX VM PW	Secret text	PX VM PW
8c185007-3c8c-444b-9315-dbf28ff5aa78	Git	Username with password	Git
2088cac1-c042-4946-0000-75e9e35c366d	jenkins/***** (prd gitlab)	Username with password	prd gitlab

아이콘:

S

M

L

REST API Jenkins 2.346.1

관리중인 Credential 목록

ID	대상서버	값	용도
2088cac1-c042-4946-adec-75e9e35c366d	plogitlab.kblife.co.kr	ID: jenkins PW: *****	Gitlab 접근하는 계정
22626871-9a6e-456a-944d-1cd0d2707e0e	dlosnexus.kblife.co.kr plosnexus.kblife.co.kr	*****	넥서스 비밀번호
87204c27-cdeb-48af-a177-e426ebf4812f	dlosnexus.kblife.co.kr plosnexus.kblife.co.kr	cor-admin	넥서스 ID
fe1fc089-b606-498e-a3ee-255aac95fd9b	dlosjenkins.kblife.co.kr plosjenkins.kblife.co.kr	/DATA/unt	VM 에서 관리되는 솔루션 root 경로
0002a3ba-62ef-491e-ab8a-b9923d6b0a20	t2-dv-btc-aps-01 t2-tv-btc-aps-01 pv-btc-aps-01 pv-btc-aps-02	*****	배치서버 devonboot 계정 비밀번호

309ebe01-b2ad-4cd7-903b-c759a8696e1f	dv-cal-aps-01 tv-cal-aps-01 pv-cal-aps-01 pv-cal-aps-02	*****	PX 서버 계정 비밀번호
--------------------------------------	--	-------	---------------

3.3.6.1. Credentials 생성

Jenkins 관리 > Manage Credentials > Jenkins Store 클릭 > Global credentials 클릭 > Add Credentials 클릭

- Kind 를 종류에 맞춰 선택
 - Username with password: 계정 정보 (ID / PW)
 - Secret text: 노출하지 않을 문자열
- ID: 파이프라인에서 사용할 변수명, default: 랜덤 UUID
- Description: credential 을 설명하는 설명란

The screenshot shows the Jenkins 'New credentials' form. The browser address bar indicates the URL: https://pjosjenkins.kblife.co.kr/credentials/store/system/domain/_/newCredentials. The Jenkins logo and navigation menu are visible at the top. The breadcrumb trail is: Dashboard > Credentials > System > Global credentials (unrestricted). The sidebar on the left has a red box around the '+ Add Credentials' button. The main form area is titled 'New credentials'. It contains the following fields:

- Kind:** A dropdown menu with 'Username with password' selected. It is highlighted with a red box.
- Scope:** A dropdown menu with 'Global (Jenkins, nodes, items, all child items, etc)' selected.
- Username:** A text input field.
- Treat username as secret:** An unchecked checkbox.
- Password:** A text input field.
- ID:** A text input field, highlighted with a red box.
- Description:** A text input field, highlighted with a red box.
- Create:** A blue button at the bottom.

3.3.6.2. Credentials 수정

Jenkins 관리 > Manage Credentials > Jenkins Store 클릭 > Global credentials 클릭 > 수정할 credential 의 오른쪽 끝 옵션 클릭

- Change Password 클릭 후, 기존 값 제거 및 변경할 값 입력
- Username with password 종류의 credential 은 Username 필드도 존재하는데 이는 계정의 ID 를 의미한다.

System » Global credentials (unrestricted)

https://pjosjenkins.kblife.co.kr/credentials/store/system/domain/_/

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins 검색 ? 정민성 로그아웃

Dashboard > Credentials > System > Global credentials (unrestricted) >

↑ Back to credential domains

+ Add Credentials

Global credentials (unrestricted)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description	
2088cac1-c042-4946-adeb-75e9e35c366d	jenkins/*****	Username with password		
22626871-9a6e-456a-944d-1cd0d2707e0e	nexus pw	Secret text	nexus pw	
87204c27-cdeb-48af-a177-e426ebf4812f	nexus id	Secret text	nexus id	
fe1fc089-b606-498e-a3ee-255aac95fd9b	solution home	Secret text	solution home	
0002a3ba-62ef-491e-ab8a-b9923d6b0a20	배치 devonboot 계정 password	Secret text	배치 devonboot 계정 password	
309ebe01-b2ad-4cd7-903b-c759a8696e1f	PX VM PW	Secret text	PX VM PW	
8c185007-3c8c-444b-9315-dbf28ff5aa78	Git	Username with password	Git	
GITLAB_INT_DEV	GITLAB_INT_DEV	Username with password	GITLAB_INT_DEV	
GITLAB_INT_UAT	GITLAB_INT_UAT	Username with password	GITLAB_INT_UAT	
GITLAB_INT_PRD	GITLAB_INT_PRD	Username with password	GITLAB_INT_PRD	
TAS_INT_DEV	TAS_INT_DEV	Username with password	TAS_INT_DEV	

Update credentials [Jenkins] x +

https://plosjenkins.kblife.co.kr/credentials/store/system/domain/_/credential/2088cac1-c042-4946-adec-75e9e35c366...

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

Dashboard > Credentials > System > Global credentials (unrestricted) > jenkins/*****

Back to Global credentials (unrestricted)

Update

Delete

Move

Update credentials

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

jenkins

☐ Treat username as secret ?

Password ?

Concealed

Change Password

ID ?

2088cac1-c042-4946-adec-75e9e35c366d

Description ?

Save

REST API Jenkins 2.346.1

3.3.7. API Token 관리

우측 상단 계정명 클릭 > 좌측 메뉴 설정 으로 진입 > Add new Token

- 토큰 이름 기입 후 Generate 클릭
- 생성된 token 번호 복사 (최초 1회만 값 확인 가능하며 이후 확인 불가)
- ITSM 연동을 위한 token으로 삭제 주의

User '정민성' Configuration [Jenl...]

https://pjosjenkins.kblife.co.kr/user/op928/configure

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

검색

정민성

로그아웃

Dashboard > 정민성

↑ 사용자

🔍 상태

📁 빌드

⚙️ 설정

👤 My Views

🔑 Credentials

이름 ?

정민성

소개 ?

API Token

Current token(s) ?

There are no registered tokens for this user.

Add new Token

Credentials

Credentials are only available to the user they belong to

E-mail

Save

Apply

User '정민성' Configuration [Jenl...]

https://pjosjenkins.kblife.co.kr/user/op928/configure

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Dashboard > 정민성

상태

빌드

설정

My Views

Credentials

정민성

소개 ?

API Token

Current token(s) ?

There are no registered tokens for this user.

외부에서 호출용 토큰

Generate

Add new Token

Credentials

Credentials are only available to the user they belong to

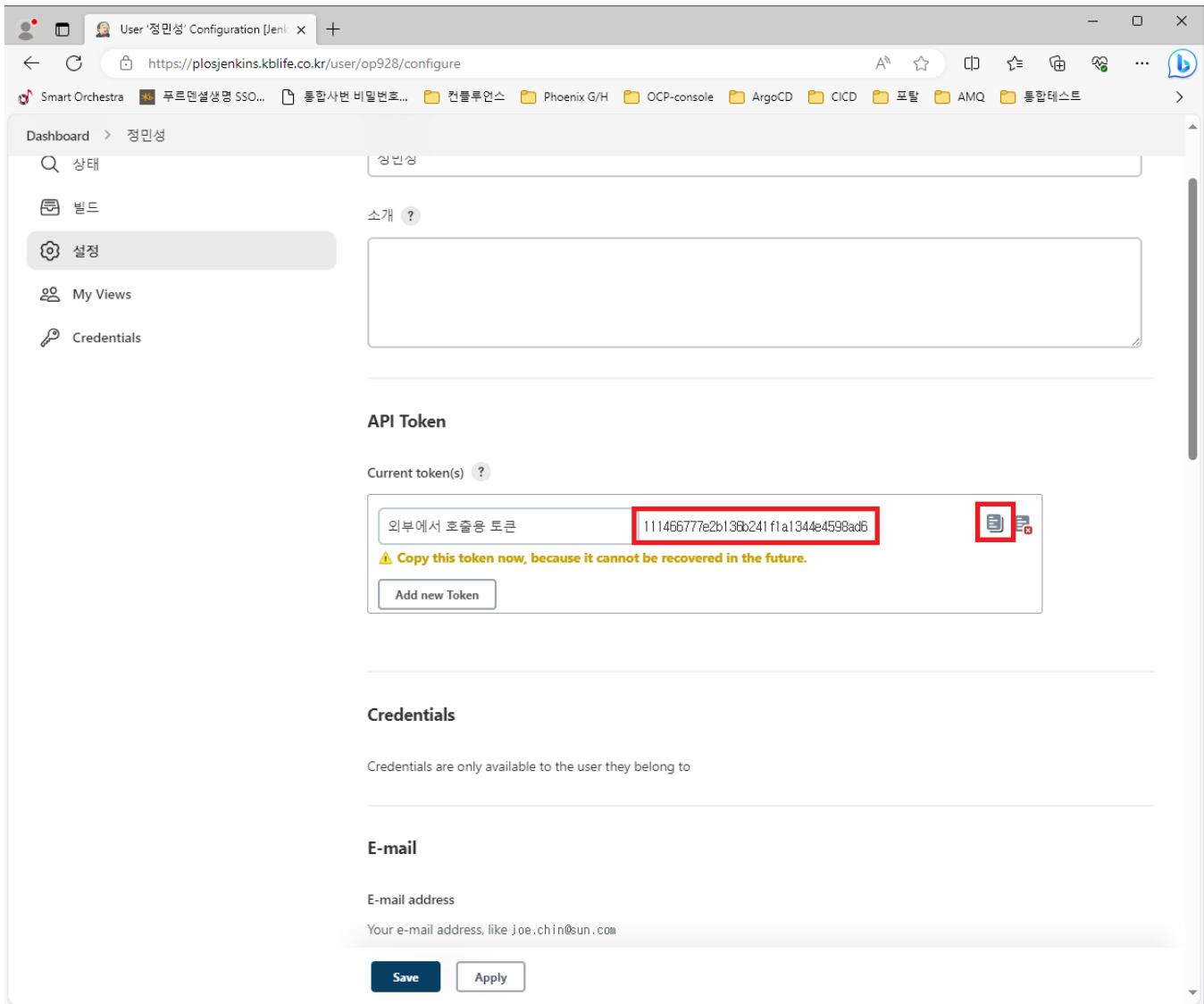
E-mail

E-mail address

Your e-mail address, like joe.chin@sun.com

Save

Apply



3.3.8. 신규 프로젝트 생성

3.3.8.1. 어플리케이션 Gitlab 리파지토리 생성 (Gitlab 관리자 협업)

- <https://plogitlab.kblife.co.kr> 에서 네이밍 규칙 준수하여 프로젝트 생성
- 생성되는 gitlab 리파지토리 주소 : <https://plogitlab.kblife.co.kr/los-> [시스템 코드] / [시스템 코드] _ [업무 코드] _ [어플리케이션 종류]
- LOS_SWA_DE_01 (운영자매뉴얼)_Gitlab_v.1.0.pdf 참고

3.3.8.2. 배포 Gitlab 리파지토리 생성 (OCP 인프라 관리자 협업)

- <https://plogitlab.kblife.co.kr/los-cd> 에서 네이밍 규칙 준수하여 프로젝트 생성
- 생성되는 gitlab 리파지토리 주소 : <https://plogitlab.kblife.co.kr/los-cd/> [시스템 코드] / [시스템 코드] _ [업무 코드] _ [어플리케이션 종류]
- deployment.yaml, service.yaml 등의 리소스 복사 후 신규 생성된 네이밍 적용
- Kubernetes 환경 설정 파일에 대한 지식 및 스킬 필요 (OCP 담당자와 협업)

3.3.8.3. (Optional) 빌드 파이프라인 Jenkinsfile 생성 (Jenkins 빌드 관리자 & OCP 인프라 관리자 협업)

- 신규 파이프라인 구축시 진행
- <https://plogitlab.kblife.co.kr/los-ci> 에서 네이밍 규칙 준수하여 프로젝트 생성
- 생성되는 gitlab 리파지토리 주소 : <https://plogitlab.kblife.co.kr/los-ci/> [프레임워크] / 제품명
- Dockerfile, Jenkinsfile, entrypoint.sh 등의 리소스 복사 후 신규 생성된 네이밍 적용
- Dockerfile, Jenkinsfile, entrypoint.sh 은 컨테이너 환경 구성 관련 지식 및 스킬 필요

- OCP 인프라 관리자 통해 POD 에서 사용할 UID, GID 설정 필요
- apm 및 제품의 빌드 경우 솔루션 업체에서 구성 필요

3.3.8.4. (Optional) Jenkins 환경변수 추가 (Jenkins 관리자 협업)

- 기존에 구성된 업무 외 신규 업무 추가시 진행
- Jenkins 관리 → System > Global properties
- [시스템 코드]_[신규 추가되는 업무 코드] 변수의 값으로 LOS 기입 후 저장

The screenshot displays the Jenkins web interface. At the top, there's a black header with the Jenkins logo. Below it, a breadcrumb trail shows 'Dashboard > Jenkins 관리'. The left sidebar contains several menu items: '+ 새로운 Item', '사람', '빌드 기록', '프로젝트 연관 관계', '파일 핑거프린트 확인', 'Jenkins 관리' (highlighted with a red box), 'My Views', and 'P4 Trigger'. The main content area is titled 'Jenkins 관리' and features a 'System Configuration' section where the 'System' option (with a gear icon and description '환경변수 및 경로 정보등을 설정합니다.') is highlighted with a red box. Other options in this section include 'Tools' (Configure tools, their locations and automatic installers.), 'Configuration as Code' (Reload your configuration or update configuration source), and 'Clouds' (Add, remove, and configure cloud instances to provision agents on-demand.). Below this is a 'Security' section with 'Security' (Secure Jenkins; define who is allowed to access/use the system.) and 'Manage Credentials' (Configure credentials).

Global properties

☐ Disable deferred wipeout on this node ?

☒ Environment variables

키-값 목록 ?

이름

APPLICATION_TYPE_IDX

값

9

이름

CBH_NEW

값

LOS

추가

☐ Prepare jobs environment ?

☐ Tool Locations

Copyartifact: Upstream build that triggered this job

Which for multiple unstream ?

저장

Apply

3.3.8.5. Jenkins View(카테고리) 생성 (Jenkins 관리자 협업)

우측 상단 view 리스트의 + 버튼 클릭 > view 이름 입력 > list view 선택 > create

- View 이름: [프로젝트 코드] - [시스템 코드] 으로 작성

Dashboard [Jenkins]

https://pjosjenkins.kblife.co.kr

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

Q 검색

정민성

로그아웃

Dashboard

+ 새로운 Item

사람

빌드 기록

프로젝트 연관 관계

파일 핑거프린트 확인

Jenkins 관리

My Views

P4 Trigger

새로운 뷰

빌드 대기 목록

빌드 대기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

5 대기 중

6 대기 중

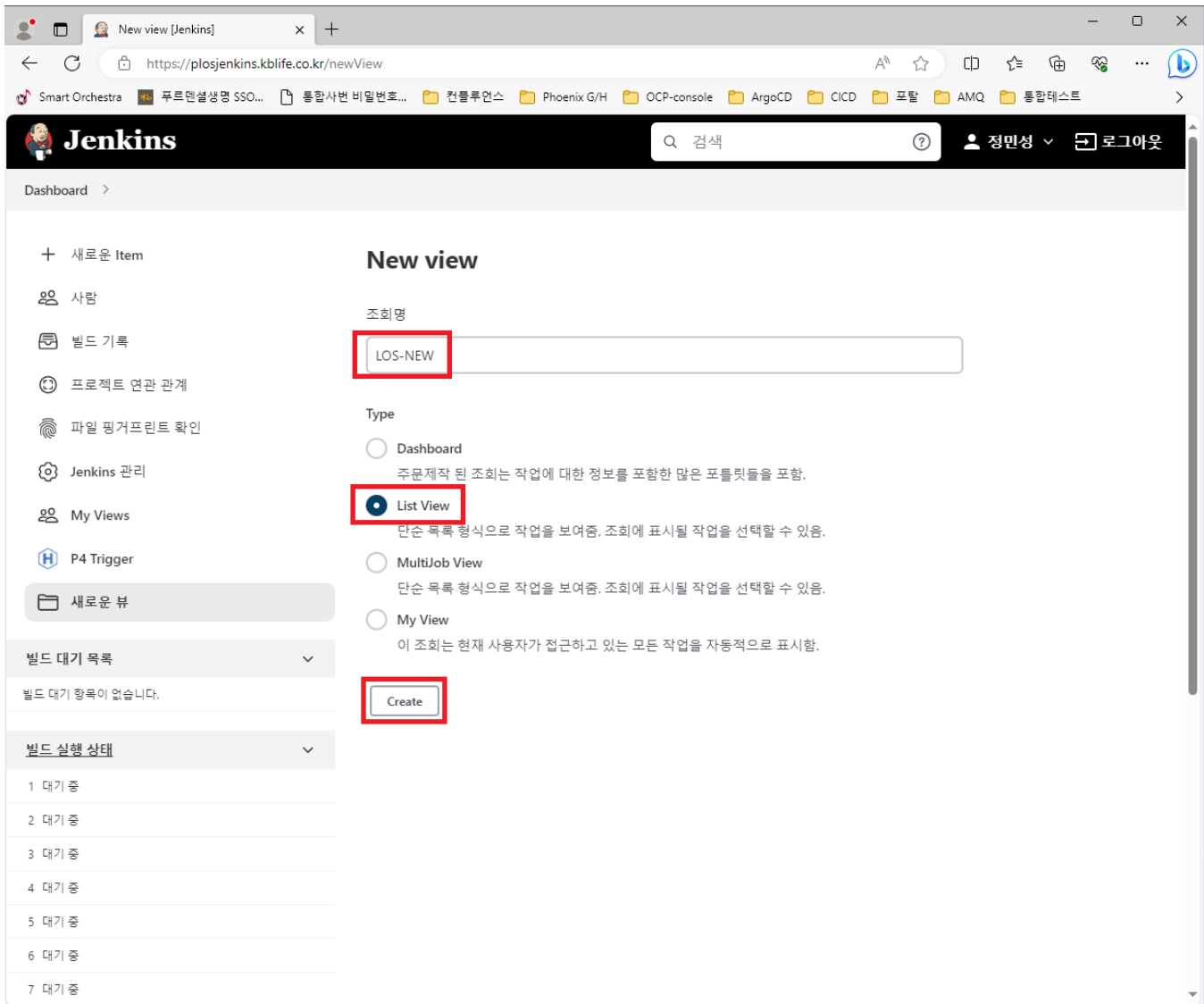
7 대기 중

AllCYBLOS-CBHL0S-CDPL0S-CHNL0S-FEPL0S-GLBL0S-PFGL0S-UNT

TMP+

상세 내용 입력

S	W	Name ↓	최근 성공	최근 실패	최근 소요 시간	Built On	Coverage	# Issues
✖	☁	0. ALL_REGULAR_MERGE_AND_BUILD	1 day 3 hr #570	3 hr 14 min #573	19 min	Jenkins	n/a	-
✔	⚙	1. ALL_GIT_MERGE	38 min #797	—	2 min 18 sec	Jenkins	n/a	-
✖	☁	2. ALL_BUILD_JOB_PRD	1 day 2 hr #586	3 hr 12 min #589	16 min	Jenkins	n/a	-
✔	⚙	CBH_BIP_FRT_PRD	3 hr 12 min #704	1 day 2 hr #699	10 min		n/a	-
✔	⚙	CBH_BKA_BAT_PRD	3 hr 12 min #602	—	1 min 14 sec		n/a	-
✔	⚙	CBH_BKA_BND_PRD	3 hr 12 min #637	—	14 min		n/a	-
✔	⚙	CBH_BPS_BND_PRD	3 hr 12 min #613	—	9 min 44 sec		n/a	-



3.3.8.6. CI/CD Jenkins Job 생성 (Jenkins 관리자 협업)

생성할 Job 이 소속된 시스템 view 클릭 > 좌측 상단 새로운 Item 클릭 > Item 이름 입력 > 하단 Copy From에 유사한 Job 이름 입력 > OK 클릭 > 설명 수정 > 저장 클릭

- Item(Job) 이름은 [시스템 코드]_[업무 코드]_[어플리케이션 종류]_[배포 환경] 유형으로 작성(ITSM 명명 규칙 준용)
- 마지막 코드인 [배포 환경]은 실제 배포되는 배포 환경 및 Branch 이름과 연관되어 동작하므로 준용
- 파이프라인 재활용을 위해 동일한 종류의 CI를 사용하는 Job 이름을 입력하여 복사

LOS-CBH [Jenkins]

https://pjosjenkins.kblife.co.kr/view/LOS-CBH/

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

검색

정민성

로그아웃

Dashboard > LOS-CBH >

+ 새로운 Item

사람

빌드 기록

보기 수정

뷰 삭제

프로젝트 연관 관계

파일 핑거프린트 확인

Jenkins 관리

My Views

P4 Trigger

새로운 뷰

빌드 대기 목록

빌드 대기 항목이 없습니다.

빌드 실행 상태

1 대기 중

2 대기 중

3 대기 중

4 대기 중

AllCYBLOS-CBHLOS-CDPLOS-CHNLOS-FEPLOS-GLBLOS-PFGLOS-UNT

TMP+

S

W

Name ↓

최근 성공

최근 실패

최근 소요 시간

Built On

Coverage

Issues

✓

⚙

CBH_BIP_FRT_PRD

3 hr 32 min
#704

1 day 2 hr
#699

10 min

▶

n/a -

✓

⚙

CBH_BKA_BAT_PRD

3 hr 32 min
#602

—

1 min 14 sec

▶

n/a -

✓

⚙

CBH_BKA_BND_PRD

20 min
#638

—

1 min 15 sec

▶

n/a -

✓

⚙

CBH_BPS_BND_PRD

3 hr 32 min
#613

—

9 min 44 sec

▶

n/a -

✓

⚙

CBH_CBH_BAT_PRD

3 hr 32 min
#615

—

2 min 13 sec

▶

n/a -

✓

⚙

CBH_CCT_BND_PRD

1 hr 41 min
#433

1 day 21 hr
#423

2 min 21 sec

▶

n/a -

✓

☁

CBH_CCT_FRT_PRD

3 hr 32 min
#429

1 day 2 hr
#425

10 min

▶

n/a -

✓

⚙

CBH_CLM_BND_PRD

3 hr 32 min
#611

10 days
#580

10 min

▶

n/a -

✓

⚙

CBH_CME_BND_PRD

3 hr 32 min
#610

—

10 min

▶

n/a -

✓

⚙

CBH_CMN

3 hr 32 min

7 hr 3 min

14 min

▶

n/a -

New Item [Jenkins]

https://pjosjenkins.kblife.co.kr/view/LOS-CBH/new/job

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨틀루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Jenkins

검색

정민성

로그아웃

Dashboard > LOS-CBH >

Enter an item name

CBH_NEW_BND_PRD

Required field

Freestyle project

이것은 Jenkins의 주요 기능입니다. Jenkins은 어느 빌드 시스템과 어떤 SCM(형상관리)으로 묶인 당신의 프로젝트를 빌드할 것이고, 소프트웨어 빌드보다 다른 어떤 것에 자주 사용될 수 있습니다.

Maven project

Maven 프로젝트를 빌드합니다. Jenkins은 POM 파일의 이점을 가지고 있고 급격히 설정을 줄입니다.

Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

If you want to create a new item from other existing, you can use this option:

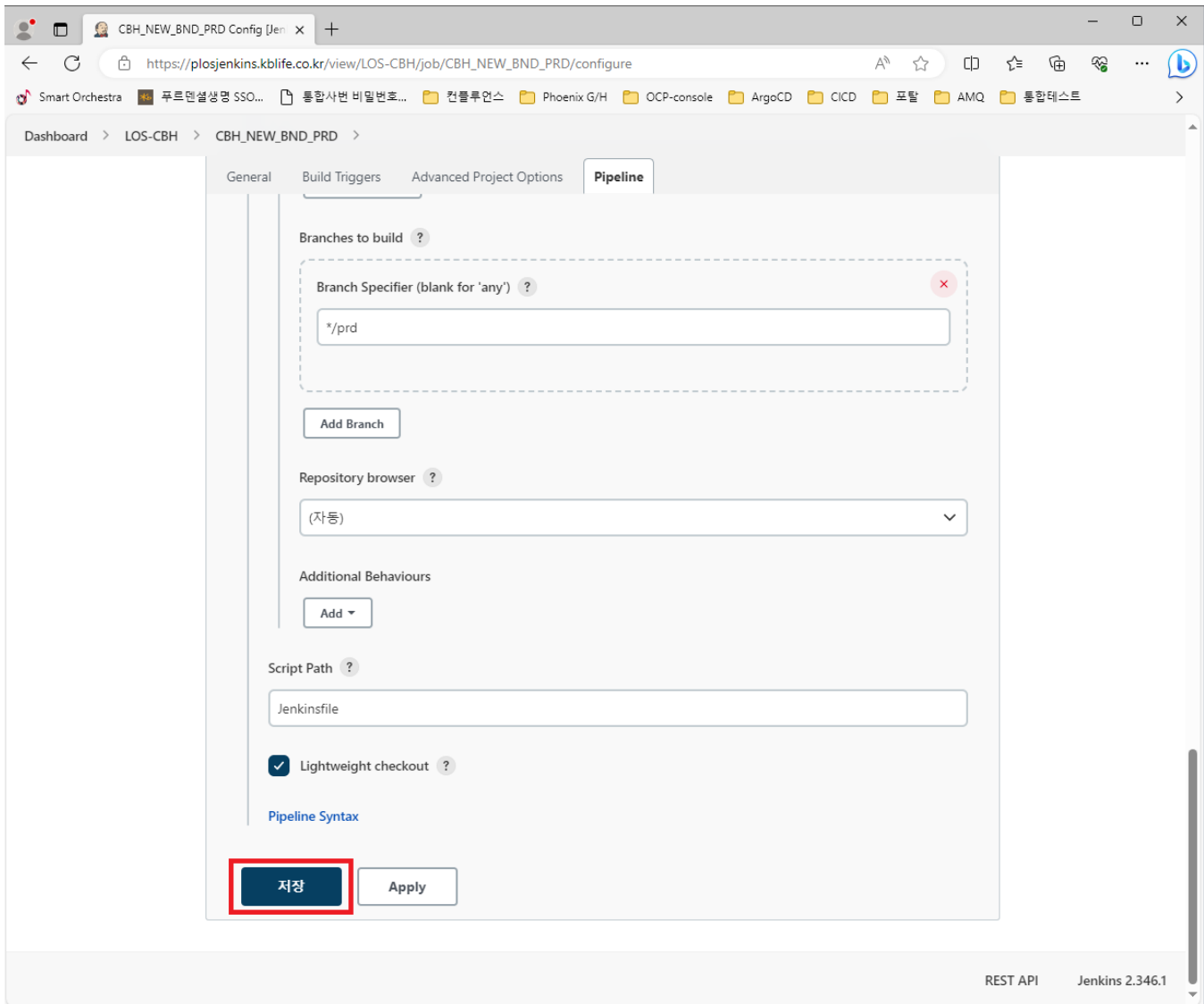
Copy from

CBH_BKA_BND_PRD

OK

☒ Add to current view

REST API Jenkins 2.346.1



3.3.8.7. ArgoCD application 생성 (OCP 인프라 관리자 협업)

본 가이드의 "3.4.3. Application 생성" 절을 참고하여 신규 추가되는 서비스의 application 생성

- Application명은 모두 소문자로 기입하며, [시스템 코드] - [업무 코드] - [어플리케이션 타입] - [배포 환경] 으로 작성한다.
- ArgoCD는 네이밍에 '-'를 허용하지 않기 때문에 구분자로 '-'를 사용한다.

3.3.9. PRD Job 긴급 수동 실행

PRD Job 실행 권한이 있는 계정으로 로그인 > 배포할 업무의 실행 화살표 클릭 > psrNo 입력 > 빌드하기 버튼 클릭

- 실행 이전에 소스코드를 PRD branch에 merge 하고 psr 번호로 Tag 생성
- psrNo 파라미터에 psr 번호 기입

Dashboard [Jenkins]									
https://pjosjenkins.kblife.co.kr									
Smart Orchestra 푸트앤셀생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트									
Dashboard >									
14 대기 중	✓	☁	CBH_CCT_FRT_PRD	3 hr 38 min #429	1 day 3 hr #425	10 min	▶	n/a	-
15 대기 중	✓	⚙	CBH_CLM_BND_PRD	3 hr 38 min #611	10 days #580	10 min	▶	n/a	-
16 대기 중	✓	⚙	CBH_CME_BND_PRD	3 hr 38 min #610	—	10 min	▶	n/a	-
17 대기 중	✓	☁	CBH_CMN_BND_PRD	3 hr 38 min #634	7 hr 9 min #632	14 min	▶	n/a	-
18 대기 중	✓	⚙	CBH_DEP_BND_PRD	3 hr 38 min #608	10 days #576	14 min	▶	n/a	-
	✓	⚙	CBH_DEP_BND_PRD_ITSM	1 mo 24 days 240086-1-12-15	—	9.3 sec	▶	n/a	-
	✓	⚙	CBH_IAP_BND_PRD	3 hr 38 min #379	—	10 min	▶	n/a	-
	✓	⚙	CBH_JPM_BND_PRD	3 hr 38 min #631	10 days #603	3 min 15 sec	▶	n/a	-
	✓	⚙	CBH_JPS_BND_PRD	3 hr 38 min #630	—	15 min	▶	n/a	-
	✓	⚙	CBH_MHC_BND_PRD	3 hr 38 min #429	1 day 21 hr #420	14 min	▶	n/a	-
	✓	☁	CBH_MHC_FRT_PRD	3 hr 38 min #421	1 day 3 hr #417	1 min 28 sec	▶	n/a	-
	✓	⚙	CBH_NBS_BND_PRD	3 hr 38 min #640	7 days 2 hr #619	10 min	▶	n/a	-
	⋯	⚙	CBH_NEW_BND_PRD	—	—	—		n/a	-

Jenkins

Dashboard > CBH_PAY_BND_PRD >

Pipeline CBH_PAY_BND_PRD

매개변수가 필요한 빌드입니다.

psrNo

240087-1

☐ isRollback

롤백시 선택하고 psrNo 에 값에는 롤백할 psrNo 값을 job build number 와 함께 명확히 기입한다.
psrNo 값 예시: 238872-1-32

빌드하기

Build History

Filter builds...

- 240087-1-5-7 | 2023. 10. 6 오후 2:58
- 240089-1-6 | 2023. 10. 6 오후 2:56
- 240087-1-5 | 2023. 10. 6 오후 2:53
- 240087-1-4 | 2023. 10. 6 오후 2:52

3.3.10. PRD Job 롤백

PRD Job 실행 권한이 있는 계정으로 로그인 > 배포할 업무의 실행 화살표 클릭 > 롤백할 psrNo 입력 > isRollback 체크박스 클릭 > 빌드하기 버튼 클릭

- psrNo 파라미터에 이전 실행한 psrNo 와 build number 가 포함된 버전 기입
- isRollback 체크박스 체크 상태 확인

Jenkins

Dashboard > CBH_PAY_BND_PRD >

Pipeline CBH_PAY_BND_PRD

매개변수가 필요한 빌드입니다.

psrNo

240087-1-5

☒ isRollback

롤백시 선택하고 psrNo 에 값에는 롤백할 psrNo 값을 job build number 와 함께 명확히 기입한다.
psrNo 값 예시: 238872-1-32

빌드하기

Build History

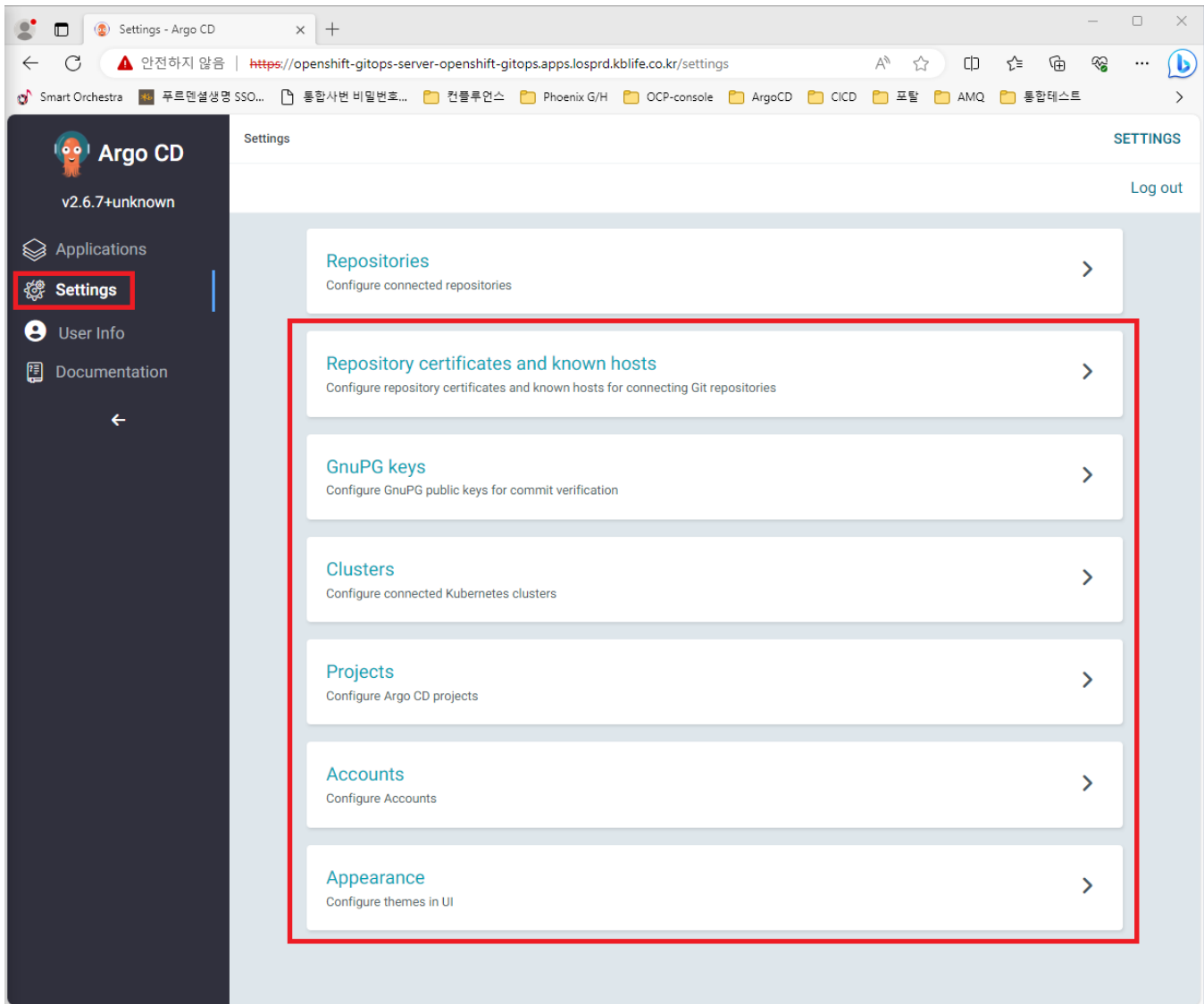
Filter builds...

- 240087-1-5-7
2023. 10. 6 오후 2:58
- 240089-1-6
2023. 10. 6 오후 2:56
- 240087-1-5 롤백할 psrNo 버전
2023. 10. 6 오후 2:53
- 240087-1-4
2023. 10. 6 오후 2:53

3.4. ArgoCD 주요 관리

3.4.1. 세팅 관리

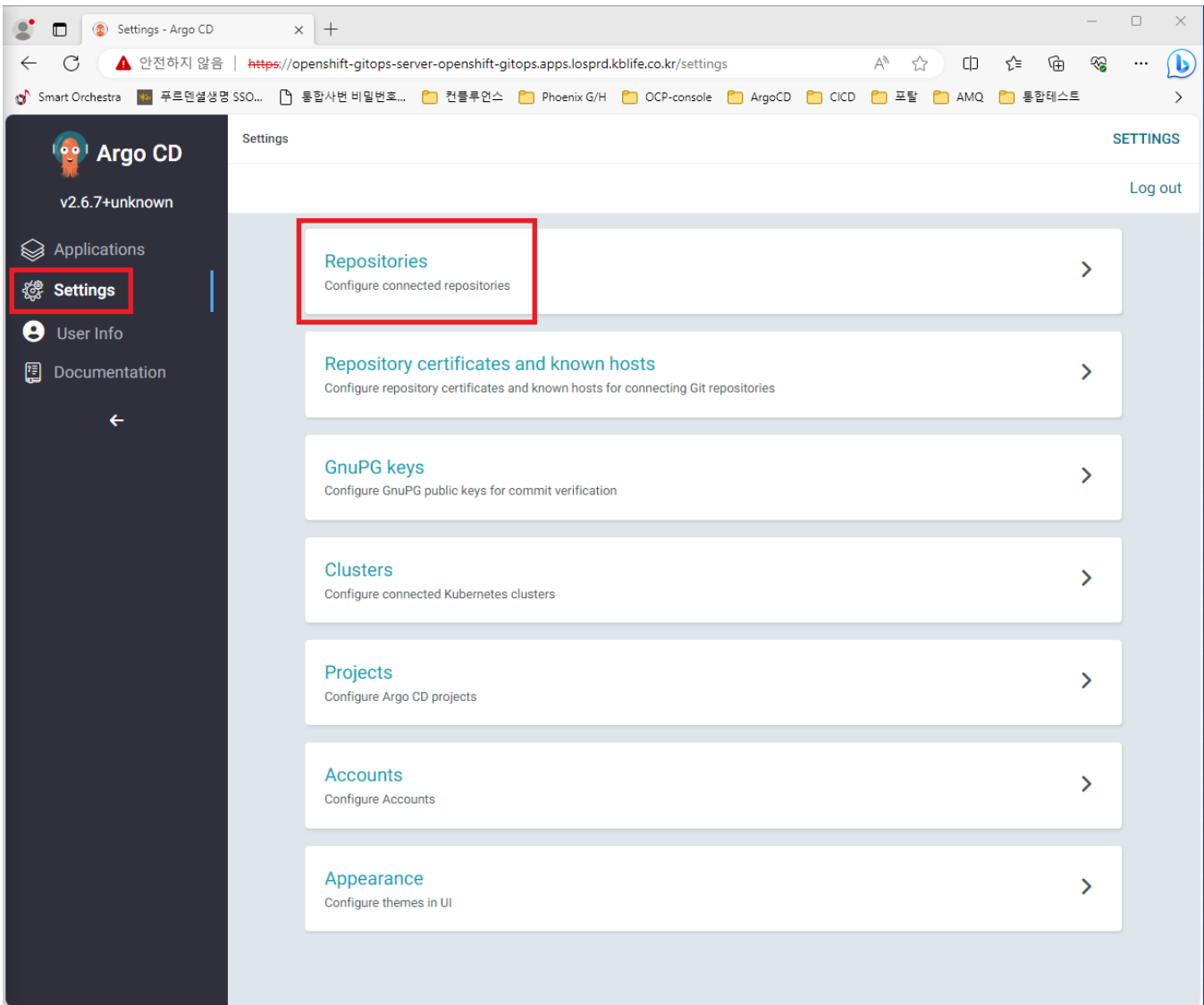
ArgoCD 는 Red Hat OpenShift 의 Operator 로 구성되어 Repositories 를 제외한 모든 설정은 Operator 로 관리되기에 OCP 관리자의 협업을 통해 세팅 할 수 있다.



3.4.2. Repository 추가

Settings > Repositories > + CONNECT REPO > VIA HTTPS 선택 > 배포할 gitlab LOS CD 그룹의 repository 정보 입력 > CONNECT 클릭

- Choose your connection method: "VIA HTTPS" 선택
- Type: "git" 선택
- Project: "default" 선택
- Repository URL: 배포할 서비스의 LOS CD repository 주소 기입 (반드시 주소 끝에 .git 포함)
- Username: argocd 가 사용할 gitlab 계정 기입
- Password: argocd 가 사용할 gitlab 비밀번호 기입



Argo CD

v2.6.7+unknown

Applications

Settings

User Info

Documentation

Settings / Repositories

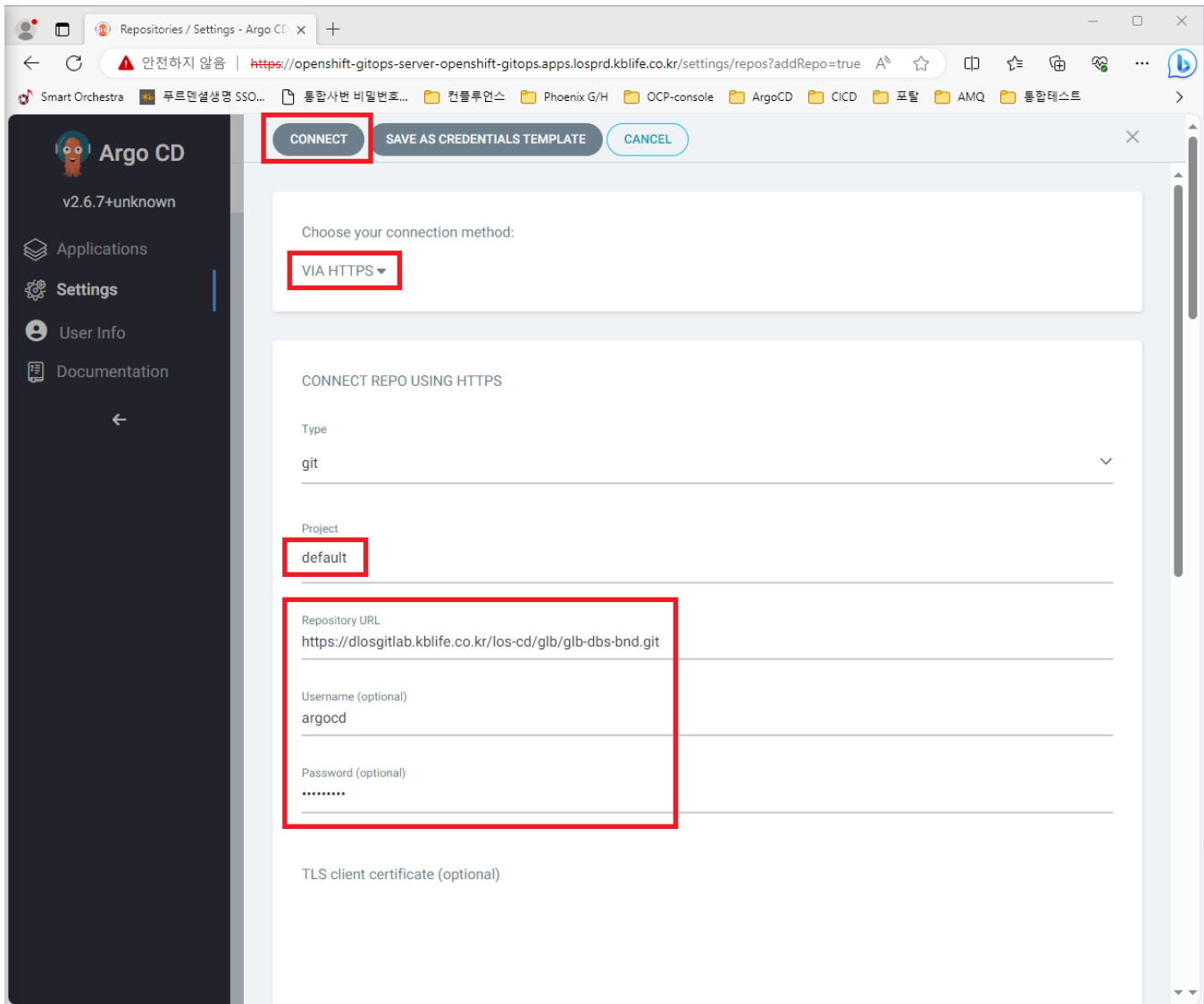
REPOSITORIES

Log out

+ CONNECT REPO

REFRESH LIST

TYPE	NAME	REPOSITORY	CONNECTION STATUS
git		https://dlogitlab.kblife.co.kr/los-cd/pfg/pfg-plm-bnd...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/unt/unt-amp-bat...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/unt/unt-amq-dm...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/chn/chn-pmd-b...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/unt/unt-cvt-frt.git	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/unt/unt-efm-frt...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/config-map.git	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/pfg/pfg-pim-bnd...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-pgh-bn...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-clm-bn...	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/virtual-service.git	Successful
git		https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh_pay_bn...	Successful



3.4.3. Application 생성

Settings > Repositories > ctrl + f 눌러 생성할 업무명 검색 > 오른쪽 ... 클릭 > Create application 클릭 > 포맷에 맞추어 작성 > 상단에 위치한 CREATE 클릭

- Application Name: [시스템 코드] - [업무 코드] - [어플리케이션 종류] - [배포 환경] 으로 작성 (예: glb-dbs-bnd-prd)
- Project Name: "default" 선택
- SYNC POLICY: "Automatic" 선택, SELF HEAL 체크박스 체크
- Repository URL: 생성할 서비스의 LOS CD gitlab repository 주소인지 확인
- Revision: 배포 환경 입력 (예: 개발=dev, 테스트=uat, 운영=prd)
- Path: "." 입력
- Cluster URL: dev, prd - <https://kubernetes.default.svc> 선택, uat - <https://api.losuat.kblife.co.kr:6443> 선택
- Namespace: los- [시스템 코드] - [배포 환경] 으로 작성 (예: los-glb-prd)

Argo CD

v2.6.7+unknown

Applications

Settings

User Info

Documentation

Repositories / Settings - Argo CD

Repositories

+ CONNECT REPO

REFRESH LIST

Log out

git	https://dlogitlab.kblife.co.kr/los-cd/chn/chn-sfa-frt...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-cct-bnd...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-iap-bnd...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/chn/chn-sfa-fmr...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/unt/unt-img-frt.git	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh_dep_b...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-dep-bn...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/chn/chn-sfa-bnd...	Successful	
git	cbh-bip-frt-repo https://plogitlab.kblife.co.kr/los-cd/cbh/cbh-bip-frt...	Failed	
git	https://dlogitlab.kblife.co.kr/los-cd/unt/unt-ass-sck...	Successful	
git	https://dlogitlab.kblife.co.kr/los-cd/cbh/cbh-cmn-bn...	St...	Create application Disconnect
git	https://dlogitlab.kblife.co.kr/los-cd/glb/glb-dbs-bnd...	Successful	

Applications Tiles - Argo CD

안전하지 않음 | https://openshift-gitops-server-openshift-gitops.apps.losprd.kblife.co.kr/applications?showFavorites=...

Smart Orchestra 푸르덴셜생명 SSO... 통합사번 비밀번호... 컨플루언스 Phoenix G/H OCP-console ArgoCD CICD 포탈 AMQ 통합테스트

Argo CD

v2.6.7+unknown

Applications

Settings

User Info

Documentation

FILTERS

★ Favorites Only

SYNC STATUS

Unknown 1

Synced 56

OutOfSync 0

HEALTH STATUS

Unknown 0

Progressing 0

Suspended 0

Healthy 57

Degraded 0

Missing 0

LABELS

LABELS

CREATE CANCEL

EDIT AS YAML

GENERAL

Application Name

glb-dbs-bnd-prd

Project Name

default

SYNC POLICY

Automatic

PRUNE RESOURCES

☒ SELF HEAL

☐ SET DELETION FINALIZER

SYNC OPTIONS

☐ SKIP SCHEMA VALIDATION

☐ PRUNE LAST

☐ RESPECT IGNORE DIFFERENCES

☐ AUTO-CREATE NAMESPACE

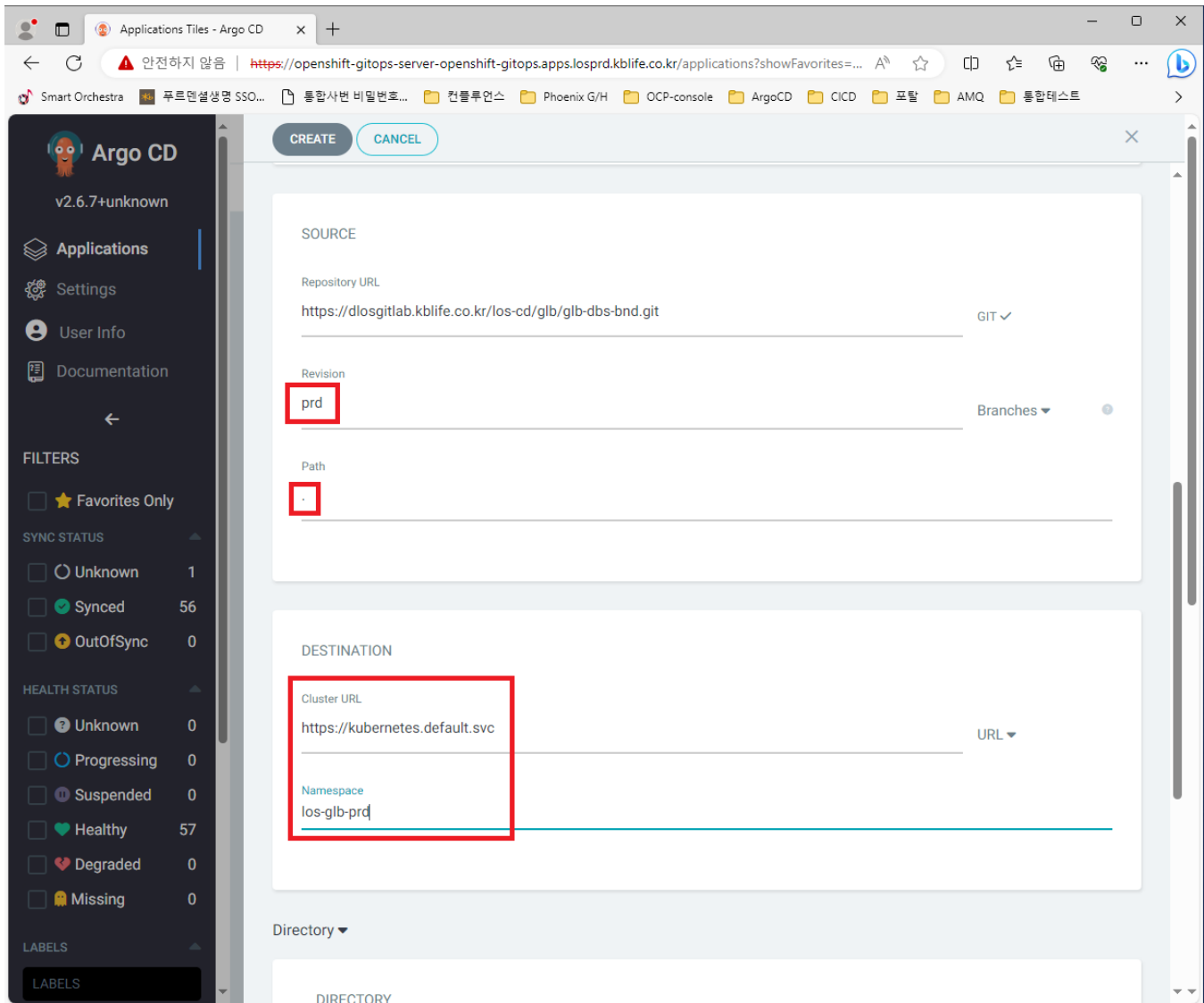
☐ APPLY OUT OF SYNC ONLY

☐ SERVER-SIDE APPLY

PRUNE PROPAGATION POLICY: foreground

☐ REPLACE

☐ RETRY



3.4.4. Application 수동 배포

Applications > 수동 배포할 서비스 검색 / 클릭 > SYNC 클릭 > REFRESH 클릭

- 5분(300초) 마다 argocd 가 자동 배포를 하지만, 빠른 수동 배포가 필요한 경우 사용

The screenshot shows the Argo CD web interface. On the left, a dark sidebar contains navigation links: 'Applications' (highlighted with a red box), 'Settings', 'User Info', and 'Documentation'. Below these are sections for 'FILTERS' (including 'Favorites Only'), 'SYNC STATUS' (Unknown, Synced, OutOfSync), 'HEALTH STATUS' (Unknown, Progressing, Suspended, Healthy, Degraded, Missing), and 'LABELS'. The main content area is titled 'Applications' and features a search bar at the top right containing the text 'dbs' (highlighted with a red box). Below the search bar, a list of applications is displayed. One application, 'glb-dbs-bnd-prd', is highlighted with a red box. This application is in a 'Synced' state, indicated by a green heart icon and the text 'Healthy Synced'. Other details for this application include its project name 'default', repository URL 'https://dlogitlab.kblife.co.kr/l...', target 'prd', path '.', destination 'in-cluster', name 'los-glb-prd', and creation time '05/19/2023 09:50:10 (6 mont...)'. At the bottom of the application card, there are buttons for 'SYNC', 'Refresh', and 'Delete'.

glb-dbs-bnd-prd - Application D

+

안전하지 않음 | https://openshift-gitops-server-openshift-gitops.apps.losprd.kblife.co.kr/applications/openshift-gitop...

Smart Orchestra | 푸르덴셜생명 SSO... | 통합사번 비밀번호... | 컨플루언스 | Phoenix G/H | OCP-console | ArgoCD | CICD | 포탈 | AMQ | 통합테스트

Argo CD

v2.6.7+unknown

Applications

Settings

User Info

Documentation

←

FILTERS

NAME

NAME

KINDS

KINDS

SYNC STATUS

☐ Synced 1

☐ OutOfSync 0

HEALTH STATUS

☐ Healthy 2

☐ Progressing 0

☐ Degraded 0

☐ Suspended 0

☐ Missing 0

☐ Unknown 0

Applications / glb-dbs-bnd-prd

APPLICATION DETAILS NETWORK

APP DETAILS

APP DIFF

SYNC

SYNC STATUS

HISTORY AND ROLLBACK

DELETE

REFRESH

Log out

Author: Jenkins System <jenkins@kblife.co.kr> -
Comment: deploy glb-dbs-bnd-prd.0.0.40
Auto sync is enabled.

Succeeded a day ago (Wed Nov 29 2023 14:45:28 GMT+0900)
Author: Jenkins System <jenkins@kblife.co.kr> -
Comment: deploy glb-dbs-bnd-prd.0.0.40

glb-dbs-bnd

svc

6 months

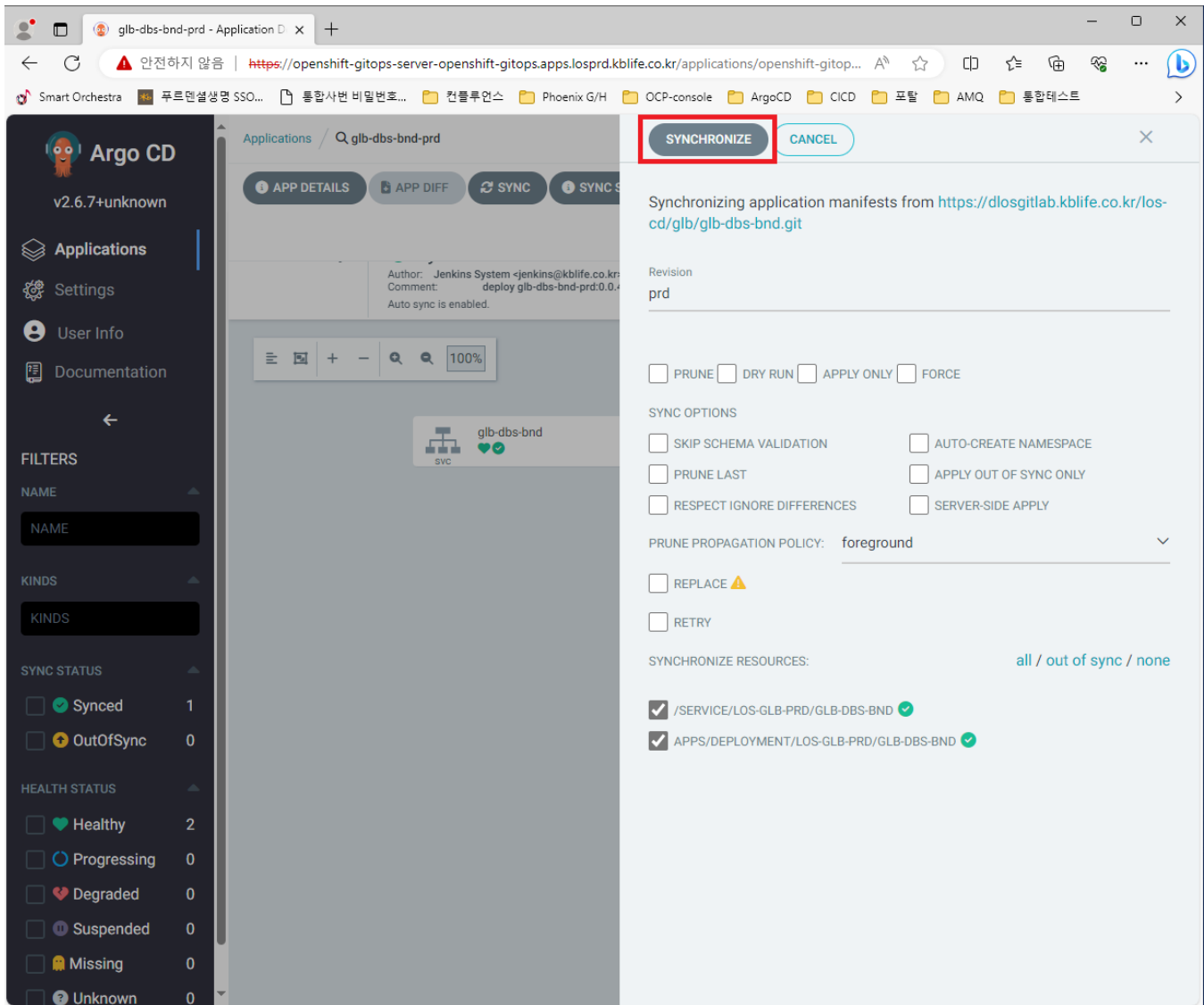
glb-dbs-bnd-66dcf695b8-vbx...

pod

a day

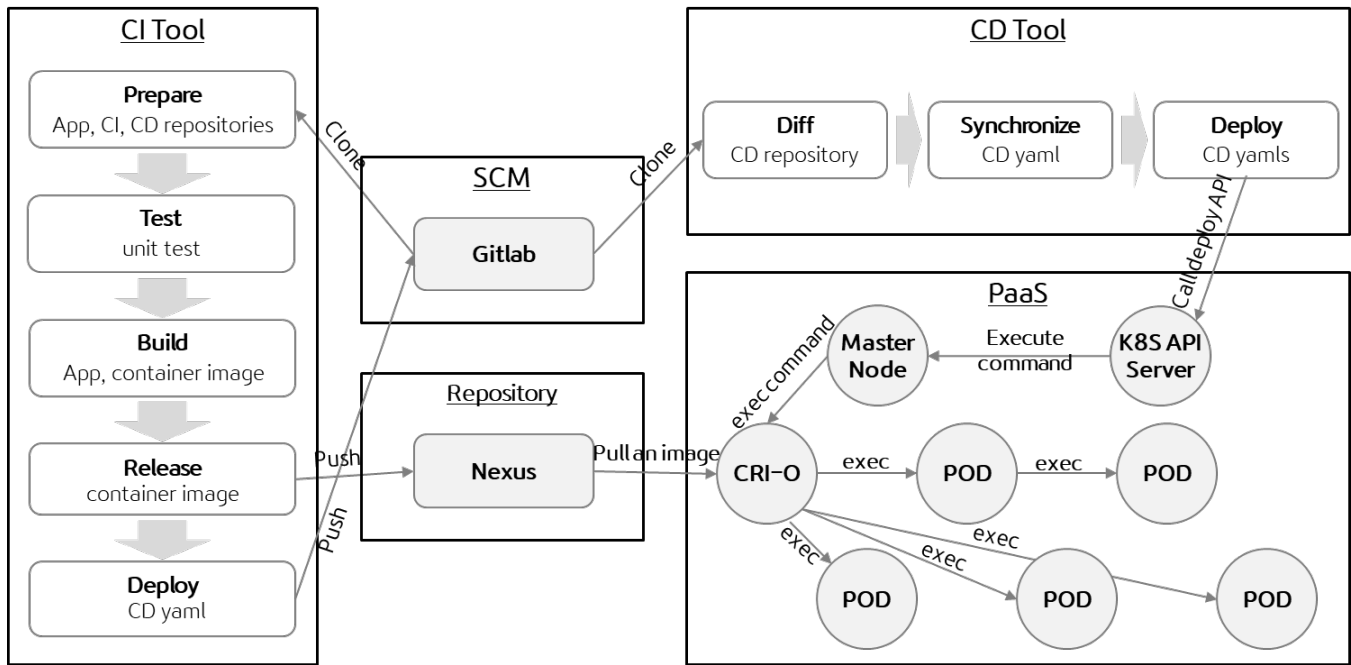
running

2/2



3.5. CICD 이관 절차 및 방법

CICD 이관 절차는 SFA 프로젝트(Track#1) 관리 방법을 준용하여 다음과 같이 어플리케이션과 컨테이너 이미지를 빌드하는 CI 단계과 PaaS 에 실제 배포하는 CD 단계로 분리하여 구성한다.



3.5.1. 환경별 소스 배포

기능 개발을 위해 PSR 을 작성하고, PSR 단위로 기능 개발을 진행한다.

생성된 PSR number 를 기반으로 subpsr 브랜치를 생성하고, 개발자들은 PSR 과 1:1 로 매핑되는 subpsr 브랜치에서 실제 기능 개발을 진행한다.

각 환경별 서버에 빌드/배포하는 방식은 동일하나 서버 브랜치(dev, uat, prd)별 소스를 merge 하는 방식은 다음과 같다.

1. DEV 환경 배포
 - a. DEV 환경 배포는 별도의 승인 없이 subpsr 브랜치를 gitlab 에 push 하고 subpsr 브랜치를 dev 브랜치에 merge 하여, jenkins 빌드/배포 job 을 직접 실행하여 DEV 환경에 배포한다.
2. UAT 환경 배포
 - a. UAT 환경 배포는 PL 이 PSR 에 해당하는 subpsr 브랜치를 UAT 브랜치에 merge 하고, ITSM 에서 Jenkins 빌드/배포 job 을 원격 실행하여 UAT 환경에 배포한다.
3. PRD 환경 배포
 - a. PRD 환경 배포는 PL 이 UAT merge 된 것을 확인하고 ITSM 에서 Jenkins 빌드/배포 job 을 원격 실행하여 PRD 환경에 배포한다.

배포 가이드는 QA 파트에서 관리중인 [LOS_배포프로세스_GIT_개발자_v1.0.pptx](#) 을 준수한다.

3.5.2. CICD 파이프라인 스크립트

3.5.2.1. DEVON-BOOT

단 계	스크립트
--------	------

1	Initi alize	<pre>def accountMap = ["chn-voc": 21001 , "chn-sfa": 21002 , "chn-stc": 21003 , "chn-pmd": 21004 , "cbh-nbs": 22001 , "cbh-pos": 22002 , "cbh-var": 22003 , "cbh-dep": 22004 , "cbh-pay": 22005 , "cbh-clm": 22006 , "cbh-cmn": 22007 , "cbh-bka": 22008 , "cbh-pgh": 22009 , "cbh-jpm": 22011 , "cbh-bps": 22012 , "cbh-cme": 22013 , "cbh-cct": 22014 , "cbh-mhc": 22015 , "cbh-iap": 22016 , "pfg-pis": 23001 , "pfg-pim": 23002 , "pfg-plm": 23003 , "pfg-pmm": 23004 , "pfg-sft": 23005 , "cdp-cim": 24001 , "cdp-civ": 24002 , "cdp-clc": 24003 , "unt-amp": 25001 , "unt-rbs": 25002 , "unt-efs": 25003 , "unt-rpt": 25004 , "unt-img": 25005 , "unt-cvt": 25006 , "unt-ass": 25007 , "unt-tpb": 25008 , "unt-pms": 25009 , "unt-ucg": 25010 , "unt-pgw": 25011 , "unt-amq": 25012 , "glb-asc": 25012 , "glb-asp": 25012 , "glb-dbs": 29001]</pre>
		라인별 설명 1. OCP 인프라 관리자에게 협업을 통해 공유받은 POD 의 username 과 UID 값을 설정한다. (문자열 key = username, 숫자 value = UID 로 map 구조로 구성한다.)
2	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}* git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done sed -i 's/servicename=.*servicename=\${GITLAB_REPO_NAME}/g' copy/APP/ucoverage/jacocoConfig.conf</pre>
		라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다. 4. 동적 검사를 위해 U-Coverage 의 jacocoConfig 설정을 현재 서비스에 맞게 변경한다. (개발 환경 배포시에만 수행한다.)
3	Test	<pre>pushd \${GITLAB_REPO_NAME} sh gradlew test --no-build-cache --fail-fast -Pprofile=\${BRANCH_NAME} rm -rf /LOG/\${SYSTEM_CODE}-\${BIZ_CODE} popd</pre>
		라인별 설명 1. 단위 테스트를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. 단위 테스트를 수행한다. 3. 단위 테스트를 수행하며 쌓인 로그를 제거한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다.

4	Build	<pre> pushd \${GITLAB_REPO_NAME} sh gradlew build -Pprofile=\${BRANCH_NAME} --no-build-cache --exclude-task test mv build/libs/\${GITLAB_REPO_NAME}-0*-SNAPSHOT.jar ../copy/APP/devon/app.jar popd rm -rf /DATA/unt/ucg/\${GITLAB_REPO_NAME} mkdir -p /DATA/unt/ucg/\${GITLAB_REPO_NAME}/source && mkdir -p /DATA/unt/ucg/\${GITLAB_REPO_NAME} /class && cp copy/APP/devon/app.jar /DATA/unt/ucg/\${GITLAB_REPO_NAME}/class/app.jar cp -R \${GITLAB_REPO_NAME}/* /DATA/unt/ucg/\${GITLAB_REPO_NAME}/source/ pushd "/DATA/unt/ucg/\${GITLAB_REPO_NAME}/class" jar xf app.jar popd podman build --no-cache --build-arg JOB_NAME=\${JOB_NAME} --build-arg SYSTEM_CODE=\${SYSTEM_CODE} -- build-arg BIZ_CODE=\${BIZ_CODE} --build-arg BRANCH_NAME=\${BRANCH_NAME} --build-arg ACCOUNT_NAME=\${SYSTEM_CODE}-\${BIZ_CODE} --build-arg ACCOUNT_ID='' + accountMap["\${SYSTEM_CODE} -\${BIZ_CODE}"] + ' ' -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre>
		라인별 설명 1. application 소스 빌드를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. application 소스를 빌드한다. 3. 빌드되어 생성된 실행가능한 파일(jar) 을 컨테이너 복사 대상인 파일 경로로 이동한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다. 5. 과거에 U-Coverage 에 배포된 파일을 모두 제거한다. 6. U-Coverage 에 배포할 수 있는 디렉토리 생성과 생성된 실행가능한 파일(jar) 을 복사하여 넣어준다. 7. 현재 버전의 application 소스를 U-Coverage 에 복사하여 넣어준다. 8. 빌드된 application 클래스를 U-Coverage 에 반영하기 위해 U-Coverage 클래스 관리 디렉토리로 이동한다. 9. 빌드된 실행가능한 파일을 압축 풀어 class 를 U-Coverage 에 압축 해제한다. 10. 8번에서 이동하기 전 디렉토리로 빠져나온다. 11. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
5	Release	<pre> podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} </pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

6	Deploy	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.* /image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니페스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.2. EX-BUILDER

	단계	스크립트
1	Prepare	<pre> rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE} </pre>
라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.		
2	Build	<pre> java -jar e6-compiler.jar -s ./\${GITLAB_REPO_NAME} -o ./copy/APP/ui/src --relativize-less-url true --language true podman build --no-cache --build-arg JOB_NAME=\${JOB_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre>
라인별 설명 1. application 소스를 빌드한다. 2. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.		

3	Release	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_ \${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_ \${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Deploy	<pre>pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\ /\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.*/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\ /\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git</pre> 라인별 설명 1. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 2. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 3. git commit 에 사용할 사용자 email 정보를 저장한다. 4. git commit 에 사용할 사용자 name 정보를 저장한다. 5. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 6. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 7. gitlab 에 저장정보를 업로드한다. 8. 1번에서 이동하기 전 디렉토리로 빠져나온다. 9. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.3. SPRING-BATCH

	단계	스크립트
1	Pre prepare	<pre>rm -rf \${GITLAB_REPO_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다.

2	Test	<pre>pushd \${GITLAB_REPO_NAME} sh gradlew test --no-build-cache --fail-fast -Pprofile=\${BRANCH_NAME} rm -rf /LOG/\${SYSTEM_CODE}-\${BIZ_CODE} popd</pre>
		라인별 설명 1. 단위 테스트를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. 단위 테스트를 수행한다. 3. 단위 테스트를 수행하며 쌓인 로그를 제거한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다.
3	Build	<pre>pushd \${GITLAB_REPO_NAME} sh gradlew build -Pprofile=\${BRANCH_NAME} --no-build-cache --exclude-task test mv build/libs/*-SNAPSHOT.jar ../app.jar popd</pre>
		라인별 설명 1. application 소스 빌드를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. application 소스를 빌드한다. 3. 빌드되어 생성된 실행가능한 파일(jar) 을 Job workspace root 로 이동한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다.
4	Release	<pre>sshpass -p "\${BATCH_DEVONBOOT_PW}" scp -r app.jar devonboot@\${BATCH_SERVER_URL}:/APP /batch\${BIZ_CODE}/src</pre>
		라인별 설명 1. 생성된 실행가능한 파일(jar) 을 배치 서버에 원격으로 복사하여 업로드한다.
5	Deploy	<pre>sshpass -p "\${BATCH_DEVONBOOT_PW}" ssh devonboot@\${BATCH_SERVER_URL} "ls -al /APP/batch\${BIZ_CODE} /src && echo Build id of Jenkins : \${BUILD_ID} > /APP/batch\${BIZ_CODE}/src/version-info.txt && chown devonboot:solution /APP/batch\${BIZ_CODE}/src/app.jar && chown devonboot:solution /APP /batch\${BIZ_CODE}/src/version-info.txt" rm -rf * && rm -rf .git</pre>
		라인별 설명 1. 배치 서버에 업로드된 실행가능한 파일(jar) 의 권한을 설정(devonboot:solution 으로)하고 빌드된 버전 정보를 업데이트한다. 2. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.4. SPRING-ANT

단 계	스크립트
--------	------

1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE}* && rm -rf copy/usr/local/tomcat/webapps/ROOT/* git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done</pre>
		라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>pushd \${GITLAB_REPO_NAME} cp META-INF/context-\${BRANCH_NAME}.xml META-INF/context.xml ant cp -R build/* ../copy/usr/local/tomcat/webapps/ROOT/ popd podman build --no-cache --build-arg BRANCH_NAME=\${BRANCH_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre>
		라인별 설명 1. application 소스 빌드를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. 환경에 맞는 META-INF/context.xml 을 적용한다. 3. application 소스를 빌드한다. 4. 빌드된 파일들을 컨테이너 복사 대상인 파일 경로로 이동한다. 5. 1번에서 이동하기 전 디렉토리로 빠져나온다. 6. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}</pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

4	Deploy	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}:.*/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니페스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.5. SPRING-BOOT

	단계	스크립트
1	Initialize	<pre> def accountMap = ["pfg-sft": 23005 , "unt-pgw": 25011] </pre>
라인별 설명 1. OCP 인프라 관리자에게 협업을 통해 공유받은 POD 의 username 과 UID 값을 설정한다. (문자열 key = username, 숫자 value = UID 로 map 구조로 구성한다.)		
2	Prepare	<pre> rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE} </pre>
라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.		

3	Build	<pre> pushd \${GITLAB_REPO_NAME} sh gradlew build -Pprofile=\${BRANCH_NAME} --no-build-cache --refresh-dependencies mv build/libs/*-SNAPSHOT.war ../copy/APP/spring/app.war popd podman build --no-cache --build-arg ACCOUNT_NAME=\${SYSTEM_CODE}-\${BIZ_CODE} --build-arg ACCOUNT_ID='' + accountMap["\${SYSTEM_CODE}-\${BIZ_CODE}"] + '' -t \${NEXUS_IMAGE_UPLOAD_URL} /\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre> 라인별 설명 1. application 소스 빌드를 위해 다운로드받은 application 소스 프로젝트 디렉토리로 이동한다. 2. application 소스를 빌드한다. 3. 빌드된 실행가능한 파일(war) 을 컨테이너 복사 대상인 파일 경로로 이동한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다. 5. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
4	Release	<pre> podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} </pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
5	Deploy	<pre> pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\ /\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.* /image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\ /\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git </pre> 라인별 설명 1. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 2. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 3. git commit 에 사용할 사용자 email 정보를 저장한다. 4. git commit 에 사용할 사용자 name 정보를 저장한다. 5. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 6. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 7. gitlab 에 저장정보를 업로드한다. 8. 1번에서 이동하기 전 디렉토리로 빠져나온다. 9. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.6. API-GATEWAY

	단계	스크립트
--	----	------

1	Prepare	<pre>rm -rf \${GITLAB_REPO_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL} /\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for API_SPEC_NAME in \$(cat \${GITLAB_REPO_NAME}/\${API_SPEC_FILE}); do API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\r//") API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\n//") pushd \${GITLAB_REPO_NAME}/\${API_SPEC_NAME} zip -r \${API_SPEC_NAME}.\${API_SPEC_EXT} ./ mv \${API_SPEC_NAME}.\${API_SPEC_EXT} ../ popd done</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. API_SPEC_FILE 환경변수에 해당하는 파일 목록에 대해 반복문을 실행한다. 4. 파일 목록에 있는 windows 의 개행 문자 중 'wr' 를 제거한다. 5. 파일 목록에 있는 windows 의 개행 문자 중 'wn' 를 제거한다. 6. 배포할 API spec 을 압축하기 위해 해당 디렉토리로 이동한다. 7. zip 으로 압축한다. 8. 압축된 파일을 상위로 이동한다. 9. 6번에서 이동하기전, 디렉토리로 복귀한다. 10. 모든 파일에 대한 작업이 완료되었으면, 반복문을 종료한다.
2	Delivery	<pre>pushd \${GITLAB_REPO_NAME} for API_SPEC_NAME in \$(cat \${API_SPEC_FILE}); do API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\r//") API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\n//") scp \${API_SPEC_NAME}.\${API_SPEC_EXT} \${TARGET_ID}@\${TARGET_IP}:\${TARGET_PATH} done popd</pre> 라인별 설명 1. api spec 배포를 위해 배포할 api spec 압축 파일이 있는 디렉토리로 이동한다. 2. API_SPEC_FILE 환경변수에 해당하는 파일 목록에 대해 반복문을 실행한다. 3. 파일 목록에 있는 windows 의 개행 문자 중 'wr' 를 제거한다. 4. 파일 목록에 있는 windows 의 개행 문자 중 'wn' 를 제거한다. 5. 배포 대상인 압축파일을 api gateway 서버로 복사하여 업로드한다. 6. 배포 대상을 모두 배포하였으면 반복문을 종료한다. 7. 1번에서 이동하기전, 디렉토리로 복귀한다.
3	Import packages	<pre>for API_SPEC_NAME in \$(cat \${GITLAB_REPO_NAME}/\${API_SPEC_FILE}); do API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\r//") API_SPEC_NAME=\$(echo "\${API_SPEC_NAME}" sed "s/\\n//") curl http://\${TARGET_IP}\${UPDATE_API}?packageName=\${API_SPEC_NAME} done</pre> 라인별 설명 1. API_SPEC_FILE 환경변수에 해당하는 파일 목록에 대해 반복문을 실행한다. 2. 파일 목록에 있는 windows 의 개행 문자 중 'wr' 를 제거한다. 3. 파일 목록에 있는 windows 의 개행 문자 중 'wn' 를 제거한다. 4. api gateway 에서 제공하는 api spec 적용 기능을 호출하여 실제 반영한다. 5. 배포 대상을 모두 배포하였으면 반복문을 종료한다.

4	Clean all	<pre>rm -rf *</pre> 라인별 설명 1. 빌드하며 생성된 불필요한 파일을 모두 제거한다.
---	-----------	---

3.5.2.7. API-PORTAL

	단계	스크립트
1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>pushd \${GITLAB_REPO_NAME}/openworks-parent mvn -gs ../../conf/settings.xml clean install popd pushd \${GITLAB_REPO_NAME}/openworks-admin mv target/openworks-admin-1.0.war ../../copy/usr/local/tomcat/webapps/openworks-admin.war popd podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre> 라인별 설명 1. application 빌드를 위해 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 1번에서 이동하기전, 디렉토리로 복귀한다. 4. 빌드된 실행가능한 파일(war) 을 이동하기위해 디렉토리로 이동한다. 5. 빌드된 실행가능한 파일(war) 을 컨테이너 복사 대상인 파일 경로로 이동한다. 6. 4번에서 이동하기 전 디렉토리로 빠져나온다. 7. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.

3	Release	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Delivery	<pre>pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.* /image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git</pre> 라인별 설명 1. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 2. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 3. git commit 에 사용할 사용자 email 정보를 저장한다. 4. git commit 에 사용할 사용자 name 정보를 저장한다. 5. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 6. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 7. gitlab 에 저장정보를 업로드한다. 8. 1번에서 이동하기 전 디렉토리로 빠져나온다. 9. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.8. BLAZE-ADVISOR

	단계	스크립트
1	Initialize	<pre>def accountMap = ["unt-rbs": 25002, "unt-rbm": 25013]</pre> 라인별 설명 1. OCP 인프라 관리자에게 협업을 통해 공유받은 POD 의 username 과 UID 값을 설정한다. (문자열 key = username, 숫자 value = UID 로 map 구조로 구성한다.)

2	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} - * git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
3	Build	<pre>pushd \${GITLAB_REPO_NAME} ant popd pushd \${GITLAB_REPO_NAME}/trunk/WebContents tar zcf app.tar.gz * popd podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} --build-arg BRANCH_NAME=\${BRANCH_NAME} --build-arg ACCOUNT_NAME=\${SYSTEM_CODE}-\${BIZ_CODE} --build-arg ACCOUNT_ID='' + accountMap["\${SYSTEM_CODE}-\${BIZ_CODE}"] + '' -t \${NEXUS_IMAGE_UPLOAD_URL} /\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre> 라인별 설명 1. application 빌드를 위해 application 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 1번에서 이동하기전, 디렉토리로 복귀한다. 4. 빌드된 파일들로 압축 파일(tar.gz) 을 생성하기위해 디렉토리로 이동한다. 5. 빌드된 파일들을 압축 파일(tar.gz) 을 생성한다. 6. 4번에서 이동하기 전 디렉토리로 빠져나온다. 7. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
4	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

5	Deli very	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}.*:image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니페스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.9. MINERVA-HTML5VIEWER

	단 계	스크립트
1	Pre pare	<pre> rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE}* git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done </pre>
라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.		

2	Build	<pre> pushd \${GITLAB_REPO_NAME} ant cp -R build/* ../copy/usr/local/tomcat/webapps/ROOT/ popd podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre>
		라인별 설명 1. application 빌드를 위해 application 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 빌드된 파일들(build/*)을 컨테이너 복사 대상인 파일 경로로 이동한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다. 5. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Release	<pre> podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} </pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Delivery	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.*/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
		라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니팩스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.10. MINERVA-IMAGE-CONVERTER

	단계	스크립트
1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE}* && rm -rf copy/usr/local/tomcat/webapps/unt-cvt/* git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>pushd \${GITLAB_REPO_NAME} ant cp -R build/* ../copy/usr/local/tomcat/webapps/unt-cvt/ popd podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre> 라인별 설명 1. application 빌드를 위해 application 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 빌드된 파일들(build/*)을 컨테이너 복사 대상인 파일 경로로 이동한다. 4. 1번에서 이동하기 전 디렉토리로 빠져나온다. 5. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

4	Delivery	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}:.*image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}-\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니페스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.11. NEXS-FRAMEWORK

단 계	스크립트
1 Initia lize	<pre> def accountMap = ["unt-efs": 25003, "unt-efm": 25014] </pre>
라인별 설명 1. OCP 인프라 관리자에게 협업을 통해 공유받은 POD 의 username 과 UID 값을 설정한다. (문자열 key = username, 숫자 value = UID 로 map 구조로 구성한다.)	
2 Pre pare	<pre> rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} - * git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done </pre>
라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.	

3	Build	<pre> pushd \${GITLAB_REPO_NAME} ant popd pushd \${GITLAB_REPO_NAME}/trunk/WebContents tar zcf app.tar.gz * popd podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} --build-arg BRANCH_NAME=\${BRANCH_NAME} --build-arg ACCOUNT_NAME=\${SYSTEM_CODE}-\${BIZ_CODE} --build-arg ACCOUNT_ID='' + accountMap["\${SYSTEM_CODE}-\${BIZ_CODE}"] + '' -t \${NEXUS_IMAGE_UPLOAD_URL} /\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre>
		라인별 설명 1. application 빌드를 위해 application 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 1번에서 이동하기전, 디렉토리로 복귀한다. 4. 빌드된 파일들로 압축 파일(tar.gz) 을 생성하기위해 디렉토리로 이동한다. 5. 빌드된 파일들을 압축 파일(tar.gz) 을 생성한다. 6. 4번에서 이동하기 전 디렉토리로 빠져나온다. 7. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
4	Release	<pre> podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} </pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
5	Delivery	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\ /\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\ /\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
		라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니팩스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.12. OZ-SCHEDULER

	단 계	스크립트
1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}* \${GITLAB_REPO_NAME}_\${INT_EXT_TYPE} -\${CD_CODE} conf bin lib license git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP} /\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${INT_EXT_TYPE}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME} _\${INT_EXT_TYPE}-\${CD_CODE}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다. 4. (only dev) ArgoCD 가 바라보는 external 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} ./ podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _mark_any_\${BRANCH_NAME}:\${IMAGE_VERSION} -f Dockerfile.mark-any</pre> 라인별 설명 1. OZ 스케줄러 컨테이너 이미지를 생성한다. 2. Markany 컨테이너 이미지를 생성한다.
3	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _mark_any_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _mark_any_\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 OZ 스케줄러 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. 생성된 Markany 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 4. Jenkins 서버에 남아있는 OZ 스케줄러 컨테이너 이미지를 제거한다. 5. Jenkins 서버에 남아있는 Markany 컨테이너 이미지를 제거한다.

4	Deli very	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_mark_any_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_mark_any_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done pushd \${GITLAB_REPO_NAME}-\${INT_EXT_TYPE}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_mark_any_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_mark_any_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니패스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 OZ 스케줄러 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. deployment.yaml 에 기재되어있는 Markany 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 5. git commit 에 사용할 사용자 email 정보를 저장한다. 6. git commit 에 사용할 사용자 name 정보를 저장한다. 7. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 8. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 9. gitlab 에 저장정보를 업로드한다. 10. 2번에서 이동하기 전 디렉토리로 빠져나온다. 11. 배포할 모든 환경의 매니패스트 값을 변경 완료하면, 반복문을 종료한다. 12. 외부에서 접근가능한 external 서비스의 CD 매니패스트가 존재하는 디렉토리로 이동한다. 13. deployment.yaml 에 기재되어있는 OZ 스케줄러 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 14. deployment.yaml 에 기재되어있는 Markany 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 15. git commit 에 사용할 사용자 email 정보를 저장한다. 16. git commit 에 사용할 사용자 name 정보를 저장한다. 17. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 18. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 19. gitlab 에 저장정보를 업로드한다. 20. 11번에서 이동하기 전 디렉토리로 빠져나온다. 21. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.13. OZ-SERVER

단 계	스크립트
--------	------

1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} - * git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH}; done</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre> 라인별 설명 1. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

4	Delivery	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}-\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.*image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. git commit 에 사용할 사용자 email 정보를 저장한다. 5. git commit 에 사용할 사용자 name 정보를 저장한다. 6. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 7. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 8. gitlab 에 저장정보를 업로드한다. 9. 2번에서 이동하기 전 디렉토리로 빠져나온다. 10. 배포할 모든 환경의 매니페스트 값을 변경 완료하면, 반복문을 종료한다. 11. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.2.14. PX-CALCULATOR

	단계	스크립트
1	Prepare	<pre> rm -rf \${GITLAB_REPO_NAME} && rm -rf Calculator git clone https://:\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} </pre>
라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다.		
2	Upload files	<pre> pushd \${GITLAB_REPO_NAME} scp JavaPXCalculationDeployment_0_0@_0_1.pxdpz \${PX_CALCULATOR_ID}@\${PX_CALCULATOR_IP}:\${PX_CALCULATOR_DIR}/Deployment/ scp -r Calculator/Etc \${PX_CALCULATOR_ID}@\${PX_CALCULATOR_IP}:\${PX_CALCULATOR_DIR}/ popd </pre>
라인별 설명 1. CalcServer 파일 및 설정값 배포를 위해 디렉토리로 이동한다. 2. JavaPXCalculationDeployment_0_0@_0_1.pxdpz 파일을 CalcServer 에 복사하여 업로드한다. 3. Calculator/Etc 하위 설정 파일들을 CalcServer 에 복사하여 업로드한다. 4. 1번에서 이동하기 전 디렉토리로 복귀한다.		

3	Restart CalcServer	<pre>ssh \${PX_CALCULATOR_ID}@\${PX_CALCULATOR_IP} "rm -f \${PX_CALCULATOR_DIR}/Deployment/DefCache.dat" ssh \${PX_CALCULATOR_ID}@\${PX_CALCULATOR_IP} "\${PX_CALCULATOR_DIR}/script/restartServer.sh" & sleep 140s rm -rf * && rm -rf .git</pre> 라인별 설명 1. CalcServer 에 기존에 존재하는 DefCache 를 원격으로 제거한다. 2. CalcServer 에 존재하는 restartServer.sh 을 원격으로 실행한다. 3. 140초간 기다린다. 4. 빌드하며 생성된 불필요한 파일을 모두 제거한다.
---	-----------------------	--

3.5.2.15. SVN

	단계	스크립트
1	Checkout	<pre>if [! -d \${SVN_DIRECTORY}]; then svn checkout \${SVN_REPOSITORY}/\${SVN_DIRECTORY}; fi</pre> 라인별 설명 1. 기존 svn 프로젝트가 존재하지 않으면 svn 에서 application 소스를 다운로드한다.
2	Update	<pre>pushd \${SVN_DIRECTORY} svn update popd</pre> 라인별 설명 1. 소스 최신화를 위해 application 디렉토리로 이동한다. 2. svn 서버의 소스를 기반으로 소스를 다운로드 받아 최신화 한다. 3. 1번에서 이동하기 전 디렉토리로 복귀한다.
3	Compression	<pre>pushd \${SVN_DIRECTORY}/\${SOURCE_PATH} tar cf \${CD_FILE_NAME} * popd</pre> 라인별 설명 1. 소스 압축을 위해 application 소스 디렉토리로 이동한다. 2. 디렉토리에 존재하는 모든 파일을 tar 로 묶는다. 3. 1번에서 이동하기 전 디렉토리로 복귀한다.

4	Delivery	<pre>scp \${SVN_DIRECTORY}/\${SOURCE_PATH}/\${CD_FILE_NAME} \${SCT_SERVER_ID} @\${SCT_SERVER_IP}:\${TARGET_PATH}</pre> 라인별 설명 1. tar 로 묶은 파일을 배포 서버에 복사하여 업로드한다.
5	RPC - decompression	<pre>ssh \${SCT_SERVER_ID}@\${SCT_SERVER_IP} "cd \${TARGET_PATH} && tar xf \${CD_FILE_NAME} && rm \${CD_FILE_NAME}"</pre> 라인별 설명 1. 서버에 배포된 tar 를 해제하고 tar 파일을 제거하는 명령어를 원격으로 수행한다.

3.5.2.16. TOBE-SFA

	단 계	스크립트
1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP} /\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>pushd \${GITLAB_REPO_NAME} rm -rf ./package-lock.json npm run install:\${BRANCH_NAME} --no-audit --verbose npm run build:\${BRANCH_NAME} popd podman build --no-cache --build-arg JOB_NAME=\${JOB_NAME} --build-arg SYSTEM_CODE=\${SYSTEM_CODE} -- build-arg BIZ_CODE=\${BIZ_CODE} --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} --build-arg BRANCH_NAME=\${BRANCH_NAME} --build-arg ACCOUNT_NAME=\${SYSTEM_CODE}-\${BIZ_CODE} --build-arg ACCOUNT_ID='' + accountMap["\${SYSTEM_CODE}-\${BIZ_CODE}"] + '' -t \${NEXUS_IMAGE_UPLOAD_URL} /\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} -f Dockerfile.\${GITLAB_REPO_NAME}</pre> 라인별 설명 1. application 빌드를 위해 디렉토리로 이동한다. 2. package-lock.json 을 제거한다. 3. node_modules 를 설치한다. 4. 배포할 파일을 생성한다. 5. 1번에서 이동하기전, 디렉토리로 복귀한다. 6. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.

3	Release	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_ _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_ _\${BRANCH_NAME}:\${IMAGE_VERSION}</pre> 라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Delivery	<pre>pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\ /\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.*/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\ /\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git</pre> 라인별 설명 1. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 2. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 3. git commit 에 사용할 사용자 email 정보를 저장한다. 4. git commit 에 사용할 사용자 name 정보를 저장한다. 5. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 6. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 7. gitlab 에 저장정보를 업로드한다. 8. 1번에서 이동하기 전 디렉토리로 빠져나온다. 9. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.17. U-COVERAGE

	단계	스크립트
1	Prepare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP} /\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니팩스트를 gitlab 으로부터 다운로드한다.

2	Build	<pre> pushd \${GITLAB_REPO_NAME} mv orcaboot-batch-0.0.1-SNAPSHOT.jar ../copy/APP/reverse/app.jar mv Coverage.war ../copy/APP/ucoverage/app.war popd podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} ./ </pre>
		라인별 설명 - 컨테이너 이미지에 파일을 복사하기 위해 디렉토리를 이동한다. - orcaboot jar 파일을 컨테이너 이미지 복사 디렉토리로 이동한다. - Coverage war 파일을 컨테이너 이미지 복사 디렉토리로 이동한다. 1번에서 이동하기 전 디렉토리로 복귀한다. - Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Release	<pre> podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION} </pre>
		라인별 설명 - 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. - 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. - Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Delivery	<pre> pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git </pre>
		라인별 설명 - 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. - deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. - git commit 에 사용할 사용자 email 정보를 저장한다. - git commit 에 사용할 사용자 name 정보를 저장한다. - git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. - Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. - gitlab 에 저장정보를 업로드한다. 1번에서 이동하기 전 디렉토리로 빠져나온다. - 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.18. VANILLA-HTML

	단계	스크립트
--	----	------

1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME}-\${CD_CODE}</pre>
		라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>podman build --no-cache --build-arg GITLAB_REPO_NAME=\${GITLAB_REPO_NAME} -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} ./</pre>
		라인별 설명 1. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.
3	Rel ease	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} _\${BRANCH_NAME}:\${IMAGE_VERSION}</pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.
4	Deli very	<pre>pushd \${GITLAB_REPO_NAME}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:.*/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git</pre>
		라인별 설명 1. 배포할 환경의 CD 매니페스트가 존재하는 디렉토리로 이동한다. 2. deployment.yaml 에 기재되어있는 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 3. git commit 에 사용할 사용자 email 정보를 저장한다. 4. git commit 에 사용할 사용자 name 정보를 저장한다. 5. git 으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 6. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 7. gitlab 에 저장정보를 업로드한다. 8. 1번에서 이동하기 전 디렉토리로 빠져나온다. 9. 빌드하며 생성된 불필요한 파일을 모두 제거한다.

3.5.2.19. ZIPI-T

	단계	스크립트
1	Pre pare	<pre>rm -rf \${GITLAB_REPO_NAME} && rm -rf \${GITLAB_REPO_NAME}_\${CD_CODE}* && rm -rf \${GITLAB_REPO_NAME} _\${INT_EXT_TYPE}-\${CD_CODE} git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_APPLICATION_GROUP} /\${GITLAB_REPO_NAME}.git -b \${BRANCH_NAME} for TARGET_BRANCH in \${TARGET_BRANCHES}; do git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}.git -b \${TARGET_BRANCH} \${GITLAB_REPO_NAME}_\${CD_CODE}-\${TARGET_BRANCH}; done git clone https://\${GITLAB_CREDS_USR}:\${GITLAB_CREDS_PSW}@\${GITLAB_URL}/\${GITLAB_CD_GROUP} /\${SYSTEM_CODE}/\${GITLAB_REPO_NAME}_\${INT_EXT_TYPE}.git -b \${BRANCH_NAME} \${GITLAB_REPO_NAME} _\${INT_EXT_TYPE}-\${CD_CODE}</pre> 라인별 설명 1. 이전 빌드에 의해 생성된 workspace 의 잔여물을 제거한다. 2. 배포할 서비스의 application 소스를 gitlab 으로부터 다운로드한다. 3. ArgoCD 가 바라보는 서비스의 매니페스트를 gitlab 으로부터 다운로드한다.
2	Build	<pre>podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}_zipbae_\${BRANCH_NAME}:\${IMAGE_VERSION} -f Dockerfile.zipbae podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}_zipit_\${BRANCH_NAME}:\${IMAGE_VERSION} -f Dockerfile.zipit podman build --no-cache -t \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE} /\${GITLAB_REPO_NAME}_srchsrv_\${BRANCH_NAME}:\${IMAGE_VERSION} -f Dockerfile.srchsrv</pre> 라인별 설명 1. application 빌드를 위해 application 디렉토리로 이동한다. 2. application 빌드를 한다. 3. 1번에서 이동하기전, 디렉토리로 복귀한다. 4. 빌드된 파일들로 압축 파일(tar.gz) 을 생성하기위해 디렉토리로 이동한다. 5. 빌드된 파일들을 압축 파일(tar.gz) 을 생성한다. 6. 4번에서 이동하기 전 디렉토리로 빠져나온다. 7. Dockerfile 을 이용하여 PaaS 에 배포할 컨테이너 이미지를 생성한다.

3	Release	<pre>podman login -u \${NEXUS_JENKINS_ID} -p \${NEXUS_JENKINS_TOKEN} \${NEXUS_IMAGE_UPLOAD_URL} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} zipbae_\${BRANCH_NAME}:\${IMAGE_VERSION} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} zipit_\${BRANCH_NAME}:\${IMAGE_VERSION} podman push \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} srchsrv_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} zipbae_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} zipit_\${BRANCH_NAME}:\${IMAGE_VERSION} podman rmi -f \${NEXUS_IMAGE_UPLOAD_URL}/\${PROJECT_CODE}/\${SYSTEM_CODE}/\${GITLAB_REPO_NAME} srchsrv_\${BRANCH_NAME}:\${IMAGE_VERSION}</pre>
		라인별 설명 1. 넥서스 이미지 레지스트리에 컨테이너 이미지를 업로드 가능한 권한이 있는 계정으로 로그인한다. 2. 생성된 컨테이너 이미지를 넥서스 이미지 레지스트리에 업로드한다. 3. Jenkins 서버에 남아있는 컨테이너 이미지를 제거한다.

4	Deli very	<pre> for TARGET_BRANCH in \${TARGET_BRANCHES}; do pushd \${GITLAB_REPO_NAME}_\${CD_CODE}-\${TARGET_BRANCH} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipbae_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipbae_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipit_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipit_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_srchsrv_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_srchsrv_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd done pushd \${GITLAB_REPO_NAME}_\${INT_EXT_TYPE}-\${CD_CODE} sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipbae_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipbae_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipit_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_zipit_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml sed -i "s/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_srchsrv_\${BRANCH_NAME}:/image: \${NEXUS_IMAGE_DOWNLOAD_URL}\\/\${PROJECT_CODE}\\/\${SYSTEM_CODE}\\/\${GITLAB_REPO_NAME}_srchsrv_\${BRANCH_NAME}:\${IMAGE_VERSION}/g" deployment.yaml git config user.email 'jenkins@kblife.co.kr' git config user.name 'Jenkins System' git add deployment.yaml git commit -m "deploy \${GITLAB_REPO_NAME}_\${BRANCH_NAME}:\${IMAGE_VERSION}" git push popd rm -rf * && rm -rf .git </pre>
라인별 설명 1. 환경별로 반복문을 수행한다. 2. 배포할 환경의 CD 매니팩스트가 존재하는 디렉토리로 이동한다. 3. deployment.yaml에 기재되어있는 zipbae 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 4. deployment.yaml에 기재되어있는 zipit 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 5. deployment.yaml에 기재되어있는 srchsrv 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 6. git commit에 사용할 사용자 email 정보를 저장한다. 7. git commit에 사용할 사용자 name 정보를 저장한다. 8. git으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 9. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 10. gitlab에 저장정보를 업로드한다. 11. 2번에서 이동하기 전 디렉토리로 빠져나온다. 12. 배포할 모든 환경의 매니팩스트 값을 변경 완료하면, 반복문을 종료한다. 13. 배포할 환경의 external CD 매니팩스트가 존재하는 디렉토리로 이동한다. 14. deployment.yaml에 기재되어있는 zipbae 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 15. deployment.yaml에 기재되어있는 zipit 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 16. deployment.yaml에 기재되어있는 srchsrv 이미지 버전을 현재 새로 생성된 컨테이너 이미지의 버전으로 변경한다. 17. git commit에 사용할 사용자 email 정보를 저장한다. 18. git commit에 사용할 사용자 name 정보를 저장한다. 19. git으로 업로드할 파일 대상에 버전을 수정한 deployment.yaml 파일을 추가한다. 20. Jenkins 서버 내의 git 리파지토리에 git 저장정보를 저장한다. 21. gitlab에 저장정보를 업로드한다. 22. 13번에서 이동하기 전 디렉토리로 빠져나온다. 23. 빌드하며 생성된 불필요한 파일을 모두 제거한다.		

3.5.3. ArgoCD를 통한 배포 결과 확인

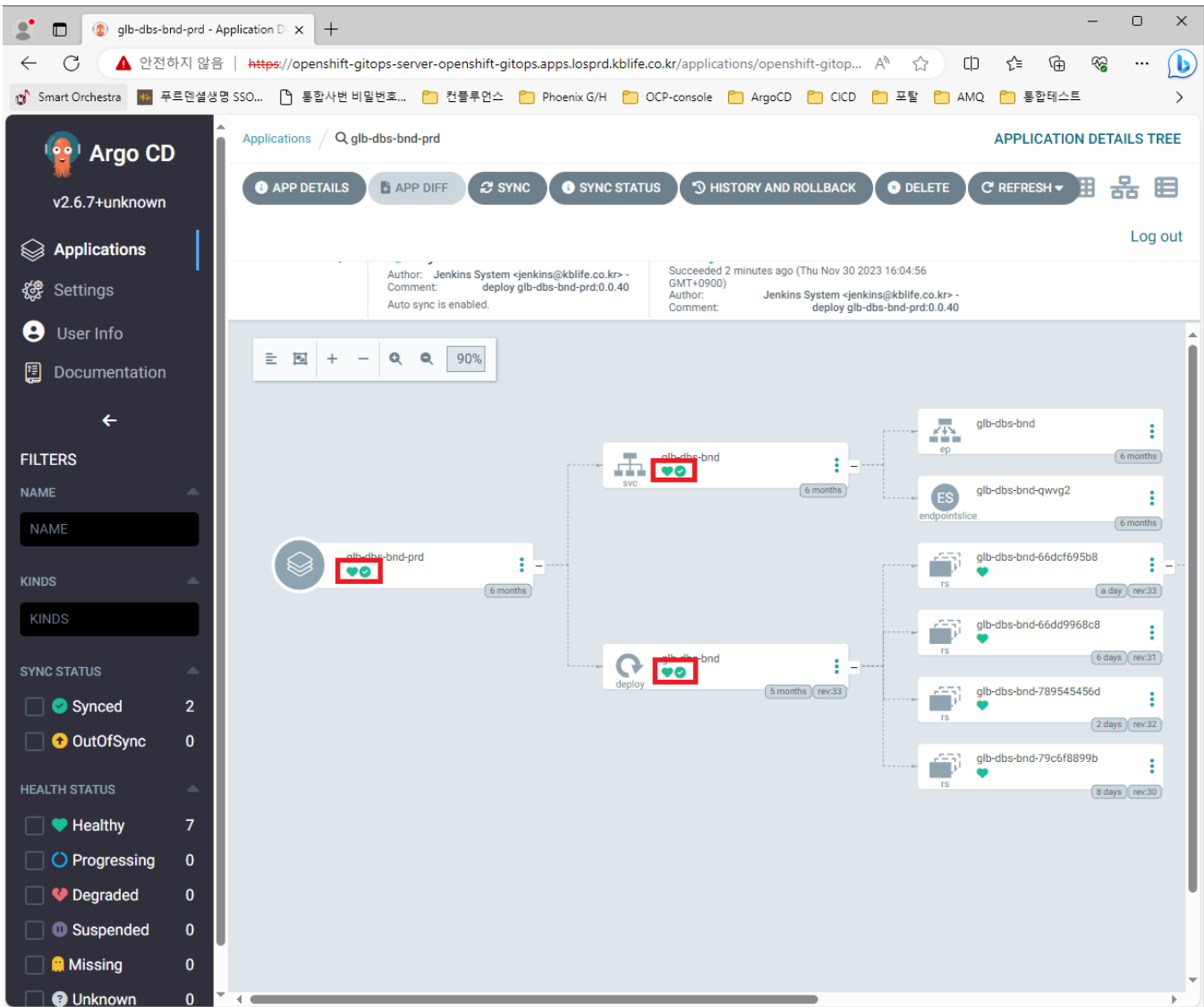
Applications > 수동 배포할 서비스 검색 / 클릭 > 상태 확인

The screenshot shows the Argo CD web interface. On the left is a dark sidebar with navigation options: 'Applications' (highlighted with a red box), 'Settings', 'User Info', and 'Documentation'. Below these are 'FILTERS' and status counts for 'SYNC STATUS' (Unknown: 1, Synced: 56, OutOfSync: 0) and 'HEALTH STATUS' (Unknown: 0, Progressing: 0, Suspended: 0, Healthy: 57, Degraded: 0, Missing: 0). At the bottom of the sidebar is a 'LABELS' section.

The main content area is titled 'Applications'. At the top, there are buttons for '+ NEW APP', 'SYNC APPS', and 'REFRESH APPS'. A search bar contains the text 'dbs' (highlighted with a red box). To the right of the search bar are icons for a list view, a grid view, and a 'Log out' button. Below the search bar, a list of applications is shown. One application, 'glb-dbs-bnd-prd', is highlighted with a red box. Its details are as follows:

- Project:** default
- Labels:**
- Status:** Healthy (green heart icon) Synced (green checkmark icon)
- Repo:** https://dlogitlab.kblife.co.kr/l...
- Target:** prd
- Path:** .
- Destination:** in-cluster
- Name:** los-glb-prd
- Created:** 05/19/2023 09:50:10 (6 months ago)

Below the application details are three buttons: 'SYNC', a refresh icon, and a delete icon.



3.6. 운영간 주의사항

3.6.1. 주기적 SSL 인증서 교체

개발망과 운영망 Jenkins 에서 사용중인 SSL 인증서는 *.kblife.co.kr 인증서로 매년 갱신되는 *.kblife.co.kr 인증서를 적용해주어야 서비스에 정상 접근이 가능하다.

인증서가 새로 갱신되면 갱신된 인증서를 개발망과 운영망 모두 /DATA/ssl/credential/Wildcard.kblife.co.kr_jks.jks 로 갱신하여야 한다.

3.6.2. 반복적인 다수의 job 빌드 timeout 실패 발생시

모든 job 은 빌드 시간이 15분으로 설정되어있다.

평균 front-end 는 1분 내, back-end 는 2분 내, solution 은 5분 내 빌드가 성공하지만 서버에 빌드가 물린 경우 더 늦어질 수 있지만, 정상적인 경우라면 15분 내에 모두 빌드가 완료되어야한다.

하지만 특정 시점 이후로 job 들의 빌드 시간이 15분이 지나 timeout 이 다량으로 발생하는 경우 jenkins vm 에 관리되는 dummy container image 를 확인해야한다.

dummy container image 는 container image 를 조회, 생성, 삭제할 때 모두 지연 시간을 발생시키므로 container image 를 다루는데서 병목이 발생할 가능성이 있다.

따라서 dummy container image 를 정리하는 job 이 정상적으로 수행되고 있는지 확인하고, 비정상 동작중이라면 jenkins 가 바쁘지 않은 시간에 아래 job 을 수행한다.

psrNo 는 수동 배포이기에 default 로 설정된 값으로 실행하면 된다.

1. 개발 Jenkins job: [95. CLEAN_DUMMY_CONTAINER_IMAGE \[Jenkins\] \(kblife.co.kr\)](#)
2. 운영 Jenkins job: [97. CLEAN_DUMMY_CONTAINER_IMAGE \[Jenkins\] \(kblife.co.kr\)](#)

3.6.3. Jenkins 로그인 안된다는 유저 발생시

Jenkins 에서는 사용자 계정을 관리하지 않고 LDAP 에 위임한다.

따라서 Jenkins 계정이 로그인 되지않는다면, LDAP 의 계정 패스워드 만료로 로그인이 안되는 상황으로 [통합사번 비밀번호 초기화 \(kblife.co.kr\)](#) 을 통해 계정을 초기화 할 수 있도록 가이드한다.

3.6.4. Jenkins build 권한이 없다고 하는 경우

Jenkins 에서 해당 계정에 권한이 정상적으로 부여되어있는지 확인한다.

권한이 정상적으로 부여되어있는 경우, 로그인할 때 ID를 소문자로 입력하고 로그인하도록 가이드한다.

Jenkins 의 권한 관리 플러그인은 대소문자를 구분하여 jenkins 는 소문자를 표준으로 한다.

4. 장애 관리

4.1. 장애 케이스 별 조치방안

4.1.1. Nexus disk full

Nexus 장애 조치는 SFA 프로젝트(Track#1) 관리 방법을 준용한다.

디스크 용량이 가득 찬 경우 nexus 웹에 접속되지 않아 디스크 관리가 안되는 경우의 조치방법을 기술한다.

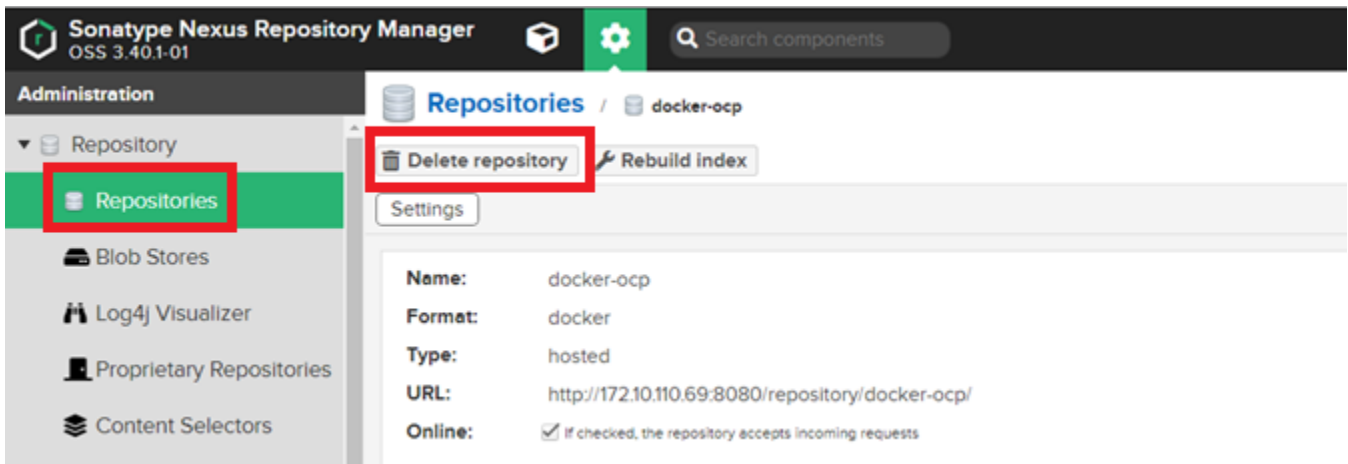
반드시 아래 4.1.1.1 부터 4.1.1.6 까지 순서대로 시행하여야 정상적으로 서비스 이용이 가능하다.

4.1.1.1. Blob 디렉토리 삭제

```
[root@pv-sfa-icm-01 ~]# cd /DATA/sfa/nexus/sonatype-work/nexus3/blobs/
[root@pv-sfa-icm-01 blobs]# ll
total 0
drwxrwxr-x. 4 nexus nexus 209 Aug 23 14:31 default
drwxrwxr-x. 4 nexus nexus 209 Sep 21 10:58 docker-aks
drwxrwxr-x. 4 nexus nexus 209 Dec 12 10:29 docker-base
drwxrwxr-x. 4 nexus nexus 209 Sep 21 10:47 docker-hosted
drwxrwxr-x. 4 nexus nexus 209 Jan 5 02:30 docker-ocp
drwxrwxr-x. 4 nexus nexus 209 Dec 26 13:17 maven_3rdparty
drwxrwxr-x. 4 nexus nexus 209 Aug 23 15:25 maven_proxy
drwxrwxr-x. 3 nexus nexus 116 Nov 28 11:03 npm_hosted
drwxrwxr-x. 4 nexus nexus 209 Dec 27 15:52 npm_proxy
[root@pv-sfa-icm-01 blobs]# rm -r -f docker-ocp/
```

4.1.1.2. Repository 삭제

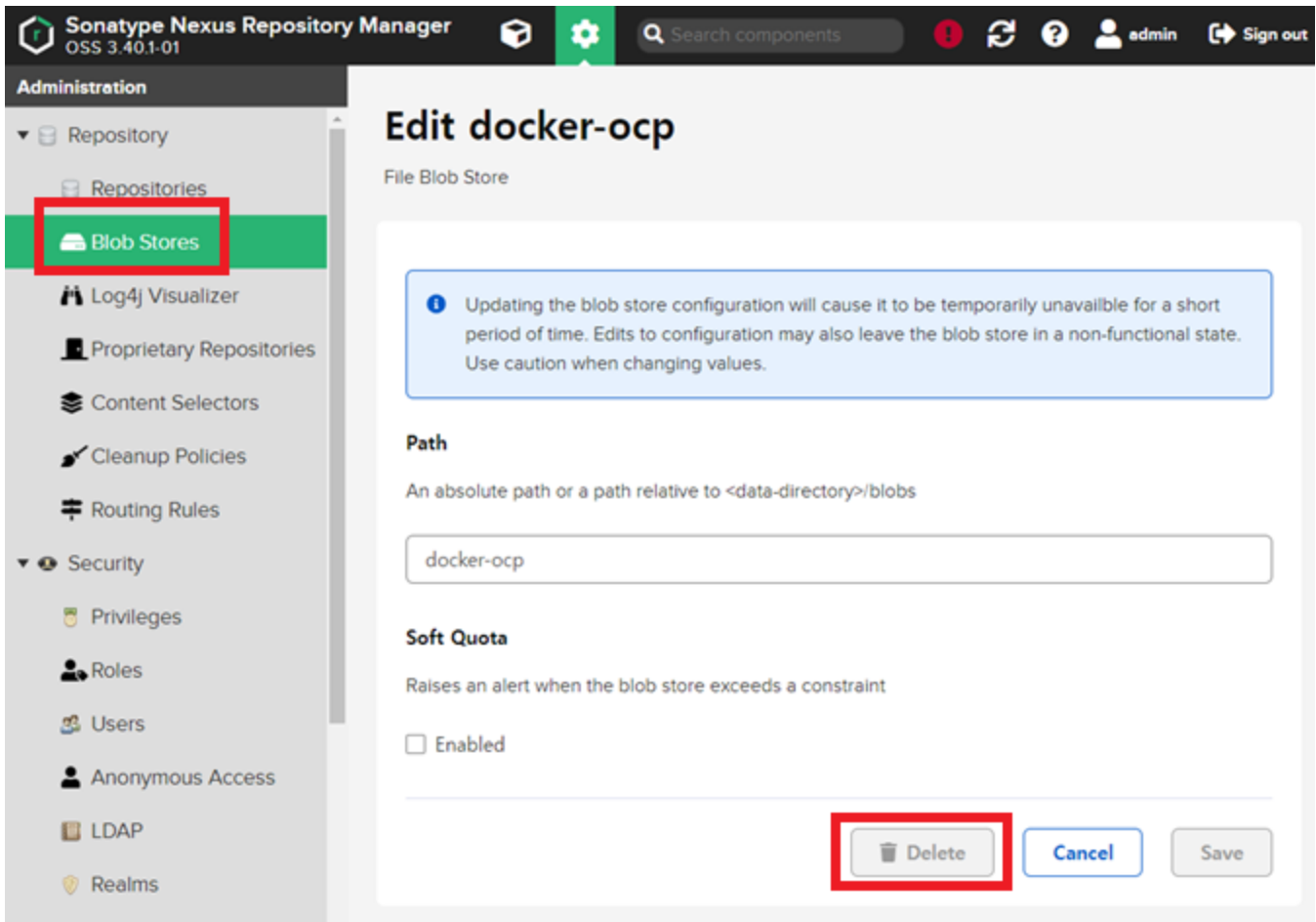
nexus 로그인 > 설정 > repository > repositories 메뉴 > docker-ocp 삭제



4.1.1.3. Blob 삭제

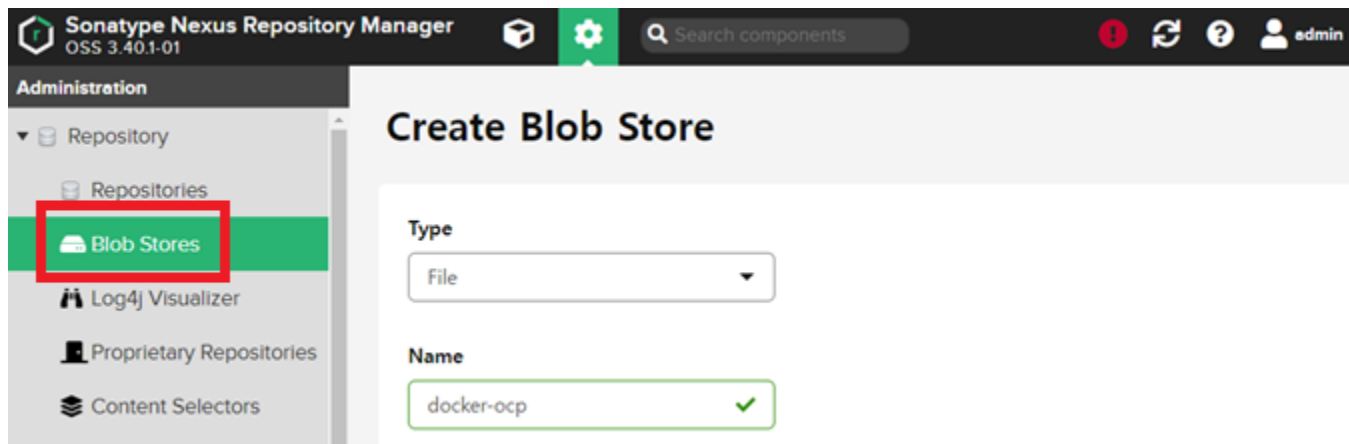
nexus 로그인 > 설정 > repository > Blob Stores메뉴 > docker-ocp 삭제

- 연결된 repository가 없으면 delete 버튼이 활성화 됨



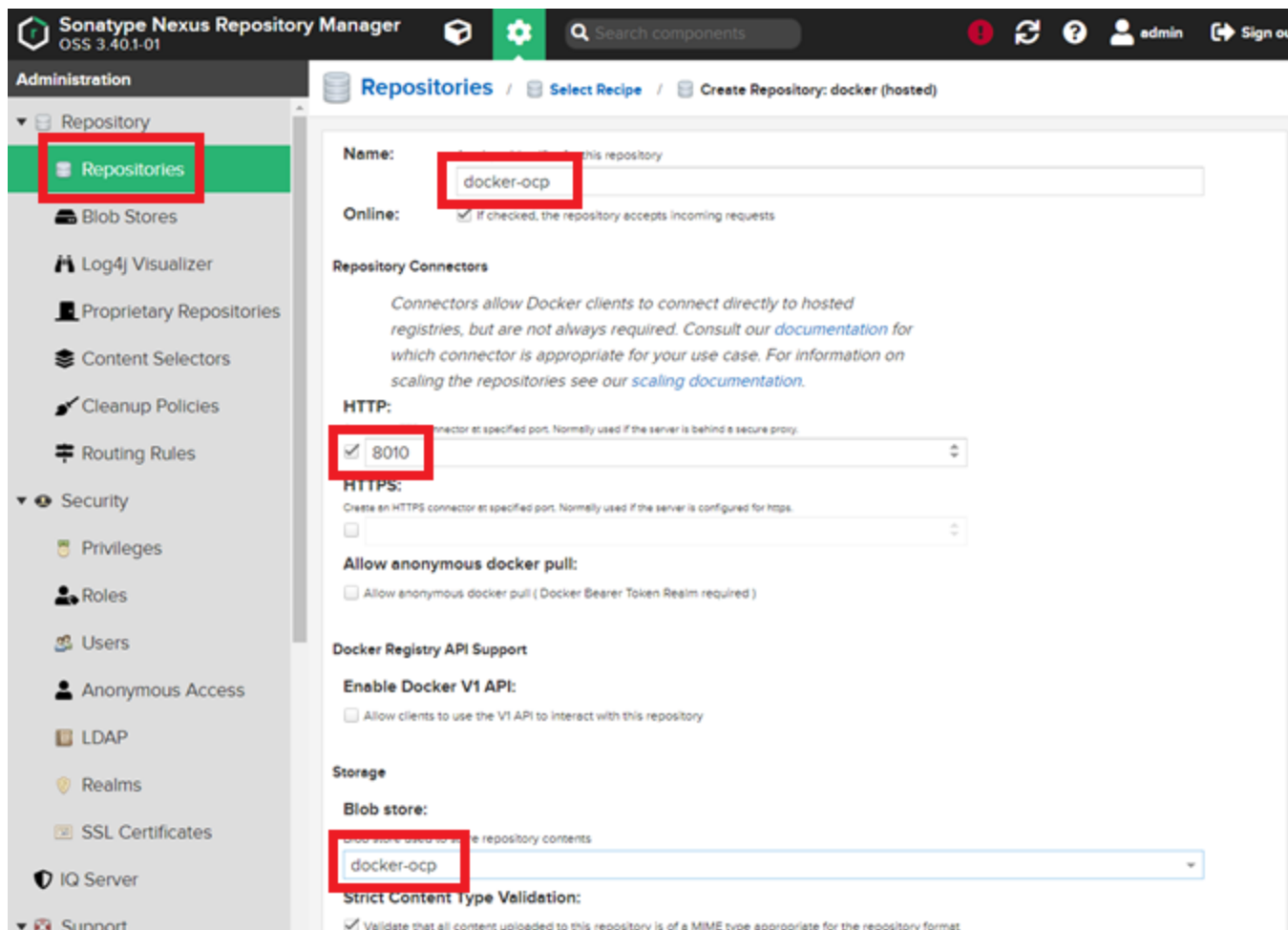
4.1.1.4. Blob 생성

nexus 로그인 > 설정 > repository > Blob Stores메뉴 > Create Blob Store 클릭



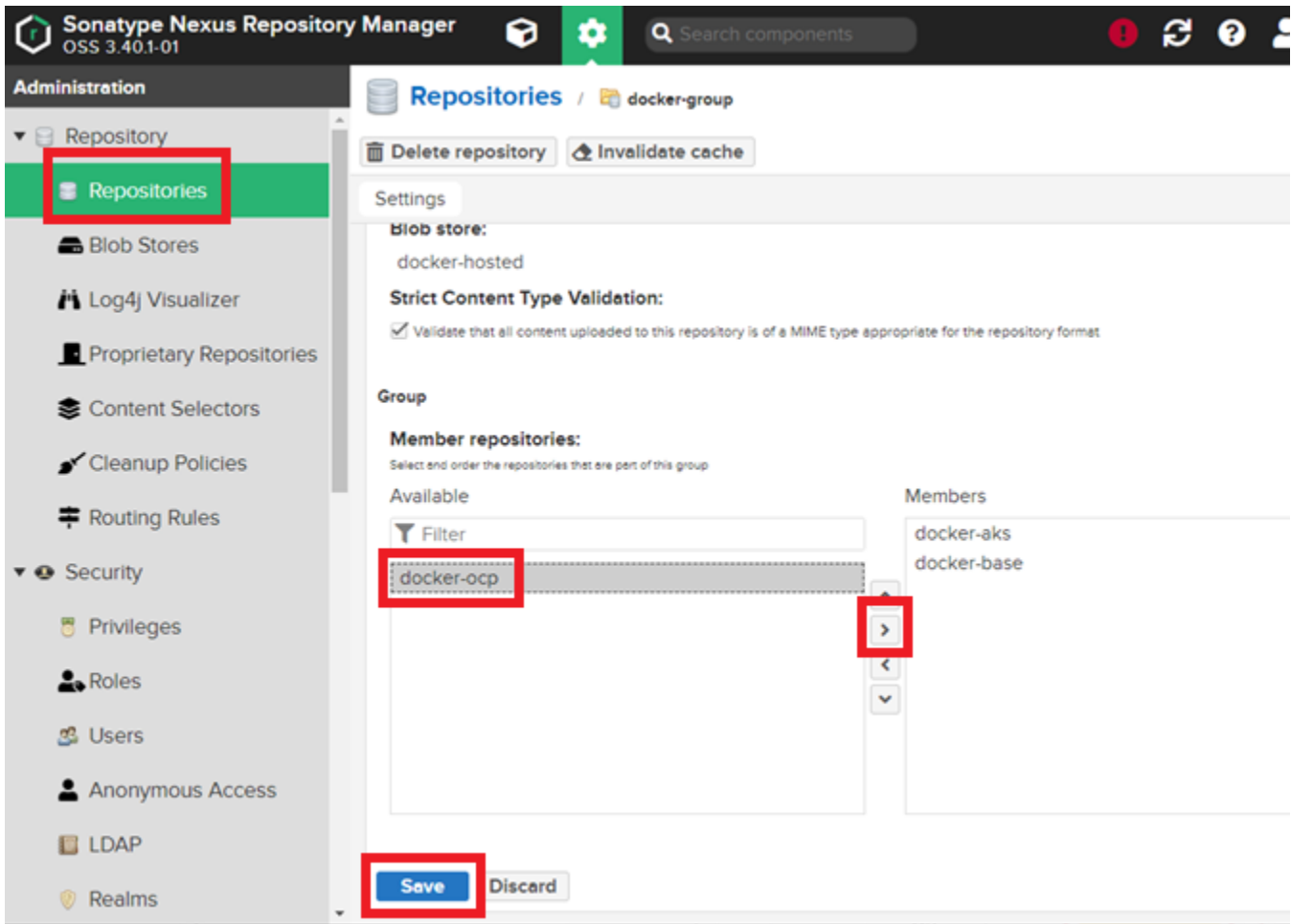
4.1.1.5. Repository 생성

nexus 로그인 > 설정 > repository > repositories > Create repository클릭 > docker(hosted) 선택



4.1.1.6. Docker-group 에 docker-ocp 포함

nexus 로그인 > 설정 > repository > repositories > docker-group클릭 > 하단 Member repositories 수정



4.1.2. Jenkins Job failure 유형 별 조치 방안

4.1.2.1. Fatal: Remote branch xxx not found in upstream origin 유형

PRD Job에서 이 오류 메시지가 발생하는 경우, psr번호에 해당하는 Tag가 없는 경우이다.

먼저 psr 번호로 tag를 생성한 후 job을 호출해야 한다.

Stage Logs (Build)

Print Message -- Build stage start (self time 18ms)

Verify if file exists in workspace -- step1 (self time 27ms)

Shell Script -- git clone -b 210891-117 http://devops:\${GIT_PASSWORD}@loggitlab.kblife.co.kr/group_sfa_bnd/sfa_bnd.git step1 --depth=1 (self time 538ms)

```

15:50:16 Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
15:50:16     Affected argument(s) used the following variable(s): [GIT_PASSWORD]
15:50:16     See https://jenkins.io/redirect/groovy-string-interpolation for details.
15:50:17 + git clone -b 210891-117 http://devops:****@loggitlab.kblife.co.kr/group_sfa_bnd/sfa_bnd.git step1 --depth=1
15:50:17 # clones repo "sfa_bnd"
15:50:17 warning: Could not find remote branch 210891-117 to clone.
15:50:17 fatal: Remote branch 210891-117 not found in upstream origin

```

Pipeline 삭제

Full Stage View

Rename

Embeddable Build Status

Pipeline Syntax

Build History

Filter builds...

212811-3-32

Stage View

Average stage times:

Average full run time: ~6min 56s

	Prepare	Build	Containerize	Operate	Declarative: Post Actions
210891-117-24	110ms	1min 53s	4min 0s	2s	16s
Jan 05 15:50	104ms	820ms	40ms	32ms	404ms
		failed	failed	failed	
210891-110-24	109ms	2min 10s	4min 4s	2s	10s
Jan 04 19:11					

별첨 - CI/CD 관련 서버

도구명	주소	비고
Jenkins	plosjenkins.kblife.co.kr	PRD 빌드
Jenkins	dlosjenkins.kblife.co.kr	DEV, UAT 빌드
Gitlab	plogitlab.kblife.co.kr	
ArgoCD	openshift-gitops-server-openshift-gitops.apps.losprd.kblife.co.kr	PRD 배포
ArgoCD	openshift-gitops-server-openshift-gitops.apps.losdev.kblife.co.kr	DEV, UAT 배포
Nexus	plosnexus.kblife.co.kr	PRD 리파지토리
Nexus	dlosnexus.kblife.co.kr	DEV, UAT 리파지토리